Resumen

Los sistemas de inspección de superficies especulares están formados principalmente por dos elementos: una o más cámaras industriales y una fuente de iluminación estructurada. Se basan en captar el reflejo de la iluminación sobre la superficie bajo inspección para detectar posibles irregularidades del brillo especular, provocadas por alteraciones en la geometría (golpes, acumulaciones de pintura...) o en el acabado de la superficie (arañazos). El objetivo de este TFM es implementar un sistema para simular este proceso de manera interactiva, utilizando para ello la funcionalidad de trazado de rayos por hardware disponible en las GPU más recientes. Se implementará un sistema que, dada una descripción de la superficie bajo inspección, la geometría de las fuentes de iluminación y la definición de la cámara, calcule una imagen de lo que vería la cámara, centrándose principalmente en la detección del reflejo especular.

Palabras clave: Trazado de rayos, Inspección de superficies, Reflejo especular

Abstract

Specular Surface inspection sistemas are mainly composed of 2 elements: one or more industrial cameras and a structured lighting. These are based on capturing the reflection on the inspectable surfaces for detecting any specular reflection irregularities such as geometry alterations (bruises, overpainting) or in the surface finish (scratches). The objective of this thesis is to implement a system to simulate this process in an interactive way using hardware-based Raytracing available in the newer GPUs. A system will be implemented which, given a definition of a inspectable Surface, the geometry of the lighting and a camera definition, will calculate that which the camera would see, mainly focused on the specular reflection detection.

Key words: RayTracing, Surface inspection, Specular reflection

Índice general

Índice general v

Índice general vii
Índice de figuras vii
1 Introducción 1
1.1 Motivación . . . 1
1.2 Objetivos . . . 1
1.3 Estructura de la memoria . . . 2
2 Estado del arte 3
2.1 Detección de defectos . . . 3
2.1.1 Métodos basados en inspección humana . . . 3
2.1.2 Adquisición de imágenes . . . 4
2.1.3 Detección mediante binarización . . . 4
2.1.4 Detección mediante reflexión difusa . . . 5
2.1.5 Detección mediante *deep learning* . . . 6
2.2 Trazado de rayos . . . 8
2.2.1 Trazado de rayos por hardware . . . 9
3 Solución propuesta 11
3.1 Plan de trabajo . . . 11
3.2 Conocimiento previo . . . 13
3.3 Arquitectura del Sistema . . . 19
3.4 Diseño detallado . . . 21
3.4.1 Núcleo *Vulkan* . . . 21
3.4.2 Núcleo de trazado de rayos . . . 25
3.4.3 Renderer . . . 32
3.5 Tecnología utilizada . . . 35
4 Desarrollo de la solución propuesta 37
4.1 Núcleo *Vulkan* . . . 37
4.2 Núcleo de trazado de rayos . . . 46
4.3 Renderer . . . 61
5 Implantación 67
5.1 Diseño experimental . . . 67
5.1.1 Pruebas de corrección . . . 68
5.1.2 Pruebas de comparación con sistema existente . . . 69
5.2 Pruebas . . . 71
5.3 Resultados . . . 78
6 Conclusiones 83
6.1 Relación con los estudios cursados . . . 84
6.2 Trabajo Futuro . . . 84

Apéndice
A Objetivos de desarrollo sostenible 87

Índice general

Índice de figuras

2.1 Túnel de inspección manual. 4
2.2 Proceso de binarización [7] 5
2.3 Imagen extraída de [6], Defectos detectados y clasificados por SVM 6
2.4 Modelado de la reflexión de la luz [1] 6
2.5 Resultados de diferentes métodos en tareas de detección de objetos pequeños [9] 7
2.6 Resultados de diferentes modelos no supervisados en tareas de detección de objetos pequeños [9] 7
2.7 Diagrama explicativo del trazado de rayos [4] 8
2.8 Tubería de trazado de rayos [4] 9

3.1 Gráfica extraída de [3] mostrando la jerarquía *Vulkan* 14
3.2 Gráfico explicativo del funcionamiento de los *Renderpass* y *Framebuffer* . . 15
3.3 Gráfico explicativo de los *TLAS* y *BLAS*, extraído de [8] 16
3.4 Gráfico extraído de [3], mostrando una disposición de *descriptor sets* 17
3.5 Gráfico extraído de [8], mostrando el flujo de los shaders 18
3.6 Gráfico extraído de [8], mostrando un ejemplo de *Shader binding table* . . . 18
3.7 Diagrama de la arquitectura del sistema 19
3.8 Diagrama de la organización del github 20
3.9 Diagrama del renderizado a textura y un ejemplo de su uso 22
3.10 Diagrama del sistema de creación de búferes, mediante copia de búfer a búfer 23
3.11 Diagrama que muestra la relación entre los shaders en GLSL con *Vulkan* . 24
3.12 Diagrama que muestra el ciclo de vida de las mallas en la aplicación . . . 25
3.13 Diagrama que muestra el ciclo de vida de las matrices de transformación en la aplicación 26
3.14 Diagrama que muestra la disposición de los *descriptor sets* en la *pipeline* . . 28
3.15 Diagrama que muestra los diferentes tipos de trazado de rayos según la recursión y el punto desde que se trazan 28
3.16 Diagrama que muestra la *Shader binding table* que se utilizará para el trazado de rayos 29
3.17 Diagrama que muestra el funcionamiento teórico del shader *raygen* 31
3.18 Ejemplo de ejecución del trazado de rayos, donde se determinó que el shader de miss devolviera un color rojo 31

4.1 Ejemplo de un error mostrado en la consola 38

4.2 *Physical device* seleccionado para el desarrollo del trabajo 38
4.3 Esquema representativo del proceso de creación de búferes. 41
4.4 Esquema representativo del ciclo de vida de un *Command Buffer* de Especificación Vulkan. 43
4.5 Esquema representativo del contenido de una malla. 44
4.6 Esquema representativo del proceso de creación de un shader. 45
4.7 Captura de la aplicación de pruebas, mostrando un par de mallas dibujadas con una textura aplicada. 45
4.8 Diagrama de la estructura final de la *Pipeline*. 52
4.9 Diagrama de la estructura final de la *Pipeline*. 53
4.10 Color utilizado en el shader de *Miss*. 56
4.11 Captura del resultado de colorear de un color rojo aquellos puntos donde se ejecuta el shader *Closest Hit*. 56
4.12 Captura mostrando de un color cada malla en función de su índice. 57
4.13 Esquema explicando el funcionamiento de coordenadas baricéntricas, extraído de Tutorial de trazado de rayos de Nvidia. 57
4.14 Diagrama explicando qué tipo de reflexión se permitirá en la ejecución del shader. 58
4.15 Captura mostrando un ejemplo de ejecución donde se colorea de diferentes colores en función de la cantidad de rebotes que se realizan a partir de cada punto. 59
4.16 Ejemplo de oclusiones en el sistema. 60
4.17 Diagrama mostrando la funcionalidad principal de dibujar mallas. 62

5.1 Capturas de los diferentes coches utilizados en las pruebas. 68
5.2 Captura mostrando la disposición de la banda frente al coche. 69
5.3 Captura mostrando la disposición de las diferentes luces en el espacio. . . 70
5.4 Diagrama mostrando el resultado esperado para ambos simuladores del sistema. 70
5.5 Capturas desde diferentes ángulos del coche de la prueba. 71
5.6 Capturas desde diferentes ángulos del coche de la prueba. 72
5.7 Capturas desde diferentes ángulos del coche de la prueba. 73
5.8 Capturas desde diferentes ángulos del coche de la prueba. 74
5.9 Capturas desde diferentes ángulos del coche de la prueba. 75
5.10 Captura tomada para ambos sistemas de la prueba, a la izquierda *OpenGL* y a la derecha *Vulkan*. 76
5.11 Captura tomada de la prueba para las 3 disposiciones explicadas. 77
5.12 Gráfica mostrando los resultados de la prueba. 79
5.13 De arriba a abajo, comparaciones de esta prueba para el tiempo de carga y de rebotes. 80
5.14 Comparación de los tres casos de prueba para ambos simuladores. 80

CAPÍTULO 1
Introducción

El principal objetivo de la industria automovilística es vender vehículos que otorguen a sus usuarios una manera cómoda y eficiente de poder desplazarse entre diferentes lugares. A pesar de ser este el objetivo principal, a la hora de comprar un vehículo, entran en juego muchos factores como precio, número de plazas, tipo de combustible, autonomía, consumo de combustible, comodidad, etc. Sin embargo, un factor muy importante a la hora de adquirir un vehículo es que este venga sin ningún fallo, dado que nadie quiere comprar un coche, por ejemplo, con una raya en la carrocería. Es por ello que existe tanta preocupación entre los fabricantes por detectar estos defectos cuanto antes.

1.1 Motivación

Existen dos motivos principales que justifican la elección de este trabajo. En primer lugar, la detección de defectos constituye un aspecto crítico en la producción de vehículos comerciales. Esta etapa es esencial, ya que un vehículo con defectos que llega al concesionario o al cliente final genera costes de reparación significativamente superiores a los que se producirían si el defecto se detectara en fábrica. Además del coste económico, las marcas enfrentan el riesgo de deterioro de su imagen. La percepción de calidad de una marca está inversamente relacionada con el número de defectos presentes en sus productos.

Por tanto, la detección de defectos no solo es fundamental en el proceso de producción, sino que debe ser abordada con profundidad para garantizar su correcta y eficiente ejecución.

En segundo lugar, este trabajo propone el uso de trazado de rayos para simular el proceso de detección de defectos. El campo de los gráficos por computador resulta de especial interés, tanto a nivel personal como profesional, y desarrollar un trabajo centrado en trazado de rayos —una técnica ampliamente utilizada en la actualidad— permite acercarse a aplicaciones reales con potencial relevancia futura. Asimismo, el uso de la librería *Vulkan*, ampliamente adoptada en la industria, representa una oportunidad de aprendizaje valiosa al tratarse de una tecnología nueva en el contexto del autor.

1.2 Objetivos

El objetivo de este trabajo es diseñar e implementar una parte del sistema de simulación utilizado en el diseño de túneles de inspección para la detección de defectos de pintura en carrocerías de vehículos.

En concreto, el trabajo se centra en el desarrollo de un simulador basado en trazado de rayos mediante la librería gráfica *Vulkan*, con el propósito de mejorar el rendimiento respecto a un simulador existente basado en *OpenGL*.

El trabajo se compone de:

- El diseño y desarrollo de un simulador por trazado de rayos para la detección de defectos.
- La comparación del simulador desarrollado con el sistema existente.

La implementación del simulador debe consistir en una librería con una API pública que permita al usuario acceder y utilizar las funcionalidades desarrolladas.

El simulador existente utiliza *OpenGL* para generar imágenes del proceso de inspección, es decir, modelos 3D de vehículos donde se visualizan los reflejos de distintas fuentes de luz. Al estar basado en *OpenGL*, se beneficia de utilidades de alto nivel que simplifican su programación.

Se espera que el nuevo simulador, al estar desarrollado con *Vulkan*, sea más rápido y eficiente. Esto se debe a que *Vulkan* opera a un nivel más cercano al hardware, lo que permite una mejor adaptación del simulador a las características específicas del sistema. Aunque las diferencias no sean perceptibles para el usuario final, internamente el simulador debería mostrar una mayor eficiencia.

1.3 Estructura de la memoria

La memoria de este trabajo se organiza en las siguientes secciones:

En primer lugar, se analiza el estado del arte, tanto en lo referente a la detección de defectos en vehículos mediante distintas técnicas, como en el ámbito del trazado de rayos. A continuación, se propone una solución junto con su plan de trabajo, incluyendo una sección dedicada al conocimiento previo necesario para comprender los conceptos que se abordarán posteriormente.

Seguidamente, se describe la arquitectura y el diseño del sistema completo, incorporando un apartado sobre las tecnologías utilizadas durante el desarrollo. La sección de desarrollo detalla los pasos seguidos y las decisiones tomadas a lo largo del proyecto.

Posteriormente, se presenta la sección de implantación, que incluye las pruebas realizadas y sus respectivos resultados. Finalmente, se expone el capítulo de conclusiones, donde se discuten los resultados obtenidos, seguido de una sección de trabajo futuro en la que se plantean posibles líneas de ampliación del proyecto.

CAPÍTULO 2
Estado del arte

Dadas las particularidades del trabajo, el análisis del estado del arte se ha dividido en dos partes: la detección de defectos en automóviles y la simulación de la luz mediante trazado de rayos.

2.1 Detección de defectos

La detección de defectos en carrocerías dentro de la industria automovilística constituye un proceso de control de calidad en el que se inspecciona la correcta aplicación de la pintura sobre la superficie del vehículo. Los defectos más comunes incluyen rayaduras y golpes. Históricamente, estas inspecciones han sido realizadas por operarios humanos; sin embargo, en los últimos años han surgido diversos sistemas orientados a automatizar esta tarea.

Los defectos se definen como cualquier anomalía en la superficie con un tamaño superior a 0,1 mm. El problema se centra en la detección precisa de las siguientes cuatro propiedades ópticas [7]:

- Reflexión especular de luz incidente normal sobre una superficie lisa.
- Reflexión difusa de luz incidente sobre una superficie rugosa.
- Reflexión direccional de luz incidente sobre un material brillante.
- Difusión y absorción de luz incidente sobre la capa de pintura.

2.1.1. Métodos basados en inspección humana

El método más común para la detección de defectos es la inspección visual realizada por expertos humanos. Estos se basan en la deformación de la luz proyectada sobre el vehículo para identificar posibles irregularidades [7].

Este enfoque presenta diversas limitaciones, todas ellas derivadas del hecho de depender del juicio humano. Entre los principales inconvenientes se encuentran:

- Alta subjetividad.
- Baja estabilidad.
- Baja eficiencia.



Figura 2.1: Túnel de inspección manual.

- Alto coste humano.

Estos problemas se resumen en la imposibilidad de cuantificar los criterios utilizados por los inspectores, así como en la variabilidad de los resultados debido a factores como la fatiga o el estado físico del operario [6]. A pesar de sus limitaciones, estos sistemas continúan utilizándose debido al bajo coste de implantación.

2.1.2. Adquisición de imágenes

El primer paso en la creación de un sistema automático de detección de defectos consiste en adquirir las imágenes de la carrocería que se va a inspeccionar. Tal como se indica en [5], para realizar esta adquisición de forma adecuada se requieren tres elementos: un sistema de iluminación, una cámara con lente, y una interfaz entre la cámara y el ordenador.

En lo referente a la iluminación, los estudios se centran habitualmente en el uso de luces LED debido a sus ventajas. Según *X. Zheng* [9], *LED has a long life cycle, and its lifetime is commonly longer than 100,000 h. Its brightness can be controlled conveniently with low power consumption and low heat production. It can be designed into different sizes and shapes and can irradiate at different angles. Guidelines for setting LED light sources include the achievement of good contrast between the foreground and background for reliable measurement and of good contrast among the internal features*.

Respecto a las cámaras, independientemente del modelo utilizado, estas se montan sobre raíles para facilitar su reposicionamiento. Generalmente se emplean cámaras digitales, dado su mejor rendimiento en comparación con las cámaras analógicas [9].

Finalmente, la interfaz entre la cámara y el ordenador es específica para cada sistema.

2.1.3. Detección mediante binarización

Un ejemplo de método básico para la detección de defectos en una imagen es la binarización.

Para aplicar este método, primero se extraen candidatos a defectos a partir de la imagen adquirida. La imagen se transforma a escala de grises y se le aplica un filtro para reducir el ruido [7]. A continuación, se identifican las regiones de interés (ROI) analizando las zonas con niveles de luminosidad superiores o inferiores a la media, utilizando un

umbral global. Para cada ROI, se realiza un análisis local comparando la luminosidad de cada píxel con el resto de la región. Aquellos puntos cuya diferencia supere un umbral igual a la media de los valores de los píxeles se marcan como posibles defectos.

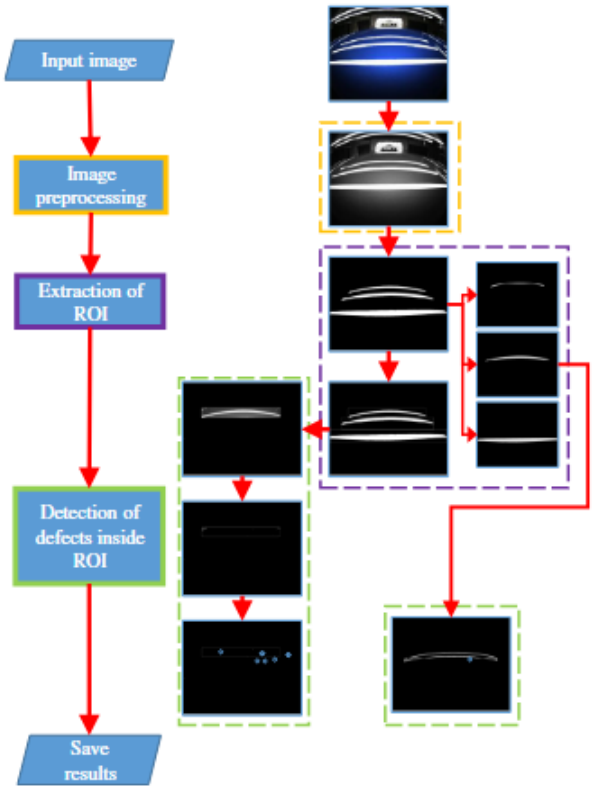


Figura 2.2: Proceso de binarización [7]

Una vez identificados los candidatos, es habitual realizar una verificación adicional. Algunos sistemas emplean *Support Vector Machines* (SVM) para clasificar los defectos a partir de la imagen [6]. Otros métodos utilizan múltiples imágenes consecutivas para confirmar la presencia del mismo defecto en todas ellas, o emplean defectos de referencia claramente visibles que sirven como patrón para validar la forma de los defectos detectados [7].

2.1.4. Detección mediante reflexión difusa

La detección basada en la inspección de la reflexión difusa consiste en proyectar luz sobre la superficie del vehículo y analizar las imágenes capturadas. Para ello, la fuente de luz se posiciona de forma que permita observar la superficie desde distintos ángulos de reflexión. Este enfoque asume un comportamiento físico realista de la luz [1], en el que la reflexión no se produce como un rayo perfecto.

Una vez construido el sistema experimental, compuesto por cámara y fuentes de luz, se capturan imágenes desde diferentes ángulos para verificar que la luz se refleja correctamente desde la fuente hasta la cámara. Para facilitar el análisis, la imagen en color se transforma a escala de grises [1]. Los puntos que no presentan una reflexión uniforme se consideran defectuosos.

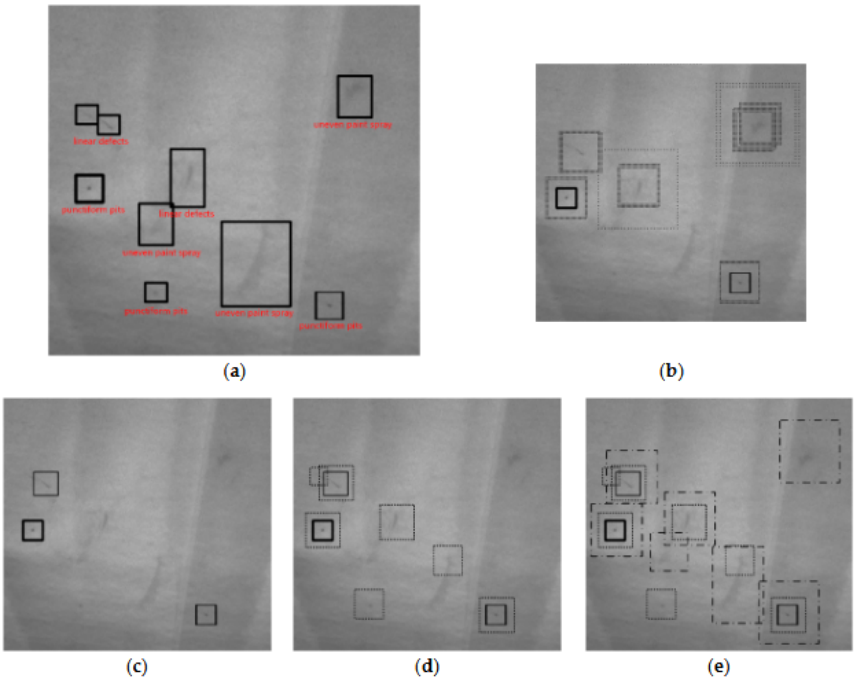


Figura 2.3: Imagen extraída de [6], Defectos detectados y clasificados por SVM

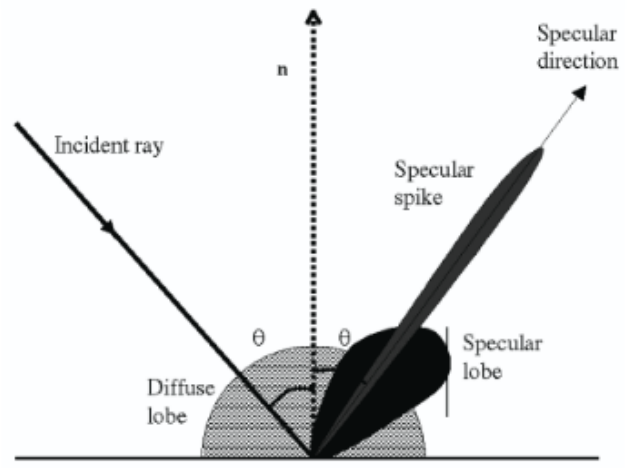


Figura 2.4: Modelado de la reflexión de la luz [1]

2.1.5. Detección mediante *deep learning*

Los métodos basados en *Deep Learning* reformulan el problema como una tarea de detección múltiple de objetos —en este caso, defectos— dentro de una imagen. Dado que las imágenes suelen tener alta resolución para permitir la identificación precisa de defectos, es habitual aplicar técnicas de *Downsampling* [5], con el fin de reducir el tamaño de la imagen manteniendo sus características relevantes.

Respecto a los modelos utilizados, diversos estudios sobre detección de objetos pequeños [9] han demostrado que las redes neuronales convolucionales (CNN) ofrecen buenos resultados en el procesamiento de imágenes. Algunos modelos representativos son *AlexNet*, *ResNet* y *GoogleNet*.

Table 5 Deep CNN-based supervised learning approaches for surface inspection in semiconductor industry

Reference	Year	Deep learning methods	Dataset[a]	Accuracy[b]	Object detected
[121]	2017	CNN (AlexNet) and sequential extreme learning machine	14,491 images (TN: 11,500; TS: the rest)	94%	Flat panel displays
[122]	2018	CNN (AlexNet and ResNet) and image transform	Not supplied	91.1%	SMT
[123]	2018	CNN (ResNet)	TN: 37,400; TS: 9444 and 220,281, respectively	97.2%	Through-silicon via (TSV)
[124]	2018	CNN using defect probability image	1500 defect and 1796 good images (TN 50%; TS 50%)	98.813%	Chip-size package (CSP)
[125]	2018	CNN	1350 images (TN: 1200; TS: 150)	Multi-categories: 89.89%; two categories: 92.86%	PCB
[126]	2018	CNN	4526 images of normal and defect images	99.7%	PCB
[127]	2018	CNN (Inception v3)	7543 images (TN 6167; VS 576; TS 800)	91.125%	PCB
[128]	2018	CNN	1818 images for six types of PCB defects (TN 1069; VS 323; TS 426)	95.7%	PCB
[129]	2018	CNN	28,600 images (TN 15,400; VS 6600; TS 6600)	98.2%	Wafer
[130]	2019	CNN and extreme gradient boosting	11,730 images (TN 9384; TS 2346)	99.2%	Wafer
[131]	2019	CNN (VGG)	WM-811K (TN 54,355 and TS 118,595)	94.7% for multiple failure pattern	Wafer map
[132]	2019	CNN and k-NN	2123 images of five defect classes (TN70%; VS 15%; TS 15%)	96.2%	Wafer
[133]	2018	CNNs (LeNet, CifarCNN, and GoogleNet)	658 images (TN: 600; VS: 48; TS: 10)	98%	Photovoltaic cell
[112]	2019	SVM and CNN(VGG-19)	2624 images (TN 1968; TS 656)	88.42%	Photovoltaic cell
[134]	2019	CNN combined with class activation mapping technique	LEDC-GD-C1: TN 24,000; TS 6000 LEDC-GD-C2: TN 10,400; TS 2600	LEDC-GD-C1: 94.96%; LEDC-GD-C2: 94.49%	LED chip

[a] Number of image samples for the experiments. *TN* training set, *TS* testing set, *VS* validation set

[b] The best or average accuracy achieved

Figura 2.5: Resultados de diferentes métodos en tareas de detección de objetos pequeños [9]

Por otro lado, algoritmos no supervisados como YOLO o SSD también han mostrado resultados destacados [9]. Estos modelos presentan una ventaja adicional: su elevada velocidad de procesamiento, lo que contribuye a reducir los costes de producción [5].

Table 8 Semi-supervised learning and unsupervised learning-based approaches for surface defect detection

Reference	Year	Deep learning methods	Dataset[a]	Accuracy[b]	Object detected
[29]	2017	Denoising auto-encoder	2600 samples of jacquard pattern fabric (TN 2000; TS 600)	98–99.47%	Fabric
[146]	2018	Convolutional denoising auto-encoder network	Four texture datasets	83.8%, 85.2%, 80.3%, and 84.0% on the four datasets	Fabric
[147]	2018	Auto-encoder network and data augmentation	Scratch images from NEU dataset, solder defect images, and industrial defect images for evaluation	True positive rate and false positive rate are given rather than accuracy	Surface defects
[148]	2018	CNN (VGG and ResNet), RPN, and Faster-RCNN	2132 images (TN 1549; VS 297; TS 286)	95.48%	Fabric
[149]	2018	YOLO	6 classes of 4655 steel strip surface images (TN 3455; TS 1200)	97.55%	Steel
[150]	2018	SSD and CNN(MobileNet)	Data augmentation based on 400 images obtained from the real-world	96.73%	Sealing surface
[151]	2019	SSD and speed model	3000 images (TN: 2140; TS: 860)	98.00%, 99.00%, 97.80%, and 79.40% for the four defect types	Darning needle
[152]	2019	Convolutional auto-encoder and semi-supervised GAN	(1) Hot rolled plates: TN 5000, TS 1400; (2) hot rolled steel strips: TN 9000, TS 1800; (3) cold rolled strips: TN 4000, TS 1800	97.2% for hot rolled plates, 98.2% for hot rolled steel strips, and 96.7% for cold rolled strips	Steel
[153]	2020	CNN with Pseudo-label	NEU dataset (TN 1500, TS 300)	90.7%	Steel

[a] Number of image samples for the experiments. TN for training set, TS for testing set, VS for validation set

[b] The best or average accuracy achieved

Figura 2.6: Resultados de diferentes modelos no supervisados en tareas de detección de objetos pequeños [9]

2.2 Trazado de rayos

Hasta tiempos recientes, la técnica predominante en el renderizado ha sido la rasterización [4]. La rasterización consiste en transformar una serie de geometrías espaciales en una imagen raster mediante la asignación de un color a cada píxel, en función de la posición de dichas geometrías. Esta técnica presenta limitaciones, ya que no considera las primitivas cercanas al objeto analizado en el cálculo del color. Por ello, efectos ópticos como la reflexión o la refracción solo pueden aproximarse. Para abordar esta limitación, se han desarrollado métodos más precisos para el cálculo de la luz, como el trazado de rayos.

El trazado de rayos es una técnica que computa los rayos de luz que alcanzan al observador, ya sean emitidos directamente por fuentes de luz o reflejados en objetos [4]. En lugar de emitir rayos desde cada fuente en todas las direcciones, se aplica un enfoque inverso que reduce significativamente el coste computacional. El algoritmo lanza rayos desde el punto de vista del observador hacia cada píxel de la pantalla. Si un rayo intersecta un objeto, la iluminación se calcula en función de las propiedades del material y la luz incidente.

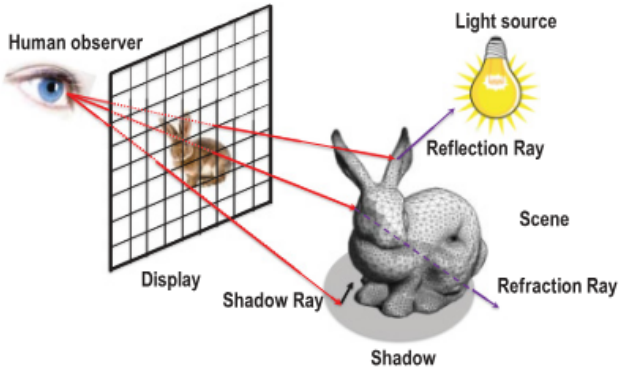


Figura 2.7: Diagrama explicativo del trazado de rayos [4]

El cómputo del trazado de rayos es altamente costoso. Para generar una imagen, se deben emitir múltiples rayos, como mínimo uno por píxel, que recorren la estructura que contiene la geometría de la escena. Esto implica la creación de millones de rayos por imagen, con el objetivo de alcanzar altas tasas de generación de imágenes por segundo. Para reducir el tiempo de procesamiento por rayo, se requieren arquitecturas especializadas. El trazado de rayos se plantea como un problema de búsqueda de intersecciones entre rayos y primitivas (triángulos), por lo que se busca minimizar el coste de cálculo de dichas intersecciones. Se han propuesto diversas estructuras de aceleración [4], entre ellas:

- Separación del espacio en áreas de igual tamaño, de modo que cada geometría se ubique en una zona distinta.
- *Kd-tree*, una variante de árbol binario que divide el espacio equilibrando la cantidad de elementos entre los nodos hijos.
- BVH, una estructura jerárquica que asocia una *bounding box* a cada elemento. Esta permite solapamientos entre objetos y sus respectivas *bounding boxes*. Un objeto puede pertenecer a varias *bounding boxes*, y una *bounding box* puede contener múltiples objetos.

Las librerías comerciales suelen emplear estructuras de aceleración específicas, optimizadas para su arquitectura.

El proceso de trazado de rayos se organiza en una tubería gráfica (*Graphics pipeline*) compuesta por las siguientes etapas [4]:

- Generación de estructuras de aceleración.
- Emisión de rayos.
- Recorrido de la estructura de aceleración por cada rayo.
- Test de intersecciones.
- Aplicación de *shaders*.

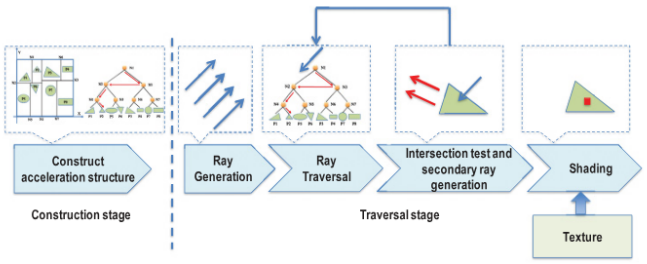


Figura 2.8: Tubería de trazado de rayos [4]

2.2.1. Trazado de rayos por hardware

El siguiente paso para mejorar el trazado de rayos consiste en emplear hardware especializado. El objetivo es aprovechar al máximo la GPU del sistema para reducir el tiempo de procesamiento.

El funcionamiento general es similar: se generan las estructuras de aceleración, se emiten los rayos, estos atraviesan las estructuras y se buscan intersecciones. La principal diferencia radica en que las GPUs están diseñadas para ejecutar tareas altamente paralelizables, mediante múltiples núcleos y colas [2], mientras que las CPUs no ofrecen estas capacidades. Por este motivo, se realizó un estudio comparativo [2] que mostró que, para hardware de gama similar, una implementación en CPU podía generar 114 millones de rayos por segundo, mientras que la implementación en GPU alcanzaba los 222 millones de rayos por segundo, sin utilizar todo el ancho de banda disponible de la GPU.

CAPÍTULO 3
Solución propuesta

El objetivo de este trabajo es, nuevamente, la creación de un simulador de detección de defectos basado en trazado de rayos. Para ello se pretende utilizar la librería gráfica de *Vulkan* y comparar su funcionamiento con otro simulador existente basado en *OpenGL*.

Para completar los objetivos propuestos es preciso definir claramente cómo se va a desarrollar esta solución de tal manera que todo el proceso quede acotado y guiado por dichas pauta propuestas.

3.1 Plan de trabajo

Para poder abordar todo el trabajo a realizar, será necesario ordenar el desarrollo de tal manera que queden objetivos claros y alcanzables durante todo el tiempo que este ocupe.

Primeramente, se comenzará con una fase de diseño donde se abordarán cuáles deben ser los objetivos del trabajo. Una vez concretados estos se comenzará con el desarrollo de un entorno base de *Vulkan* el cual permita poder implementar todas las funcionalidades necesarias más adelante. Debido a la inexperiencia este apartado durará un tiempo aproximado de 75 horas, tomando el tiempo necesario para crear correctamente toda la base sobre la que se creará el resto del proyecto. Igualmente, debido a la naturaleza de esta base se visitará de nuevo en caso de necesitar expandirla.

Una vez creada una base, será necesaria una investigación amplia de los temas a tratar, la detección de defectos y el trazado de rayos, además de las cuestiones específicas de como desarrollar este en la librería de *Vulkan*. Esto incluye la lectura de trabajos de otros autores, de libros y guías explicativas y de la anotación de toda cuestión interesante surgida en esta o cualquier otra parte del trabajo para su posterior explicación.

Una vez concretada e investigada la información pertinente, se procederá a desarrollar una versión simple del trazado de rayos, consiguiendo una funcionalidad simplificada pero concretando y configurando todas las partes necesarias para ello. Este apartado tomará alrededor de 50 horas.

Tras crear una versión simplificada, el siguiente paso será la creación de una versión final y detallada, además de la creación de una interfaz para poder acceder a esta de una manera más intuitiva. Esta parte del trabajo será la más laboriosa y complicada, debiendo aplicar todo el conocimiento específico y realizando gran cantidad de pruebas intermedias para confirmar el correcto funcionamiento. Dado esto, se espera una duración de alrededor de 150 horas de trabajo.

Una vez creado todo el sistema será necesario una fase de pruebas de aceptación donde se someterá al sistema a una serie de casos posibles. En caso de no superar las pruebas será necesario modificar el sistema. Dada la naturaleza de esta parte no se espera que su duración sea mayor a 25 horas.

Finalmente, al mismo tiempo que se realicen todas las partes comentadas, se escribirá la memoria del trabajo. Este recopilará todas las cuestiones aparecidas durante el desarrollo, las reflexiones y aportes de otros autores encontrados durante la investigación y los resultados finales. Debido a la gran cantidad de datos y información a recopilar se espera que esta parte tenga una larga duración, de alrededor de 150 horas o más debido a la necesidad de revisar y modificar partes si fuera necesario.

Con el objetivo de organizar todo el desarrollo del trabajo, se creará una lista de tareas donde se apuntarán tanto objetivos grandes generales como otras tareas más pequeñas y puntuales de tal manera que se pueda analizar el trabajo que se está realizando. Esta lista se revisará constantemente, añadiendo o quitando apartados dependiendo de si han sido completados o ya no se consideran relevantes.

Una vez organizadas las tareas es necesario colocarlas en un espacio temporal. Se creará un calendario donde se determinarán diferentes fechas para las cuales diferentes partes del trabajo deben estar completadas. El trabajo se comenzará el 13/4 y a partir de ese día se trabajará. El objetivo es ser constante, avanzando en el trabajo durante el tiempo que se está cursando en el máster y una vez acabado este dedicar más tiempo al TFM.

En concreto, se pretende completar la aplicación base para el final de abril, completar la versión básica del trazado de rayos al comienzo de julio, finalizar la versión completa al comienzo de agosto y entregar el trabajo en la convocatoria de septiembre.

3.2 Conocimiento previo

Debido a la alta especificidad de la librería utilizada, *Vulkan*, se considera necesario explicar detalladamente los elementos específicos que se emplearán durante el desarrollo. *Vulkan* es una librería muy explícita [3], en la que la responsabilidad de los procesos recae completamente en el programador. Por este motivo, se debe tener en cuenta el hardware requerido para su funcionamiento y configurarlo adecuadamente.

Además, *Vulkan* presenta una elevada fragilidad, ya que requiere una considerable cantidad de trabajo para funcionar correctamente, mientras que un único error puede provocar el fallo total de la aplicación deseada [3].

Instancia, *Device*, *Physical device* y *Queue*

Vulkan presenta una jerarquía funcional que requiere la creación de ciertos objetos en un orden específico. A continuación, se describe dicha jerarquía.

Una instancia es un subsistema que separa lógicamente la aplicación del resto de aplicaciones o librerías utilizadas. Esta contiene parámetros como la versión de la API, el nombre de la aplicación y otros datos relevantes. A partir de la instancia se crean el resto de objetos, y su destrucción impide el uso posterior de las funciones de *Vulkan*.

Una vez creada la instancia, se deben obtener los dispositivos físicos o *Physical Devices* disponibles en el sistema. Estos dispositivos, como GPUs o aceleradores, son compatibles con *Vulkan* y permiten acceder a información sobre sus capacidades, número de colas, entre otros. Una instancia puede contener varios *Physical Devices*, por lo que será necesario seleccionar cuál o cuáles se utilizarán para ejecutar la aplicación.

Tras seleccionar un *Physical Device*, se debe crear un dispositivo lógico o *Device*. Este objeto lógico permite acceder a las capacidades del *Physical Device* y operar sobre él. La mayoría de las funcionalidades de la librería se acceden a través del *Device*. Aunque se crea a partir de un *Physical Device*, este puede contener múltiples *Devices*.

Una *Queue* es una unidad de procesamiento a la que un *Device* puede enviar trabajo. Las *Queues* están agrupadas en familias que operan en paralelo y comparten características. El número de familias y *Queues* es una propiedad del *Physical Device*. Generalmente, un *Physical Device* dispone de varias familias, cada una con múltiples *Queues*. Durante la programación, el *Device* selecciona la familia y la *Queue* adecuadas, así como el trabajo que se enviará a esta.

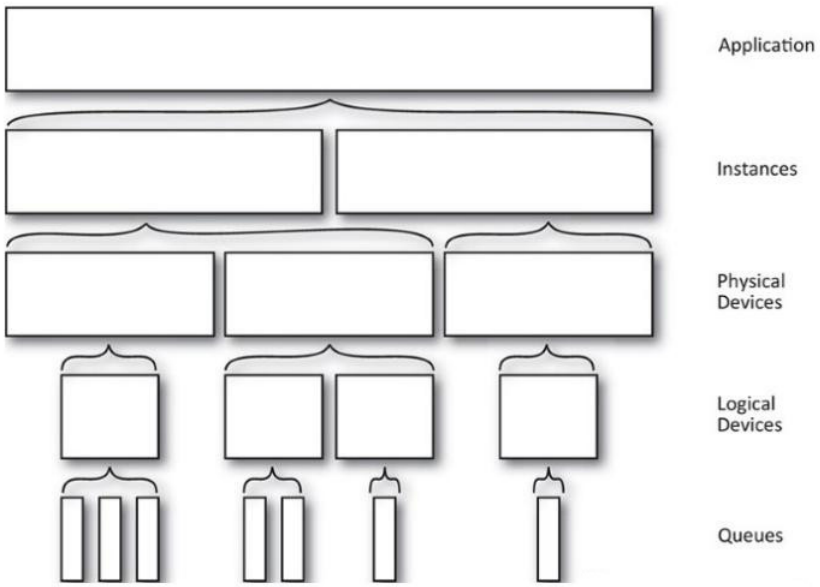


Figura 3.1: Gráfica extraída de [3] mostrando la jerarquía *Vulkan*

RenderPass y *FrameBuffer*

En *Vulkan*, el objetivo final de la aplicación consiste en generar una imagen. Esta puede ser simple o construida mediante múltiples iteraciones o pases, donde cada pase produce una parte distinta del resultado final.

Vulkan incorpora un objeto denominado *Renderpass*, encargado de encapsular y organizar los distintos pases. Su función es gestionar las entradas y salidas de cada uno hasta obtener la imagen final. Todo contenido que se desee dibujar debe definirse dentro de un *Renderpass*.

Por otro lado, el *Framebuffer* es un objeto de *Vulkan* que especifica en qué imágenes o espacios de memoria se renderizarán los resultados. Además, puede almacenar recursos adicionales, como imágenes de profundidad o de *Stencil*, lo que permite al *Renderpass* guardar o extraer dicha información.

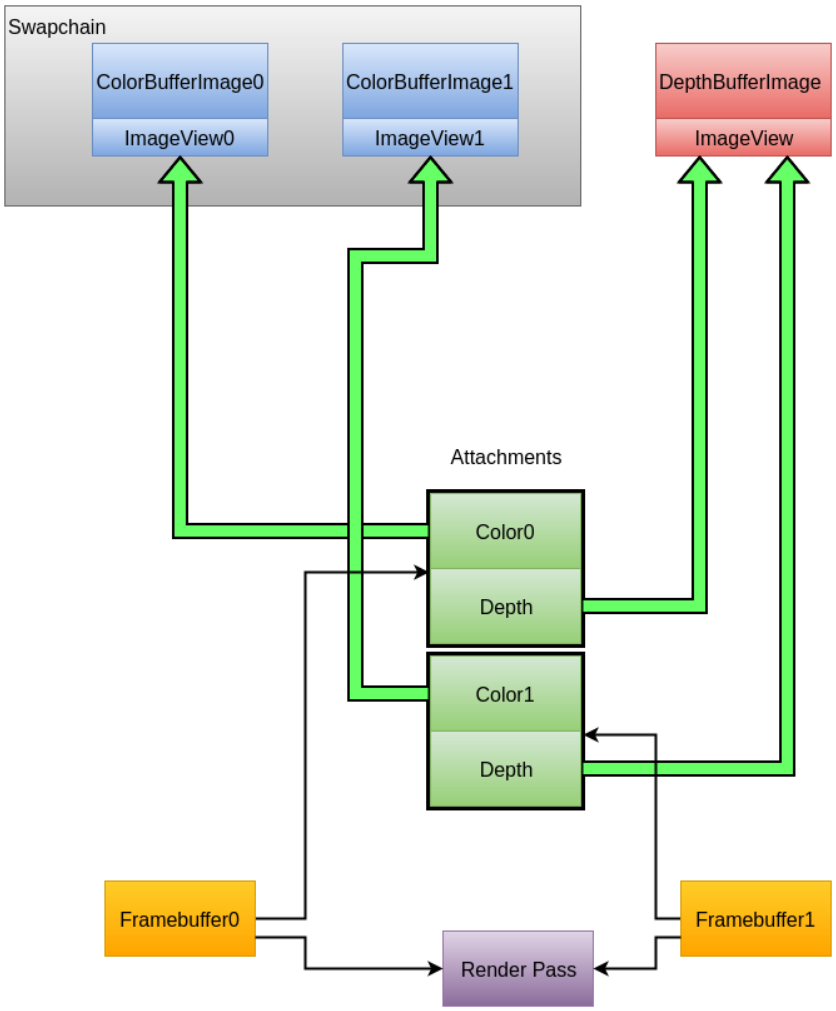


Figura 3.2: Gráfico explicativo del funcionamiento de los *Renderpass* y *Framebuffer*

Estructuras de aceleración: *BLAS* y *TLAS*

Una vez creada la geometría, en una aplicación de trazado de rayos es necesario construir una estructura de aceleración que reduzca significativamente el tiempo requerido para acceder a los triángulos de las mallas. *Vulkan* emplea un sistema jerárquico de dos niveles de estructuras de aceleración [8].

La *Top-Level Acceleration Structure* (*TLAS*) es una estructura de aceleración de instancias. Cada instancia apunta a una *Bottom-Level Acceleration Structure* (*BLAS*), que representa una estructura de aceleración para triángulos o *Bounding Boxes*. Cada *BLAS* agrupa uno o más objetos de la escena que se encuentran próximos entre sí y comparten las mismas transformaciones.

El objetivo de esta jerarquía es permitir que, al intersectar un rayo con la *TLAS*, se determine si existe intersección con alguna *BLAS*. En caso afirmativo, se invoca el *Shader* correspondiente para procesar la intersección.

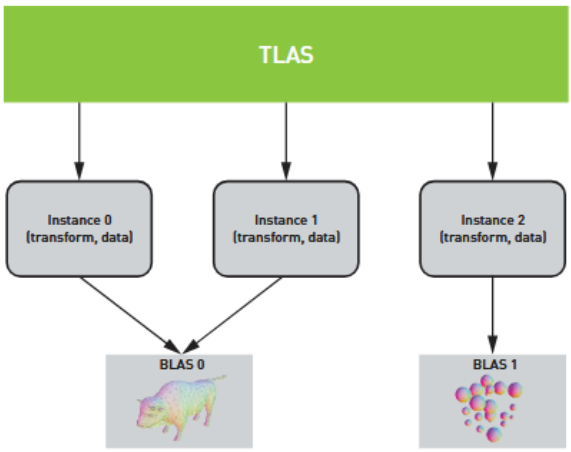


Figura 3.3: Gráfico explicativo de los *TLAS* y *BLAS*, extraído de [8]

Descriptor sets

Vulkan, al igual que otras librerías gráficas, requiere el uso de *Shaders* para completar la ejecución del programa. Estos *Shaders* suelen necesitar información proveniente del programa para funcionar correctamente, por lo que es necesario transferir dicha información desde la CPU a la GPU. *Vulkan* realiza esta tarea mediante los *Descriptor Sets*.

Los *Descriptor Sets* son conjuntos de recursos que pueden enlazarse a la *Pipeline* como grupo. Es posible enlazar múltiples *Descriptor Sets* simultáneamente, y aquellos con la misma disposición de atributos son intercambiables.

Un ejemplo se muestra en la Figura 3.4, donde se representan dos *Descriptor Sets*: uno con dos *Buffers*, una textura y su *Sampler* asociado, y otro con un mayor número de atributos.

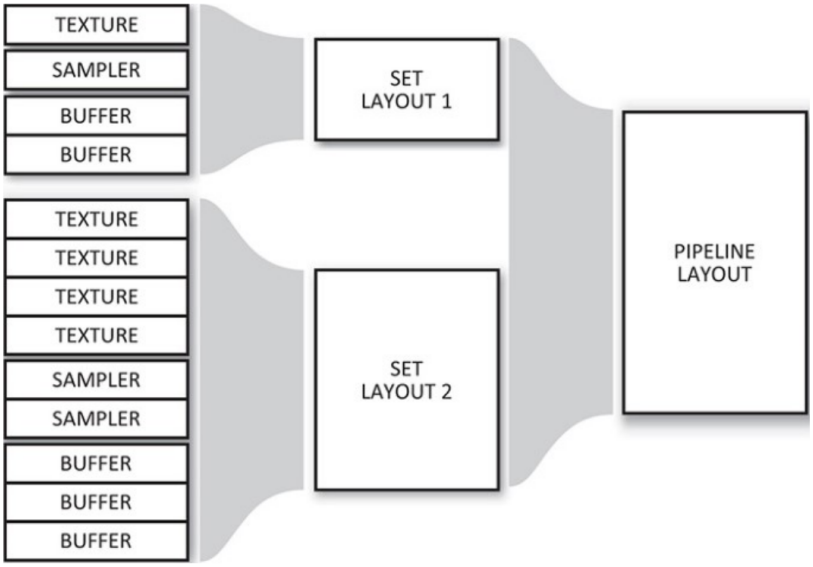


Figura 3.4: Gráfico extraído de [3], mostrando una disposición de *descriptor sets*

Pipeline y *Shader binding table*

La tubería de trazado de rayos, o *Pipeline*, es el objeto encargado de procesar toda la geometría y la escena para generar el resultado deseado. A diferencia de la tubería gráfica tradicional, esta no incluye *Shaders* de vértice o fragmento, sino una serie de *Shaders* específicos que se invocan según sea necesario. Estos son:

- *Ray Generation* o *Raygen*, genera los rayos.
- *Miss*, se ejecuta si el rayo no intersecta ningún objeto.
- *Intersection*, detecta si el rayo intersecta con algún triángulo.
- *Any Hit*, se ejecuta cuando el rayo intersecta con cualquier triángulo.
- *Closest Hit*, se ejecuta cuando el rayo intersecta con el triángulo más cercano.

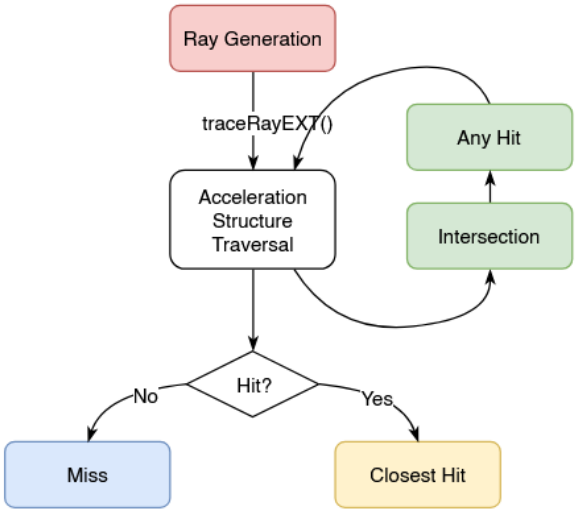


Figura 3.5: Gráfico extraído de [8], mostrando el flujo de los shaders

Es responsabilidad del programador, al construir la *Pipeline*, definir cuáles de estos *Shaders* se incluirán, lo que determinará la funcionalidad final.

Una vez construida la *Pipeline*, es necesario indicar al *Driver* cuáles *Shaders* han sido implementados y dónde se encuentran. Para ello, se crea la *Shader Binding Table*, una estructura similar a un array que almacena el código de los *Shaders*, sus referencias y los datos necesarios para su correcto funcionamiento, dado que pueden ser invocados para cualquier objeto en cualquier momento.

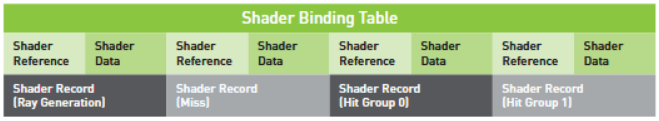


Figura 3.6: Gráfico extraído de [8], mostrando un ejemplo de *Shader binding table*

3.3 Arquitectura del Sistema

El simulador se estructura de la siguiente manera: En primer lugar, se define una clase de renderer, la cual actúa como interfaz para acceder a todas las funcionalidades del trazado de rayos. De este modo, el funcionamiento subyacente de *Vulkan* permanece opaco e inaccesible. Con esta aproximación, se asegura que el uso de la biblioteca sea sencillo y no requiera conocimientos previos de *Vulkan*.

Por otra parte, se implementa un núcleo de *Vulkan*, encargado de crear y gestionar todas las funcionalidades básicas necesarias para el resto de la aplicación. Este componente crea los objetos de *Vulkan*, a los cuales se accede mediante llamadas desde los demás módulos.

Finalmente, se desarrolla un núcleo de trazado de rayos, responsable de ejecutar dicho proceso. Este núcleo es accesible desde el renderer, que emplea sus funcionalidades para completar las distintas tareas. Además, accede al núcleo de *Vulkan* para crear los objetos requeridos para su funcionamiento, salvo aquellos específicos del trazado de rayos.

El resultado se muestra en el diagrama de la figura 3.7.

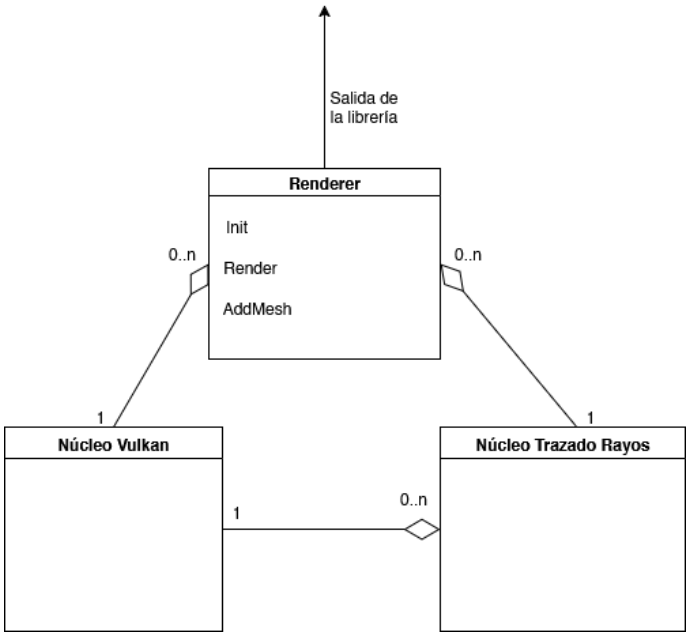


Figura 3.7: Diagrama de la arquitectura del sistema

En cuanto a la organización física del trabajo, se ha desarrollado en Visual Studio y se estructura de la siguiente manera: en primer lugar, se define un proyecto que contiene una interfaz de código, la cual es implementada posteriormente por el renderer. Esta interfaz incluye todos los métodos necesarios para el correcto funcionamiento del renderer. El objetivo de esta interfaz es permitir, en etapas posteriores del desarrollo, la incorporación de un renderer basado en *OpenGL*, que también implemente dicha interfaz. De esta forma, la funcionalidad de ambos se mantiene opaca e intercambiable, lo que facilita la comparación del tiempo de ejecución de los métodos de manera sencilla y eficiente.

Otro proyecto incluido en la librería corresponde al renderer de *Vulkan*, el cual contiene los componentes descritos previamente: el núcleo de *Vulkan*, el núcleo de trazado de rayos y el renderer de *Vulkan*. El acceso externo se limita al renderer, que mantiene la misma forma que la interfaz mencionada.

Finalmente, se incorporan dos proyectos adicionales de pruebas. Estos incluyen código destinado a verificar el correcto funcionamiento de la librería: uno mediante una consola de comandos y otro a través de una aplicación gráfica que permite mostrar las imágenes generadas por la librería.

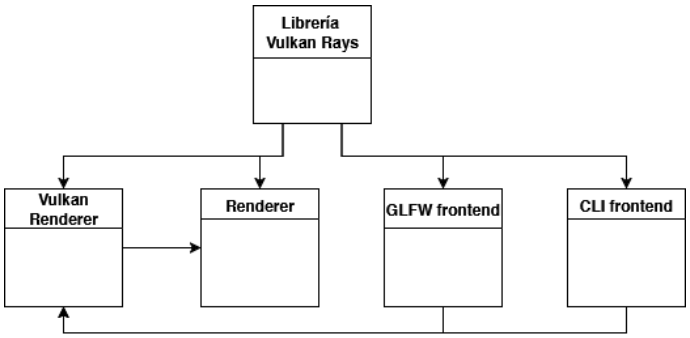


Figura 3.8: Diagrama de la organización del github

3.4 Diseño detallado

Una vez especificada la arquitectura del trabajo, la siguiente tarea es realizar un diseño de la misma que sea claro y preciso, de tal manera que a la hora de implementarla todo esté definido y referenciado.

3.4.1. Núcleo *Vulkan*

En lo que respecta al núcleo de *Vulkan*, este es el encargado de la creación de los objetos necesarios para el desarrollo del trabajo. Además, implementa las funciones de apoyo requeridas por la librería de *Vulkan*.

Este núcleo constituye la primera parte de la aplicación en ser inicializada, por lo que debe encargarse de crear los objetos utilizados por el resto de los módulos. En primer lugar, se crea la instancia de *Vulkan*, indispensable para acceder a las funcionalidades de la librería. Junto con la instancia, se inicializan también las extensiones de instancia requeridas para habilitar el trazado de rayos. Algunas de estas extensiones son específicas del sistema operativo, por lo que, para mantener consistencia durante el desarrollo, se adopta *Windows* como plataforma de referencia.

Tras la creación de la instancia, se implementa una capa de utilidades encargada de comprobar y validar el uso correcto de la librería de *Vulkan*. Es recomendable emplear esta capa desde el inicio del desarrollo debido a la fragilidad de la librería, donde un fallo puntual puede propagarse e impedir el funcionamiento de la aplicación. Esta capa de validación forma parte de *Vulkan* y se activa mediante una extensión.

A continuación, se obtienen e inicializan los *Physical Devices* disponibles en el sistema. Estos pueden ser varios y de cada uno se recopila información relevante, como:

- Número de familias de *Queues*.
- Propiedades de las familias disponibles (tipos de operaciones soportadas).
- Cantidad de *Queues* en cada familia.
- Tamaño y tipo de memoria disponible.
- Versión de la API instalada.

Los *Physical Devices* contienen más información, pero dado que en este trabajo no se presentan imágenes directamente en pantalla, no resulta necesario obtenerla. Una vez recopilados, se selecciona uno de los *Physical Devices* para su uso en la aplicación. El método de selección escoge un dispositivo siempre que cumpla una serie de requisitos mínimos; en este caso, que permita la ejecución de operaciones gráficas. El resultado de la selección es un *Physical Device* y la familia de *Queues* apropiada para realizar las tareas.

Posteriormente, se crea un *Device* encargado de ejecutar todas las llamadas necesarias a la API de *Vulkan*. Durante su creación se habilitan también las extensiones requeridas para el correcto funcionamiento de la librería, entre ellas las relacionadas con el trazado de rayos, las estructuras de aceleración y la compilación de *shaders*.

En cuanto a las *Queues*, se obtienen de la familia seleccionada y se crean semáforos para su correcta sincronización. Esta operación se lleva a cabo mediante una llamada a la API de *Vulkan*.

Otra de las tareas del núcleo es la creación del *RenderPass* y de los *Framebuffers* asociados. El *RenderPass* definido incluye un único pase, utilizado para ejecutar el trazado de

rayos de forma completa. Al final de este pase, únicamente se requiere la imagen de color resultante, sin considerar otras como la imagen de profundidad. Además, la imagen obtenida se almacena en memoria en lugar de presentarse en pantalla, lo que se conoce como *Offscreen Rendering*.

Los *Framebuffers* se crean con el objetivo de gestionar la información de entrada y salida del *RenderPass*. En este caso, la configuración es simple, ya que solo requieren una imagen de color como salida. Dicha imagen se almacena en una textura, que posteriormente se devuelve al usuario como salida de la librería.

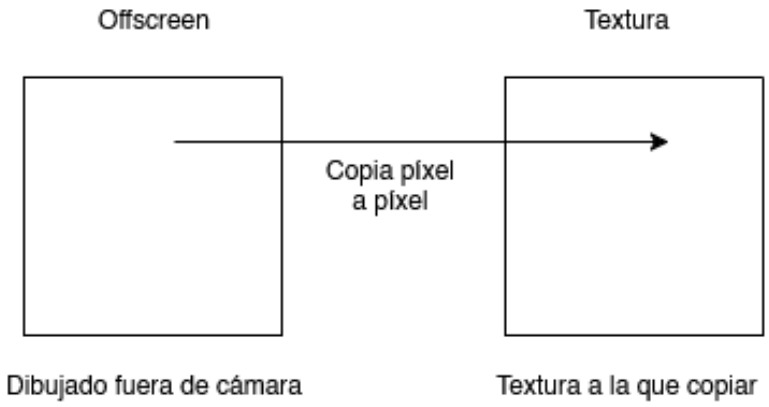


Figura 3.9: Diagrama del renderizado a textura y un ejemplo de su uso

El núcleo de *Vulkan* también se encarga de crear los *Buffers* necesarios para el funcionamiento de la aplicación. *Vulkan* requiere la creación de objetos *Buffer* propios y no permite el paso directo de información a los métodos. El proceso de creación de un *Buffer* incluye los siguientes pasos:

1. Creación de un objeto *Buffer*.
2. Obtención de los requisitos de memoria.
3. Asignación de espacio en memoria.
4. Enlace del espacio asignado al *Buffer*.

Una vez definida la creación de *Buffers*, se implementa la función encargada de llenarlos con información. Esta función crea un *Buffer* auxiliar, lo llena con los datos provistos y realiza una operación de copia hacia el *Buffer* final, de modo que solo se almacene la información relevante y el proceso se ejecute de forma rápida y eficiente. Con este procedimiento se generan más adelante los *Buffers* de vértices, normales, índices y otros atributos. Es importante destacar que deben incluirse las *Flags* adecuadas según el uso previsto del *Buffer*.

Al igual que en el caso de los *Buffers*, *Vulkan* requiere objetos propios para manejar imágenes y texturas. Por este motivo, cuando se crea una textura, ya sea de salida o a partir de una imagen, el núcleo se encarga de su gestión. En primer lugar, se define una imagen con el tamaño, formato y *Flags* especificados. Sobre esta imagen se crea un objeto *ImageView*, que permite acceder a la imagen generada. Si la imagen se utiliza como textura accesible desde un *Shader*, es necesario crear un *Sampler*, lo que implica una serie de operaciones adicionales. Las texturas pueden crearse tanto a partir de imágenes como de datos de píxeles.

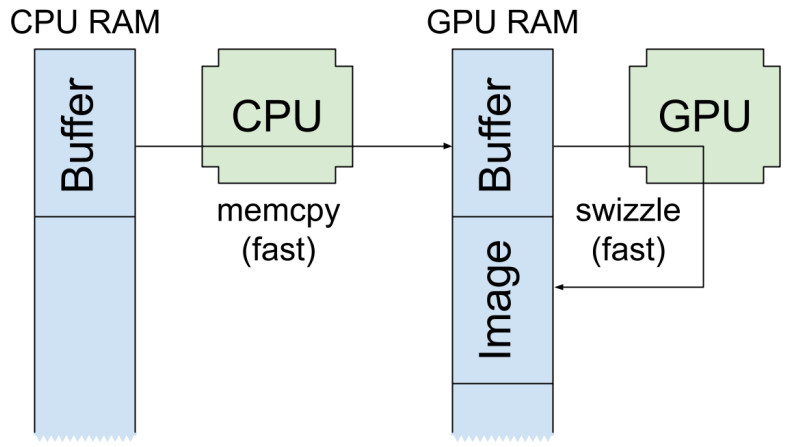


Figura 3.10: Diagrama del sistema de creación de búferes, mediante copia de búfer a búfer

El núcleo también gestiona operaciones comunes necesarias para el funcionamiento de otros componentes. Estas corresponden principalmente a operaciones de copia, aunque incluyen también:

- Copia de *Buffer* a *Buffer*.
- Copia de *Buffer* a imagen.
- Copia de imagen a *Buffer*.
- Copia de imagen a imagen.
- Copia de *Buffer* a memoria.
- Copia de imagen a memoria.
- Transición de formato de imagen.

Una tarea adicional del núcleo de *Vulkan* es definir el formato de las mallas utilizadas durante la ejecución. Estas mallas son opacas para los usuarios de la librería y se emplean únicamente de forma interna, aunque se construyen a partir de la información provista externamente. Las mallas contienen la siguiente información:

- *Buffer* de vértices.
- *Buffer* de índices.
- *Buffer* de normales.
- *Buffer* de coordenadas de textura.
- *Buffer* de color.
- Tamaño asociado a cada uno de los *Buffers*.
- Número total de vértices.
- Tipo de los índices.
- Identificador de la malla.

- Identificador de la textura utilizada.
- Matriz de modelo asociada.

Finalmente, otra tarea del núcleo es la creación y compilación de *Shaders*. *Vulkan* requiere que los *Shaders* estén escritos en *SPIR-V*, un lenguaje de bajo nivel similar al ensamblador pero adaptado a *Shaders*. Debido a la complejidad y limitada flexibilidad de este lenguaje, se emplea la librería *Glslang*. Esta librería permite traducir código escrito en *GLSL*, el lenguaje de *Shaders* utilizado por *OpenGL*, a *SPIR-V*. Este proceso puede realizarse de dos maneras: *online* u *offline*. En este trabajo se emplea compilación *online*, es decir, los *Shaders* se generan en el momento de ejecutar la aplicación.

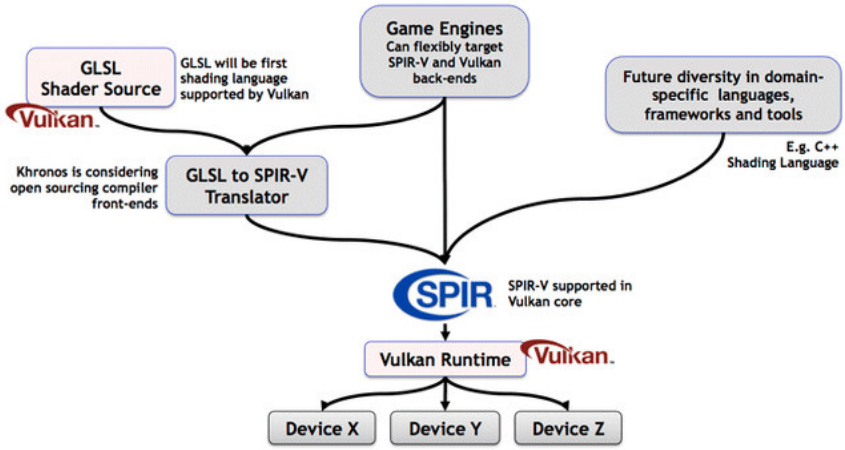


Figura 3.11: Diagrama que muestra la relación entre los shaders en GLSL con *Vulkan*

3.4.2. Núcleo de trazado de rayos

Una vez definido el núcleo de *Vulkan*, es necesario establecer el núcleo de trazado de rayos y su funcionamiento, de manera que quede claramente especificado para su implementación.

El núcleo de trazado de rayos constituye la parte encargada de gestionar todos los procesos relacionados con esta técnica, es decir, organizar y ejecutar las funciones de *Vulkan* asociadas al trazado de rayos. Otros procesos de *Vulkan*, como la creación de búferes o texturas, serán gestionados por el núcleo *Vulkan*, tal y como se ha indicado previamente.

Este núcleo se estructura en torno a la creación de un objeto denominado "*Raytracer*", que contendrá todas las funcionalidades necesarias para el trazado de rayos. En primer lugar, se debe inicializar este objeto. Para ello, se obtendrán las propiedades de trazado de rayos del *Physical device* seleccionado, proporcionadas por el núcleo de *Vulkan*. Estas propiedades incluyen el tamaño de las referencias a los shaders, la profundidad máxima de recursión permitida y la información sobre el formato de la *Shader Binding Table*, entre otros. Esta información se almacenará, ya que resulta esencial tanto para la construcción de los componentes necesarios como para la ejecución del proceso de trazado de rayos.

Durante la inicialización, también se debe crear una imagen de salida donde se escribirá el resultado del trazado de rayos. Es recomendable disponer de una imagen por defecto para evitar errores en caso de una configuración incorrecta de la librería. Como se mencionó anteriormente, esto implica crear una imagen y su correspondiente *Imageview*, ambos gestionados por el núcleo de *Vulkan*.

Construcción de estructuras de aceleración

La siguiente función que debe cumplir este núcleo de trazado de rayos es la creación de las estructuras de aceleración que se utilizarán posteriormente en el renderizado. Para ello, se seguirán una serie de pasos. El primero consiste en transformar la estructura de las mallas propias de la librería a un formato específico requerido por las funciones de *Vulkan* para su procesamiento y creación. Estas nuevas mallas en formato *Vulkan* solo incluirán los vértices y los índices en el momento de la creación, mientras que otros atributos, como las normales o las coordenadas de textura, se pasarán directamente al *Shader* para su acceso durante la ejecución.

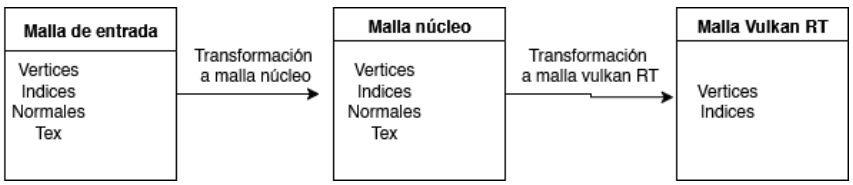


Figura 3.12: Diagrama que muestra el ciclo de vida de las mallas en la aplicación

Una vez convertidas todas las mallas al formato correcto, se podrán construir las *BLAS*. Este proceso es complejo y requiere un procedimiento similar al de crear *Buffers*. Para ello, se realizará lo siguiente:

1. Obtención de los requisitos necesarios para crear las estructuras de aceleración para cada malla.

2. Creación de un *Buffer* auxiliar, con tamaño suficiente para alojar toda la geometría.

3. Asignación del espacio requerido a la estructura de aceleración.

4. Creación de la estructura de aceleración *BLAS* para cada una de las mallas.

5. Construcción (*Build*) de la estructura de aceleración.

El resultado de este proceso será una lista de *BLAS*, una para cada malla en la escena. Cabe destacar el último paso, donde al construir las estructuras de aceleración se incluirán en las *Bounding Box* adecuadas para contenerlas y registrarlas, de modo que aparezcan en la escena.

Continuando con la construcción de las estructuras de aceleración, el siguiente paso consiste en crear una *TLAS* que agrupe todas las *BLAS* de la escena. Se creará una única *TLAS*, dado que la escena será estática y no se requiere separar diferentes partes con transformaciones independientes en el espacio. El primer paso para la creación de la *TLAS*, similar al de las *BLAS*, consiste en convertir la información al formato correcto, en este caso la matriz de transformación de la instancia. Esta matriz se transformará desde la matriz de entrada a un objeto de matriz de transformación *Vulkan*. En este caso, copiar la información de una matriz a otra completará la funcionalidad.

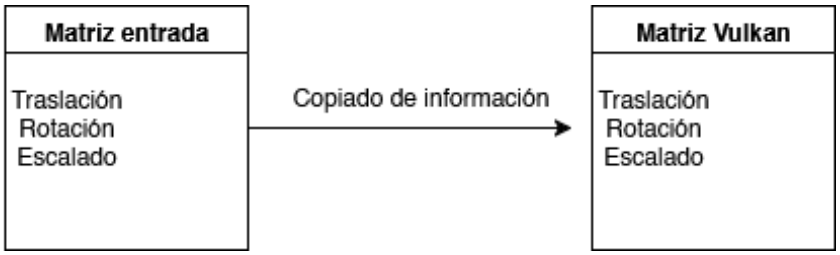


Figura 3.13: Diagrama que muestra el ciclo de vida de las matrices de transformación en la aplicación

A continuación, es necesario crear las instancias que compondrán la *TLAS*. Se generará una instancia por cada *BLAS*, la cual incluirá, además de la matriz de transformación, un índice identificador y las *Flags* pertinentes.

Al igual que con las *BLAS*, se deben construir las estructuras de aceleración a partir de las instancias previamente creadas. El proceso será el siguiente:

1. Creación de un *Buffer* para las instancias.

2. Copia de los datos de instancias al *Buffer*.

3. Configuración de la geometría de las instancias.

4. Preparación y obtención de la información para la creación de la *TLAS*.

5. Creación de un *Buffer* auxiliar y del *Buffer* que utilizará la estructura creada.

6. Creación de la estructura de aceleración.

7. Construcción de la estructura de aceleración, de forma similar a las *BLAS*.

El resultado de este proceso será un único objeto *TLAS*, que contendrá toda la información de la escena en forma de estructura de aceleración y que se pasará a los *Shaders* que lo requieran en tiempo de ejecución.

Creación de *Descriptor sets*

La siguiente función que debe cumplir el núcleo de trazado de rayos es la creación de todos los *Descriptor Sets* necesarios para el correcto funcionamiento de los *Shaders*. Estos propagarán y harán accesible información del programa a los *Shaders* y podrán modificarse en tiempo de ejecución. Se crearán tres *Descriptor Sets* en el núcleo:

- *Descriptor Set* para estructuras de aceleración.
- *Descriptor Set* para matrices de escena.
- *Descriptor Set* para información de mallas.

Cada uno de estos *Descriptor Sets* almacenará información específica y se utilizará en contextos diferentes. El de estructuras de aceleración contendrá la *TLAS* utilizada para el trazado de rayos y la imagen de salida. El de matrices almacenará las matrices de vista y proyección para transformar las coordenadas en la GPU. El de información de mallas incluirá datos como vértices, normales, índices y coordenadas de textura.

El proceso de creación de estos *Descriptor Sets* se repetirá para cada uno y consistirá en los siguientes pasos:

1. Creación de la disposición del *Descriptor Set*.
2. Asignación de la memoria.
3. Escritura de datos en memoria.

El primer paso es el más importante, ya que define la disposición del *Descriptor Set*. Este paso varía para cada set, dado que cada uno contiene información distinta. En esta fase se especifica el tipo de cada atributo y su uso mediante las *Flags* correspondientes.

El siguiente paso consiste en la asignación de memoria, donde se define el espacio que ocupará el *Descriptor Set* y se crea inicialmente vacío.

Una vez creado el *Descriptor Set*, este se poblará. En esta fase se indica tanto la localización como el set que se completará. Por ejemplo, si se desea escribir la información de la imagen de salida, se asignará un índice para su acceso y otro índice para el set completo, permitiendo acceder a la información mediante ambos. Este paso no es obligatorio realizarlo junto con los anteriores, ya que puede ejecutarse posteriormente. Esto resulta útil en casos como la inclusión de información de mallas que no se generan inicialmente, evitando reconstruir los sets o recompilar los *Shaders*.

Creación de *Pipeline*

La siguiente tarea del núcleo de trazado de rayos consiste en crear la *Pipeline* de trazado de rayos y todos los componentes necesarios para su correcto funcionamiento.

Antes de la creación de la *Pipeline*, es necesario definir qué *Shaders* se utilizarán durante el trazado de rayos, ya que deben estar especificados para incluirlos posteriormente con su código correspondiente. En este trabajo se emplearán los siguientes *Shaders*:

- *Raygen Shader*.
- *Miss Shader*.

- *Closest Hit Shader*.

Una vez definidos los *Shaders*, durante la creación de la *Pipeline* se incorporarán y, a partir de ellos, se generarán los diferentes *Stages* por los que se procesarán los rayos, uno para cada *Shader*.

A continuación, se deben incluir en la *Pipeline* los *Descriptor Sets* que se utilizarán con esta. Aquí se incorporarán los definidos anteriormente, de manera que la estructura final será la mostrada en la figura 3.14. Por ejemplo, para acceder a la matriz de proyección será necesario usar el identificador del set (en este caso, 1) y el identificador del *Binding* del atributo (también 1).

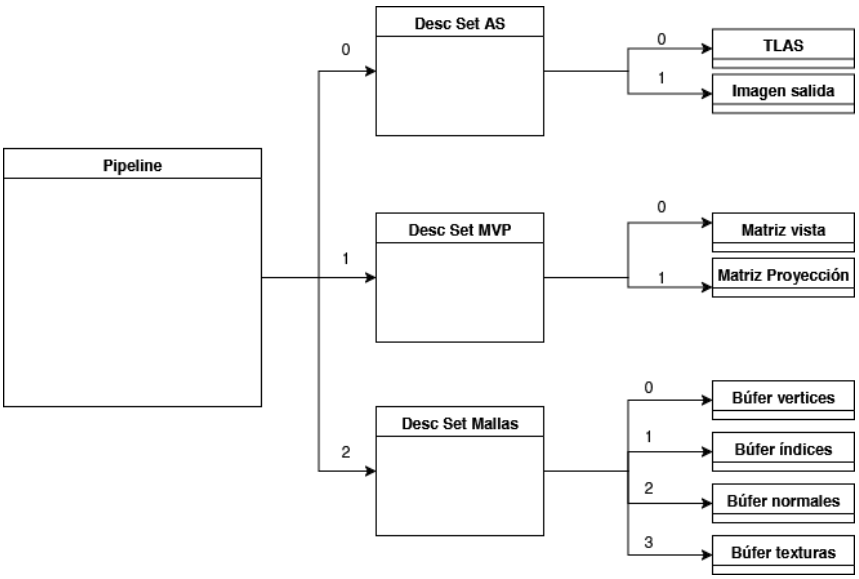


Figura 3.14: Diagrama que muestra la disposición de los *descriptor sets* en la *pipeline*

Para completar la construcción, es necesario definir cierta información adicional, principalmente el nivel de recursión de la *Pipeline*. Este parámetro indica la cantidad de llamadas recursivas, o rebotes, que puede realizar un rayo generado. Con un nivel de recursión igual a 1 no se produce ningún rebote secundario. En la figura 3.15 se muestra, a la derecha, el método utilizado para trazar los rayos aplicando esta recursión. En este trabajo se establecerá un nivel de recursión de 2, lo que permite un rebote intermedio entre el rayo y la fuente de luz, considerando así las oclusiones en el sistema.

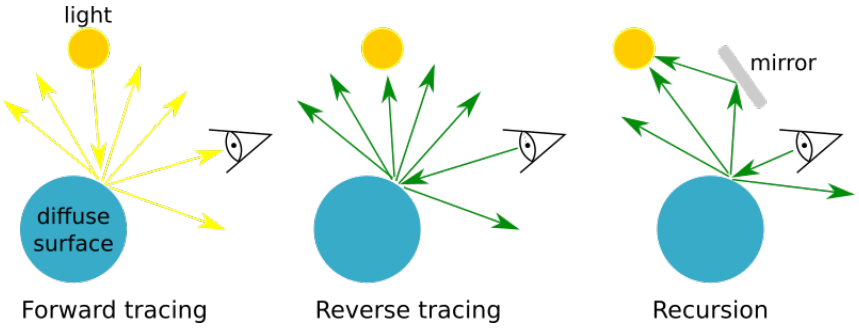


Figura 3.15: Diagrama que muestra los diferentes tipos de trazado de rayos según la recursión y el punto desde que se trazan

Una vez definido el nivel de recursión y con todos los datos previamente generados, se puede crear finalmente la *Pipeline*.

Junto con la creación de la *Pipeline*, el núcleo de trazado de rayos debe encargarse de generar la *Shader Binding Table*, elemento esencial para su correcto funcionamiento. Para su construcción, primero es necesario considerar los *Shaders* que se utilizarán: *Raygen*, *Miss* y *Closest Hit*. Esto es importante porque la tabla debe crearse siguiendo un orden específico, garantizando que, al ejecutarse, se invoque el *Shader* correcto. El orden será el siguiente:

1. *Raygen*.
2. *Miss*.
3. *Closest Hit*.

Una vez definido el orden, se procede a crear el objeto *Shader Binding Table*. Este proceso es sencillo, ya que al generar la *Pipeline* se dispone de toda la información sobre las *Stages* y su tamaño. El procedimiento se realiza de la siguiente forma:

1. Cálculo del tamaño de las *Stages*.
2. Obtención de los identificadores de las *Stages*.
3. Creación de un *Buffer* para almacenar la *Shader Binding Table*.
4. Mapeado y copia de datos al *Buffer*.
5. Configuración de la *Shader Binding Table*.

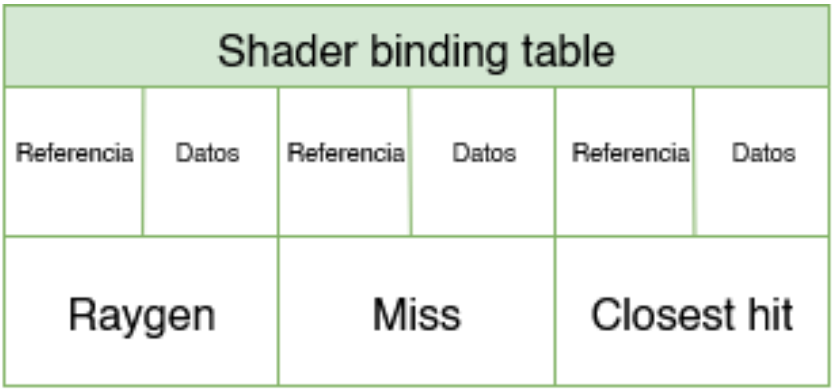


Figura 3.16: Diagrama que muestra la *Shader binding table* que se utilizará para el trazado de rayos

Renderizado

Finalmente, la última tarea del núcleo de trazado de rayos consiste en ejecutar el propio trazado de rayos. Esta operación engloba todo el núcleo y requiere que todas las partes se hayan creado previamente para garantizar su correcto funcionamiento. En concreto, necesita la *Pipeline* y la *Shader Binding Table* para ejecutar el método de trazado de rayos, los *Descriptor Sets* para transferir la información necesaria a los *Shaders* y las estructuras de aceleración para permitir el acceso a la geometría desde los *Shaders*.

El proceso sigue una serie de pasos: primero se determina la imagen de salida, luego se enlazan la *Pipeline* y los *Descriptor Sets*, y finalmente se ejecuta el trazado de rayos.

Además, se implementará un método que permita invocar el trazado de rayos de forma sencilla, dado un tamaño de imagen (ancho × alto). Este método se encargará de configurar y preparar todos los elementos necesarios para la ejecución. Una vez completado el trazado, la imagen podrá almacenarse en un archivo PNG, funcionalidad que se utilizará principalmente durante el desarrollo y no en la versión final de la aplicación.

Shaders

A continuación, se describe el diseño de los *Shaders* utilizados en la librería para el trazado de rayos. Los *Shaders* son archivos externos modificables que definen el comportamiento de la *Pipeline*. Como se indicó anteriormente, se emplearán los *Shaders Raygen*, *Miss* y *Closest Hit*.

El *Shader Raygen* es el encargado de generar los rayos y propagarlos al resto de la *Pipeline*. Cada rayo se define mediante un punto de origen y una dirección, siguiendo la línea que ambos determinan para comprobar si se produce una intersección con la geometría de la escena. Para el correcto funcionamiento del trazado de rayos, los rayos se lanzarán desde la cámara, evaluando el resultado de este proceso. Para ello, es necesario definir adecuadamente tanto el punto de origen como la dirección de cada rayo. En este caso, para trazar un rayo por cada píxel de la pantalla, el origen se situará detrás del centro de la cámara, permitiendo proyectar una línea desde dicho origen hasta cada píxel dentro del campo de visión.

Una vez definido el rayo, se invocará el método que lo crea y ejecuta, determinando qué *Shader* se aplicará en función de si existe intersección con la geometría. El centro de la cámara se ubicará en el punto (0,0,0) en el espacio de vista y, debido al funcionamiento del sistema, será necesario convertir tanto el punto de origen como la dirección del rayo desde el espacio de vista al espacio del mundo. Los rayos también incorporan un rango de validez, es decir, los límites entre los que se considera su efecto. En este trabajo, este rango no se restringirá, ya que se pretende que todos los rayos impacten con geometría o sean descartados. Finalmente, al concluir la recursión, se almacenará en el píxel correspondiente de la imagen el resultado del trazado, representado por un color que dependerá de si el rayo ha impactado o no. Este *Shader* se ejecutará al menos una vez por cada píxel de la imagen de salida, pudiendo incluso lanzar múltiples rayos por píxel.

El *Shader Miss* tendrá una funcionalidad muy simplificada: interactuar con los rayos que no intersecten con ninguna geometría. Por este motivo, se puede considerar que este *Shader* define el color de fondo. No requiere información adicional y devolverá al *Shader* que lo invoque el color correspondiente. Este puede ser el *Shader Raygen*, en cuyo caso el color se almacenará en la imagen de salida para indicar que no hubo intersección, o el *Shader Closest Hit*.

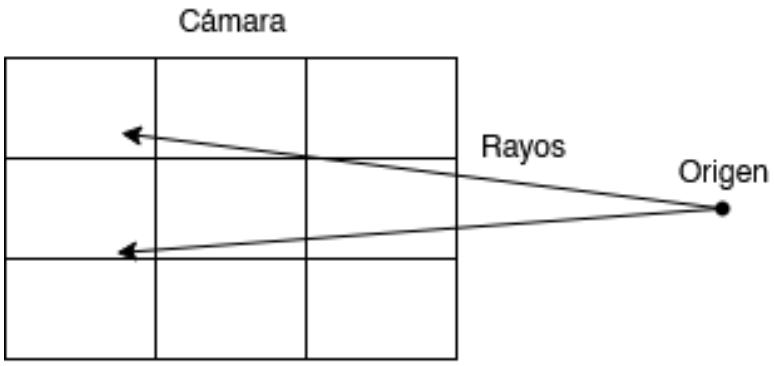


Figura 3.17: Diagrama que muestra el funcionamiento teórico del shader *raygen*

Un ejemplo del comportamiento esperado se muestra en la figura 3.18, donde los píxeles en rojo representan rayos que no intersectaron con ninguna geometría, mientras que los píxeles en azul corresponden a intersecciones procesadas por el *Shader Closest Hit*.



Figura 3.18: Ejemplo de ejecución del trazado de rayos, donde se determinó que el shader de miss devolviera un color rojo

Finalmente, el *Shader Closest Hit* se encarga de completar la funcionalidad principal del trazado de rayos. Su comportamiento es el siguiente: cuando un rayo invoca este *Shader*, significa que ha intersectado con una geometría. En ese caso, se identifica la geometría impactada y se obtiene su normal. Con esta normal, el rayo se refleja y se vuelve a lanzar con la nueva dirección calculada.

A partir de este punto existen dos posibilidades:

1. El rayo reflejado no intersecta con ninguna geometría. En este caso, el punto de intersección inicial se marca con un color predeterminado para indicar que no refleja luz.

2. El rayo reflejado intersecta con otra geometría o con una luz (considerada también como geometría, pero con una textura asociada). Si vuelve a intersectar con una geometría, se intentará reflejar el rayo nuevamente, salvo que se haya alcanzado el límite de recursión, en cuyo caso se devolverá el color predeterminado.

Por otro lado, si el rayo intersecta con una luz, se consultará la textura mediante sus coordenadas para determinar si el punto emite luz. Si el punto es emisivo, no se realizarán más reflexiones y se propagará el color de la luz a los rayos anteriores. De esta manera, todos los puntos que conduzcan a rayos que alcanzan la luz heredarán su color.

3.4.3. Renderer

El *Renderer* constituye la última parte de la librería y se encarga de organizar y contener todas las funciones necesarias que el usuario podrá emplear al utilizarla. Estas funciones se exponen mediante una interfaz que puede ser utilizada por otras librerías para crear aplicaciones con funcionalidad similar, pero basadas en diferentes núcleos, manteniendo la opacidad de su implementación interna.

El *Renderer* desarrollado implementará todos los métodos definidos en la interfaz, que son:

- Init.
- DefineMesh.
- AddMesh.
- AddTexture.
- DeleteTexture.
- AddLight.
- RemoveMesh.
- ClearScene.
- SetCamera.
- SetOutputResolution.
- Render.
- CopyResultBytes.
- GetResultTextureId.

A continuación, se describe el objetivo y funcionamiento de cada uno de estos métodos, dado que será responsabilidad del usuario utilizarlos correctamente para obtener el resultado esperado.

Init

El método *Init* se encarga de configurar y crear todos los objetos necesarios para el correcto funcionamiento del sistema. En concreto, inicializa tanto el núcleo de *Vulkan* como el núcleo de trazado de rayos. Esto implica la creación de la *RenderPass* y los *Framebuffers*, la compilación de los *Shaders*, la creación del *Raytracer* y, por lo tanto, de los *Descriptor Sets*, la *Pipeline* y la *Shader Binding Table*. También se genera la textura de salida como una textura vacía. El método devuelve **True** si la ejecución se realiza correctamente.

DefineMesh

El método *DefineMesh* recibe vértices, normales, índices y coordenadas de textura para crear una malla. En concreto, genera los *Buffers Vulkan* necesarios y construye la malla correspondiente. Una vez creada, se le asigna un identificador único que se devuelve como resultado del método. Las mallas definidas mediante este método se insertan en una lista de mallas candidatas, las cuales no se dibujan inicialmente, pero pueden añadirse al renderizado mediante el siguiente método.

AddMesh

El método *AddMesh* añade mallas candidatas a la escena utilizando una matriz de transformación, un color y el identificador de la malla. Para ello, busca en la lista de mallas candidatas la malla con el identificador proporcionado y, si la encuentra, la incluye en la lista de mallas a dibujar. Todos los vértices y normales de la malla se transforman de espacio de modelo a espacio de mundo multiplicándolos por la matriz de transformación. El método devuelve **True** si la malla se añade correctamente a la escena.

AddTexture

El método *AddTexture* incorpora una textura a la lista de texturas disponibles, que se subirán al *Descriptor Set* correspondiente. Dado un tamaño, un formato y los datos de la textura, el método devuelve un identificador único que permitirá referenciarla.

DeleteTexture

El método *DeleteTexture* elimina una textura de la lista de texturas. Su uso implica la destrucción de la textura, por lo que, para volver a utilizarla, será necesario añadirla nuevamente mediante el método *AddTexture*, obteniendo un nuevo identificador.

AddLight

El método *AddLight* añade una luz a la escena. En el contexto del trazado de rayos, una luz se representa como una malla con una textura asociada que define qué partes emiten luz. El método recibe la matriz de transformación, el identificador de la malla que define la geometría, el color de la luz y el identificador de la textura utilizada. Devuelve **True** si la luz se añade correctamente. Estas luces pueden eliminarse mediante el método *RemoveMesh*.

RemoveMesh

El método *RemoveMesh* elimina mallas de la lista de mallas a dibujar. Recibe un identificador y, si existe una malla con dicho identificador, la elimina de la lista. La malla permanece en la lista de mallas candidatas para un uso posterior. Devuelve **True** si la eliminación se realiza correctamente y **False** en caso contrario.

ClearScene

El método *ClearScene*, de forma similar a *RemoveMesh*, elimina todas las mallas de la lista de mallas a dibujar. Estas mallas permanecen en la lista de mallas candidatas.

SetCamera

El método *SetCamera* recibe las matrices de vista y proyección y las sube al *Descriptor Set* correspondiente. Estas matrices se utilizan para transformar vértices y normales, otorgando la perspectiva deseada. Es necesario ejecutar este método antes de llamar a *Render* para garantizar el correcto funcionamiento de la aplicación.

SetOutputResolution

El método *SetOutputResolution* crea la imagen de salida con las dimensiones especificadas. Esto implica la creación de la imagen *Vulkan* y su correspondiente *ImageView*. Es necesario ejecutar este método antes de *Render* para el correcto funcionamiento de la aplicación.

Render

El método *Render* invoca la funcionalidad de renderizado del núcleo de trazado de rayos. Devuelve **True** si la ejecución se realiza correctamente. Para su uso, es necesario haber ejecutado previamente los métodos *Init*, *SetCamera* y *SetOutputResolution*. El resultado del trazado de rayos se almacena en la imagen de salida, que se podrá recuperar mediante otros métodos.

CopyResultBytes

El método *CopyResultBytes* copia toda la información de la imagen de salida a un *Buffer* proporcionado como parámetro. Además del *Buffer*, se debe indicar el tamaño disponible. Si el tamaño del *Buffer* es menor que el de la imagen, el resultado no será válido. El método devuelve el tamaño de los datos copiados. Debe ejecutarse después de *Render*.

GetResultTextureId

El método *GetResultTextureId* convierte la imagen de salida del trazado de rayos en una textura de *OpenGL*. Para ello, utiliza las funcionalidades de interoperabilidad entre *Vulkan* y *OpenGL* mediante extensiones específicas. Internamente, crea una textura en *OpenGL* y copia en ella el contenido de la imagen de salida. El método devuelve el identificador de la textura en *OpenGL*, que no guarda relación con los identificadores mencionados anteriormente.

3.5 Tecnología utilizada

A continuación, se describen las tecnologías, programas y librerías empleadas en el desarrollo del trabajo. Estas herramientas son esenciales para el desarrollo y permiten reutilizar soluciones ya implementadas previamente.

En este contexto, se han utilizado principalmente las siguientes librerías:

1. *Vulkan*.
2. *Glslang*.
3. *GLM*.
4. *OpenGL*.
5. *GLEW*.

Vulkan

Vulkan es una API para gráficos 3D de bajo nivel que ofrece al programador un alto grado de control sobre la configuración del sistema, permitiendo crear soluciones altamente especializadas a costa de una mayor complejidad y riesgo de errores. Por ello, es necesario desarrollar una base robusta para garantizar un funcionamiento correcto. Esta API es la sucesora de *OpenGL*, también desarrollada por *Khronos Group*, tras el cambio de paradigma introducido en 2015 con la descontinuación de *OpenGL*.

En este trabajo, *Vulkan* se utiliza para crear y configurar la mayor parte del simulador, desde la inicialización de la instancia hasta la implementación del trazado de rayos. La versión empleada es la 1.2, requisito mínimo para soportar trazado de rayos, ya que no estaba disponible en versiones anteriores.

Glslang

Glslang es una librería en C++ que traduce diferentes lenguajes de shading, como GLSL o HLSL, al formato requerido por *Vulkan*: SPIR-V. Esta funcionalidad es esencial, dado que SPIR-V es un lenguaje de bajo nivel similar a ensamblador, lo que dificulta su programación directa. La librería compila el código GLSL y lo transforma a SPIR-V mediante varias etapas: preprocesado, análisis sintáctico, enlazado, detección de *Stage* y generación.

En este trabajo, *Glslang* se utiliza para traducir los *Shaders Raygen*, *Miss* y *Closest Hit* de GLSL a SPIR-V en tiempo de ejecución (*online*), en lugar de hacerlo de forma previa (*offline*). Además, permite configurar la versión de SPIR-V, que en este caso debe ser 1.4 o superior para garantizar la compatibilidad con el trazado de rayos.

GLM

OpenGL Mathematics (*GLM*) es una librería que proporciona estructuras y operaciones matemáticas en C++, como vectores, matrices y funciones asociadas (multiplicación, inversión, etc.). Aunque está orientada a aplicaciones basadas en *OpenGL*, puede utilizarse de forma independiente. Su sintaxis está inspirada en GLSL. El repositorio oficial se encuentra en //github.com/g-truc/glm.

En este trabajo, *GLM* se emplea principalmente en el *Renderer* para gestionar entradas de vértices, normales y operaciones con matrices, como transformaciones y proyecciones.

OpenGL

OpenGL es una API para gráficos 3D de bajo nivel que define un conjunto de estándares para garantizar la interoperabilidad entre hardware y software. Fue introducida en 1992 y se considera el precursor de *Vulkan*, al ser desarrollada también por *Khronos Group*. A diferencia de *Vulkan*, se considera de nivel más alto, ya que incluye más funcionalidades predefinidas.

En este trabajo, *OpenGL* se utiliza en dos contextos:

- Implementación de un *Renderer* alternativo para comparar su rendimiento con el desarrollado en *Vulkan*.
- Creación y configuración de la salida del *Renderer* basado en *Vulkan*, generando una textura *OpenGL* a partir de la imagen producida por el trazado de rayos.

GLEW

GLEW es una librería que gestiona extensiones de *OpenGL*, permitiendo incluir dinámicamente las funcionalidades necesarias sin intervención manual.

En este trabajo, *GLEW* se utiliza para habilitar las funciones requeridas para exportar la imagen desde *Vulkan* a *OpenGL*, incluyendo operaciones específicas del sistema operativo y la gestión de memoria compartida entre librerías.

CAPÍTULO 4
Desarrollo de la solución propuesta

A continuación, se describe el proceso de creación de la librería especificada previamente. Este capítulo incluye también las decisiones tomadas durante el desarrollo y su justificación, debido a los problemas que surgieron en el transcurso del trabajo.

4.1 Núcleo *Vulkan*

El primer paso en el desarrollo de la librería fue la creación de un nuevo proyecto en *Visual Studio*. Este proyecto sirvió como base para implementar todas las funcionalidades necesarias para completar el simulador de detección de defectos mediante trazado de rayos. El proyecto debe incluir referencias a todas las librerías mencionadas anteriormente para garantizar su correcto funcionamiento, aunque durante la mayor parte del desarrollo la única referencia necesaria fue la de *Vulkan*.

Una vez creado el proyecto, el siguiente paso consistió en la creación del núcleo de *Vulkan* y su instancia. La creación de la instancia es siempre el primer paso al utilizar *Vulkan* en cualquier dispositivo. Para ello, se requiere al menos: el nombre de la aplicación, la versión de la API, las capas adicionales y las extensiones adicionales. Las capas adicionales proporcionan funcionalidades extra, como herramientas de depuración. En este caso, se incluyó la capa de validación desarrollada por *Khronos Group*, que ofrece información sobre advertencias y errores durante la ejecución. Las extensiones adicionales amplían las funcionalidades disponibles, permitiendo el uso de métodos adicionales en *Vulkan*. Inicialmente, se añadieron extensiones para depuración y manejo de la *Surface*, aunque estas últimas se descartaron posteriormente por no ser necesarias en el resultado final.

Más adelante, se incorporaron extensiones adicionales tanto a nivel de instancia como de *Device* para habilitar funcionalidades específicas y garantizar el correcto funcionamiento del sistema. Una vez creada, la instancia se almacena para su uso posterior, principalmente en la creación de los *Physical Devices* y el *Device* lógico.

Antes de continuar, se consideró fundamental añadir mecanismos adicionales de validación para detectar y corregir errores durante el desarrollo. Para ello, se creó un *DebugUtilsMessenger*, una estructura de *Vulkan* que detecta automáticamente errores y los muestra al desarrollador, dado que por defecto *Vulkan* no genera mensajes de error. Este objeto se configuró para mostrar mensajes de:

1. Información general.

2. Advertencia.

3. Error.

Estos mensajes abarcan validaciones generales y advertencias sobre eficiencia, como liberación incompleta de memoria. Cada mensaje incluye: descripción del error, nivel de severidad, tipo de error y el objeto afectado.

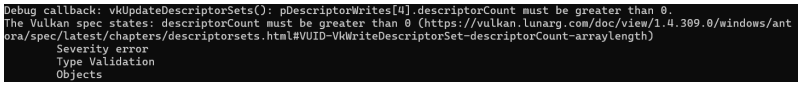


Figura 4.1: Ejemplo de un error mostrado en la consola

Una vez configurado el sistema de detección de errores, se procede a la creación de los *Physical Devices*. Tras invocar el método correspondiente, se obtienen los dispositivos físicos disponibles en el equipo. Es fundamental identificar las GPUs presentes, ya que el trazado de rayos no está soportado por todas ellas.

De cada dispositivo se consultan las siguientes propiedades: versión de los *Drivers*, número de familias de *Queues* y sus características, cantidad de *Queues* por familia y *Memory Heaps* disponibles, tipos de memoria soportados y sus propiedades.

Esta información permite determinar si la GPU está correctamente configurada y si soporta trazado de rayos, en función de la versión de los *Drivers*. Además, el análisis de las familias de *Queues* es relevante para seleccionar la más adecuada, ya que se requiere al menos una que soporte operaciones gráficas. Si también admite operaciones adicionales, como transferencia o *Sparse Binding*, se considera una ventaja.

El número de *Queues* disponibles influye en la capacidad de paralelización del programa: a mayor cantidad, mayor paralelismo. Por otro lado, los *Memory Heaps* se analizan para evitar incompatibilidades en los formatos de memoria. En etapas iniciales del proyecto también se verificó la compatibilidad con la presentación en pantalla, aunque esta funcionalidad se descartó posteriormente.

En general, la librería detecta todas las GPUs disponibles, las almacena en una lista y, en función de las *Flags* proporcionadas, selecciona el *Physical Device* adecuado.

Siguiendo este procedimiento, se obtuvieron y seleccionaron los *Physical Devices*, desarrollándose todo el trabajo con la GPU mostrada en la figura 4.2.

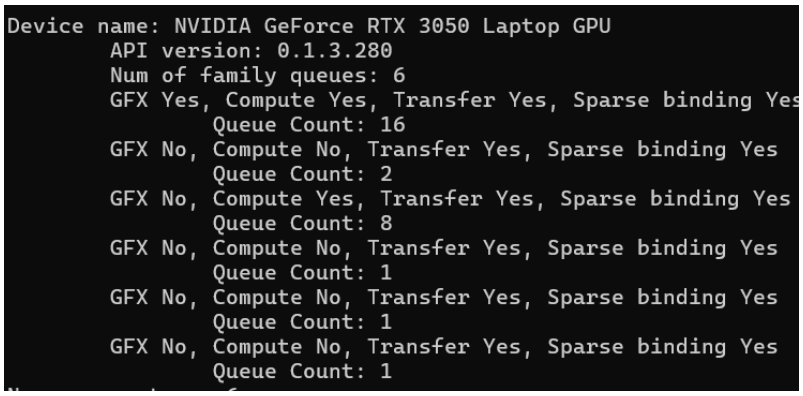


Figura 4.2: *Physical device* seleccionado para el desarrollo del trabajo

A continuación, se creó el *Device* lógico a partir del *Physical Device* seleccionado previamente. Este paso es uno de los más importantes en el núcleo de *Vulkan*, ya que todas las funciones de la API se ejecutan a través del *Device*.

En primer lugar, se asignaron la familia y las *Queues* que se utilizarán durante la ejecución, previamente seleccionadas. Posteriormente, se añadieron las extensiones necesa-

rias. En esta etapa inicial, las extensiones eran pocas y estaban orientadas a proporcionar capacidades adicionales para el dibujado y el *Shading*. Sin embargo, en fases posteriores se incorporaron múltiples extensiones específicas, que se detallarán en el apartado correspondiente al núcleo de trazado de rayos. Una vez configurada toda la información relevante, se creó el *Device*. Durante la ejecución, se utilizó un único *Device*.

Un aspecto crítico considerado en todas las estructuras creadas hasta este punto fue la gestión de memoria. En *Vulkan*, el programador es responsable de crear y destruir los objetos, asegurando la liberación de memoria para evitar fugas. Por ello, todos los objetos generados se destruyen al final del ciclo de vida del programa, incluyendo los objetos *Vulkan*, su memoria asociada y los objetos de acceso. Tanto el *Device* como la instancia deben destruirse explícitamente, asegurando que todos los objetos dependientes se eliminen antes. Los *Physical Devices* no requieren destrucción, ya que *Vulkan* los gestiona como punteros.

La siguiente tarea consistió en la creación y configuración de las *Queues*. Estas existen físicamente en la GPU, pero requieren la creación de un objeto *Queue* en la CPU para enviar tareas. Las *Queues* se encargan de enviar información y comandos a la GPU para su ejecución. Dado el alto grado de paralelismo en *Vulkan*, se implementaron semáforos para sincronizar las operaciones. Estos semáforos son gestionados por *Vulkan* y solo requieren configuración inicial por parte del programador.

Finalmente, se inicializaron las *Queues* necesarias y se prepararon para diferentes tareas, como la transferencia de datos a la GPU o la ejecución del trazado de rayos. Por lo tanto, cuando se mencionen estas operaciones más adelante, se asumirá que se realizan mediante las *Queues*, incluso si no se indica explícitamente.

Continuando con el desarrollo, la siguiente tarea consistió en la creación del *RenderPass*. Este objeto define cómo trabajará la *Pipeline* durante la ejecución, especificando el flujo a través de las diferentes *Stages*, la interacción entre ellas y el destino de los resultados.

Un *RenderPass* está compuesto por *Attachments*, que indican al *Driver* cómo debe gestionarse el proceso. Este objeto contiene información sobre el almacenamiento y carga de las imágenes, así como los formatos iniciales y finales. Además, se indica el tipo de *Pipeline* a utilizar, en este caso, la de trazado de rayos. Se ejecutará un único pase o subpase, de manera que toda la geometría se procese una sola vez. Finalmente, el resultado se almacenará en una imagen de color, ignorando atributos como la profundidad o el valor del *Stencil*. Esta imagen será el resultado de la ejecución de la *Pipeline*.

Junto con el *RenderPass*, es necesaria la creación de *Framebuffers* para complementar su funcionamiento y almacenar los recursos requeridos. Los *Framebuffers* deben contener, como mínimo, la imagen resultante del *RenderPass*, aunque pueden incluir información adicional para su uso durante la ejecución. En concreto, almacenan los *ImageViews* de las imágenes, que son los objetos que permiten acceder a su contenido. Solo se creará una imagen para el *Framebuffer*, correspondiente a la imagen de salida donde se escribirá el resultado del pase de la *Pipeline*. Esta imagen se genera a partir de la altura y anchura especificadas previamente. Finalmente, se crea el objeto mediante la llamada al método correspondiente de *Vulkan*.

Como se mencionó anteriormente, se optó por implementar *Offscreen Rendering*, es decir, generar la imagen sin presentarla directamente en pantalla, para mostrarla posteriormente de forma controlada. Esto se logra tomando la imagen del *Framebuffer* una vez completada la ejecución de la *Pipeline*.

Creación de *Buffers*

El siguiente paso consistió en crear los métodos necesarios para generar los diferentes *Buffers* requeridos durante el desarrollo de la librería. Estos *Buffers* son objetos de *Vulkan* que se envían a la GPU para realizar las operaciones correspondientes, optimizando el rendimiento frente a la ejecución en CPU.

El proceso de creación de un *Buffer* sigue una serie de pasos específicos:

- Creación del objeto *Buffer*.
- Obtención de los requisitos de memoria.
- Asignación del espacio en memoria.
- Enlace del espacio asignado al *Buffer*.

En primer lugar, se crea un objeto *Buffer* vacío a partir del tamaño requerido y las *Flags* de uso. A continuación, se obtienen los requisitos de memoria del *Buffer*, que incluyen el tamaño en bytes, el alineamiento (debido a la posible fragmentación) y los tipos de memoria permitidos. Con esta información, se asigna el espacio necesario indicando el tamaño, el tipo de memoria y las *Flags* correspondientes. Finalmente, se enlaza la memoria al *Buffer*, estableciendo la relación entre ambos.

Una vez implementada esta función, se extendió para crear métodos específicos que generen *Buffers* para vértices, normales, índices y coordenadas de textura. El procedimiento es similar, variando únicamente las *Flags* y el tamaño de los atributos. Se optó por implementar métodos independientes para facilitar el desarrollo y la depuración.

El flujo general es el siguiente:

1. Se crea un *Buffer* auxiliar con la *Flag* de origen de transferencia, se mapea y se copian los datos (por ejemplo, los vértices).
2. Se crea el *Buffer* final con la *Flag* de destino de transferencia.
3. Se realiza la copia del *Buffer* auxiliar al final y se libera el auxiliar.

Este enfoque evita mantener recursos innecesarios, ya que el *Buffer* final no queda mapeado a la memoria de la CPU, eliminando la necesidad de conservar copias adicionales.

En cuanto a las *Flags*, en el contexto del trazado de rayos se deben incluir tres adicionales:

- *Input* para estructuras de aceleración, permitiendo el acceso desde los *Shaders*.
- *Storage Buffer*, que define cómo se accede al *Buffer* en el *Shader*.
- Acceso a la dirección del *Buffer*, necesario para el trazado de rayos.

Por último, el *Buffer* de índices debe incluir la *Flag* específica para índices, además de indicar el tipo de índice para que el *Driver* interprete correctamente su disposición en memoria.

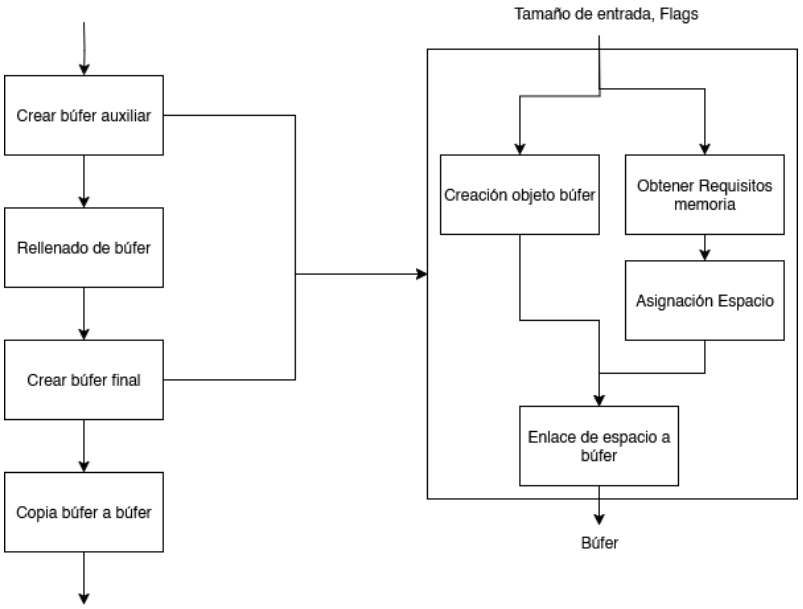


Figura 4.3: Esquema representativo del proceso de creación de búferes.

Manejo de texturas e imágenes

A continuación, se desarrolló toda la funcionalidad relacionada con el manejo de texturas. En este trabajo, las texturas se crean a partir de los píxeles que las componen, junto con su tamaño (altura × anchura) y los bytes por píxel. En el contexto de *Vulkan*, una textura no difiere de una imagen, ya que está compuesta por un objeto imagen, su correspondiente *ImageView* y la memoria asignada.

El proceso de creación de imágenes sigue los mismos pasos descritos para los *Buffers*:

- Creación del objeto imagen.
- Obtención de los requisitos de memoria.
- Asignación de memoria.
- Enlace de la memoria al objeto.

La diferencia principal respecto a los *Buffers* es que, para crear una imagen, se debe proporcionar información adicional, como:

- Tipo de imagen.
- Formato.
- Dimensiones (altura × anchura).
- Profundidad.
- Niveles de *Mipmapping*.
- *Flags* utilizadas.

En este trabajo, todas las imágenes se definieron como 2D, con profundidad y nivel de *Mipmapping* igual a 1, ya que no se pretende explorar estas características. Además, se configuraron los atributos necesarios para permitir la exportación de imágenes a *OpenGL*.

Tras crear cada imagen, se genera su correspondiente *ImageView*, que permite acceder a la imagen sin requerir información adicional. Finalmente, si la imagen se utiliza como textura y se requiere *Sampling*, es necesario crear un *Sampler*. En *Vulkan*, este es un objeto independiente que debe configurarse indicando filtros para minimización, maximización y modos de extensión. En este trabajo, todos los *Samplers* se configuraron con filtros lineales y extensión de repetición, dado que no se buscaba experimentar con otras configuraciones.

Con este procedimiento, se habilita la creación de texturas e imágenes para su uso en la librería.

Creación de funcionalidades de utilidad

Tras esto, se implementaron diversas funcionalidades generales utilizadas a lo largo de todo el desarrollo:

- Copia de *Buffer* a *Buffer*.
- Copia de *Buffer* a imagen.
- Copia de imagen a *Buffer*.
- Copia de imagen a imagen.
- Copia de *Buffer* a memoria.
- Copia de imagen a memoria.
- Transición de formato de imagen.

La implementación de la mayoría de estos métodos sigue un patrón común: se especifica el origen y el destino de la operación, así como el formato de ambos componentes para garantizar una copia correcta.

Un aspecto relevante es la gestión de imágenes. En *Vulkan*, las imágenes se organizan en diferentes formatos para optimizar cada tipo de operación. Por ejemplo, si una imagen se utiliza como origen en una copia, debe estar en el formato adecuado para dicha operación. Por este motivo, antes de ejecutar cualquier método que involucre imágenes, es necesario realizar una transición al formato óptimo. Una vez completada la operación, las imágenes se devuelven a un formato general mediante el mismo procedimiento, evitando inconsistencias.

Otro punto importante es que cualquier operación ejecutada en la GPU o entre GPU y CPU (exceptuando la carga inicial de datos) debe realizarse mediante un objeto *Vulkan* denominado *Command Buffer*. Un *Command Buffer* es un registro de instrucciones que se envía a la *Queue* correspondiente para su ejecución. Estos objetos se crean a partir de un *Command Pool* y pueden ser de un solo uso o reutilizables. En este trabajo, se emplearon principalmente *Command Buffers* de un solo uso, que se destruyen tras su ejecución.

En concreto, los *Command Buffers* se utilizan para todas las operaciones de copia entre objetos *Vulkan*, como las mencionadas anteriormente. Durante el resto de la memoria, cada vez que se haga referencia a operaciones de copia o similares, se asume la creación, configuración y envío de un *Command Buffer* a la *Queue* correspondiente para su ejecución.

Creación de mallas

El siguiente punto desarrollado tras las funciones de utilidad fue la construcción de mallas en el núcleo. Estas mallas son objetos que contienen toda la información necesaria

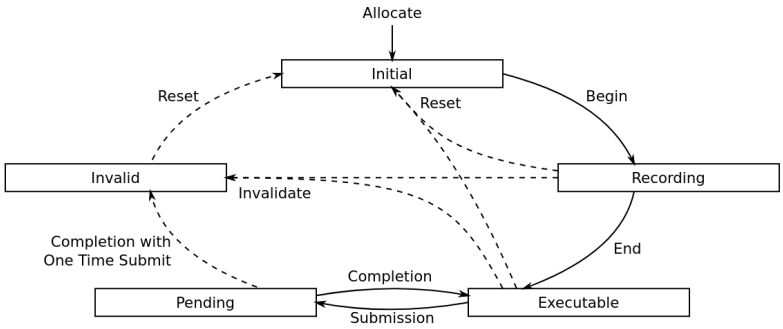


Figura 4.4: Esquema representativo del ciclo de vida de un *Command Buffer* de Especificación Vulkan.

para ser utilizadas y renderizadas mediante trazado de rayos. Cada malla incluye los siguientes atributos, evitando la necesidad de utilidades adicionales para su uso:

- *Buffer* de vértices.
- *Buffer* de índices.
- *Buffer* de normales.
- *Buffer* de coordenadas de textura.
- *Buffer* de color.
- Tamaño asociado a cada *Buffer*.
- Número total de vértices.
- Tipo de los índices.
- Identificador de la malla.
- Identificador de la textura utilizada.
- Matriz de modelo asociada.
- Copia de vértices y normales.

Los *Buffers* de vértices, índices, normales y coordenadas de textura son necesarios para transferir esta información a los *Shaders*. El *Buffer* de color almacena el color predeterminado de la malla, utilizado para aplicar un color plano.

El tamaño asociado a cada *Buffer* se almacena por dos razones:

1. Es necesario para la creación de los *Buffers*.
2. Se utiliza en la construcción de las estructuras de aceleración.

Este tamaño se calcula como el producto del tamaño de un elemento por el número de elementos. Los vértices, normales y color son vectores de tres componentes *float*; las coordenadas de textura son vectores de dos *float*; y los índices son elementos del tipo especificado (normalmente *unsigned int*).

Dado que las mallas se procesan en modo indexado, el número de vértices corresponde al número de índices, no al tamaño del *Buffer* de índices. Los identificadores de la malla y la textura se almacenan únicamente como referencia para el *Renderer*.

Finalmente, se conserva una copia de los vértices y normales antes de crear los *Buffers*, ya que serán necesarios para otras operaciones, junto con la matriz de modelo asociada. El objetivo de este objeto malla es proporcionar una estructura intermedia entre los datos del usuario y los requeridos por el trazado de rayos, permitiendo una transformación estándar y evitando adaptaciones personalizadas.

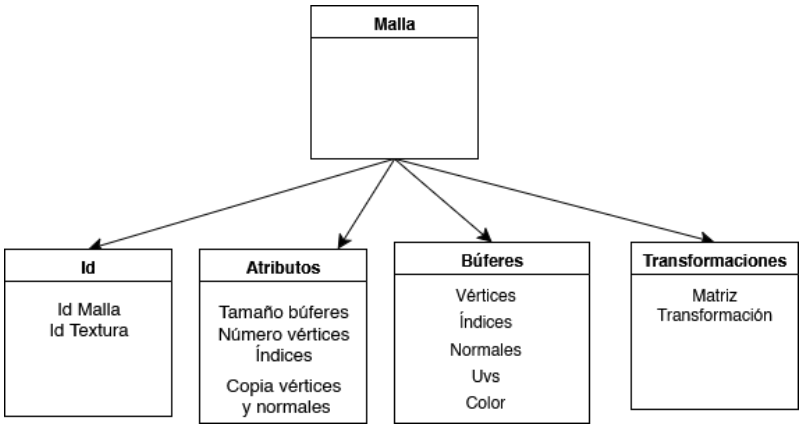


Figura 4.5: Esquema representativo del contenido de una malla.

Creación de *Shaders*

La última tarea del núcleo *Vulkan* consiste en la creación y gestión de *Shaders*. Estos son piezas de código esenciales para el funcionamiento de la librería, ya que definen cómo se representarán en pantalla las mallas creadas. El objetivo de esta tarea es implementar una función que, a partir de una ruta de archivo, genere un objeto *Shader* de *Vulkan*, que posteriormente se utilizará junto con la *Pipeline* para el procesamiento de la geometría.

Para ello, se empleó la librería *Glslang*, que permite traducir *Shaders* escritos en GLSL a SPIR-V. Esta decisión se tomó porque SPIR-V es un lenguaje de bajo nivel, más complejo y verboso, lo que ralentizaría el desarrollo si se escribiera directamente. El uso de *Glslang* introduce únicamente un tiempo adicional mínimo durante el inicio de la ejecución, sin afectar al rendimiento real.

El proceso de compilación de un *Shader* incluye:

1. Construcción de una entrada con la información necesaria (versión objetivo, versión del cliente y lenguaje de origen).
2. Ejecución de dos pasos iniciales: preprocesamiento y análisis sintáctico.
3. Traducción completa del código a SPIR-V.

Una vez compilado, se crean objetos que almacenan tanto el código SPIR-V como el objeto *Vulkan*, para su reutilización si fuera necesario. Finalmente, se invoca la función correspondiente de *Vulkan* para generar el objeto *Shader*.

Tras implementar todas las funcionalidades del núcleo *Vulkan*, se desarrolló una aplicación de prueba para verificar su correcto funcionamiento. Esta aplicación integraba todos los componentes descritos, además de otros elementos no relevantes para el desarrollo final, como la gestión de la presentación en pantalla. Un ejemplo de la salida generada por esta aplicación se muestra en la figura 4.7.

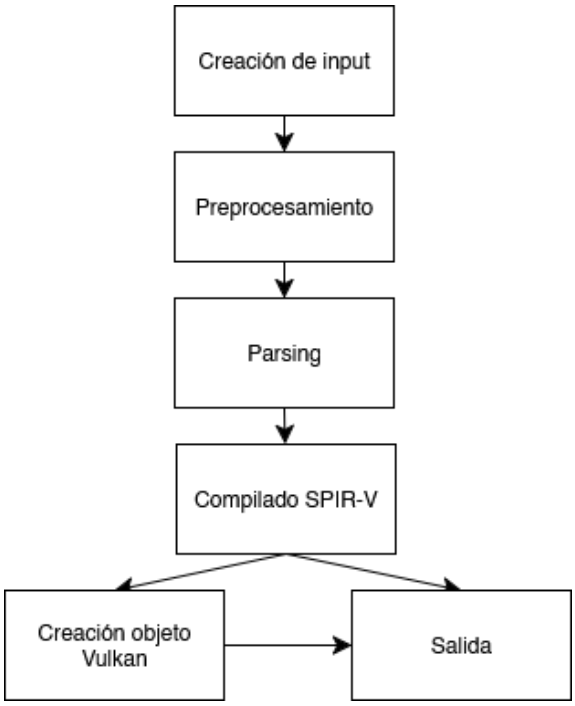


Figura 4.6: Esquema representativo del proceso de creación de un shader.

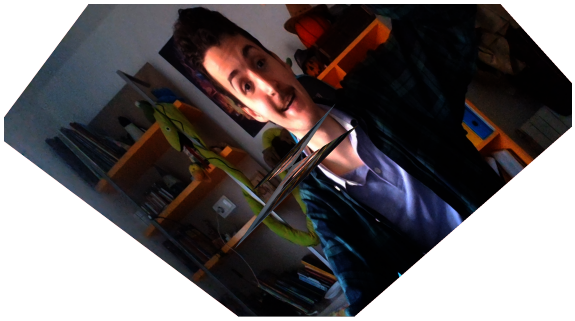


Figura 4.7: Captura de la aplicación de pruebas, mostrando un par de mallas dibujadas con una textura aplicada.

4.2 Núcleo de trazado de rayos

El siguiente paso en el desarrollo para completar las funcionalidades previstas de la librería fue la creación del núcleo de trazado de rayos, encargado de generar imágenes mediante esta técnica y gestionar las funcionalidades necesarias para su ejecución.

Antes de implementar las funciones específicas del trazado de rayos, fue necesario realizar ampliaciones en el núcleo de *Vulkan* para habilitar las extensiones correspondientes. En concreto, se activaron varias extensiones del *Device* y se obtuvieron los métodos asociados a estas.

Las extensiones requeridas amplían las capacidades en los siguientes ámbitos:

- Creación y gestión de estructuras de aceleración.
- Implementación de la *Pipeline* de trazado de rayos.
- Soporte para controles avanzados de SPIR-V.
- Acceso a direcciones de *Buffers*.
- Funcionalidades y *Queries* relacionadas con rayos.

Estas extensiones deben cargarse tanto al crear el *Device* como en la configuración adicional posterior. En detalle, permiten:

- Crear los diferentes tipos de estructuras de aceleración y *Pipeline*.
- Acceder a la versión 1.4 de SPIR-V, necesaria para el trazado de rayos.
- Obtener direcciones de *Buffers*, requisito fundamental para esta técnica.
- Gestionar el paso de información mediante rayos.

Además, es necesario importar las funciones específicas de trazado de rayos. Para ello, *Vulkan* proporciona un mecanismo que permite obtener punteros a las funciones registradas en la instancia, lo que posibilita su invocación directa. De esta manera, se accede a las funciones críticas para implementar el trazado de rayos en esta parte del desarrollo.

Una vez añadidas las extensiones pertinentes al *Device*, se procede a la construcción del sistema de trazado de rayos. El primer paso consiste en ampliar las funcionalidades implementadas para los *Physical Devices*. En concreto, es necesario volver a obtener las propiedades de dichos dispositivos para solicitar a *Vulkan* la información específica del trazado de rayos.

Estas propiedades se almacenan porque son esenciales para la creación y ejecución de operaciones en la *Pipeline*. Entre los datos más relevantes se incluyen: la profundidad máxima de recursión de rayos, la cantidad máxima de rayos a lanzar y los requisitos de alineamiento de los diferentes grupos de *Shaders* para la construcción de la *Shader Binding Table*.

Tras recopilar las capacidades de la GPU para el trazado de rayos, se crean otros componentes necesarios como la imagen de salida de la *Pipeline*. Para evitar errores en caso de ejecución prematura, se genera una imagen de salida inicial. Esta imagen será únicamente de color, con una resolución base de 800×800 píxeles, aunque se incluyen funcionalidades para definir otras resoluciones. La imagen se crea mediante el núcleo *Vulkan* y se reutiliza en todas las operaciones de la *Pipeline*, salvo que se cree una imagen diferente.

Creación de estructuras de aceleración

La siguiente funcionalidad desarrollada del *Raytracer* fue la creación y configuración de las estructuras de aceleración: *BLAS* y *TLAS*. Estas estructuras representan la geometría sobre la que se lanzarán los rayos para comprobar intersecciones. *Vulkan* define la estructura y el proceso para su creación, aunque la implementación se desarrolló desde cero. Este procedimiento es extenso y debe ser robusto para evitar errores posteriores.

En términos generales, el proceso se divide en dos pasos principales: la creación de las múltiples *BLAS* y la creación de una o varias *TLAS* a partir de las anteriores.

Aunque son solo dos pasos, ambos implican múltiples tareas intermedias, por lo que se abordaron de forma secuencial.

El flujo para crear una *BLAS* se resume en:

1. Conversión de formato de las mallas.
2. Finalización de la geometría.
3. Construcción de la *BLAS*.

El primer paso consiste en traducir las mallas al formato requerido por *Vulkan*. Para ello, y para otras operaciones posteriores, es necesario obtener la dirección en memoria de los *Buffers* creados, lo que forma parte de las extensiones habilitadas previamente. Se requiere la dirección del *Buffer* de vértices y del de índices, junto con el tamaño de cada vértice, el tipo de índice y la cantidad total de vértices.

Con esta información, se crean objetos *Vulkan* que contienen la geometría y los parámetros de construcción, como el primer vértice, el número de vértices, la primera instancia y el desplazamiento de la transformación (si existe). Respecto a las transformaciones, se decidió no subirlas a la GPU, sino aplicarlas en la CPU antes de la construcción de las estructuras de aceleración. Esta decisión se tomó para evitar cálculos adicionales en tiempo de ejecución, dado que el objetivo del proyecto no contempla escenas dinámicas. Así, las transformaciones se aplican una sola vez por cambio, a costa de un tiempo de carga ligeramente mayor, considerado irrelevante frente a los beneficios obtenidos.

Con este procedimiento, todas las mallas se transforman al formato requerido. El resto de atributos se proporcionará a la *Pipeline* mediante otros mecanismos.

Una vez transformadas todas las mallas desde su estado original a la especificación *Vulkan*, se requiere un paso adicional previo a la construcción de las estructuras de aceleración: finalizar la geometría. Este proceso permite calcular el tamaño real que ocupará la estructura de aceleración en función del número de primitivas a dibujar, el tipo de atributos que las definen, su ubicación en memoria y las *Flags* asociadas. En este caso, se ha especificado la *Flag* que prioriza la velocidad en el trazado de rayos. También existen otras *Flags*, como permitir la compactación de estructuras de aceleración, reducir el uso de memoria o acelerar el tiempo de construcción.

El paso final para crear una *BLAS* consiste en construirla a partir de la información obtenida en los pasos anteriores. Esta tarea se realiza siguiendo los siguientes pasos:

1. Preparar la información de construcción y calcular el tamaño del búfer auxiliar.
2. Crear el búfer auxiliar.
3. Crear y construir cada *BLAS*.
4. Liberar el búfer auxiliar.

El primer paso consiste en construir una estructura que contenga toda la información esencial para crear las estructuras de aceleración. Esta información incluye la geometría y el tamaño de cada malla a dibujar, así como un objeto que represente la geometría completa y su tamaño total. Estos últimos se utilizan para almacenar una referencia intermedia al conjunto de la estructura de aceleración y definir cómo debe ser creada. El segundo paso es la creación de un búfer auxiliar, donde se escribirá inicialmente el contenido de las estructuras de aceleración. Este búfer debe tener el tamaño suficiente para almacenarlas y actúa como paso intermedio entre la información en CPU y la información final escrita en la GPU.

A continuación, se procede a crear cada una de las estructuras de aceleración. Por cada malla, se crea un nuevo búfer con el tamaño adecuado y se construye la *BLAS* mediante el método *Vulkan*. Tanto el búfer como la estructura de aceleración se almacenan para su uso posterior. Una vez creada la estructura, se realiza un paso de configuración para finalizar su construcción. Finalmente, se libera el búfer auxiliar, ya que no es necesario una vez completado el proceso.

Durante el desarrollo de esta parte se prestó especial atención a cada paso, dado que esta es la sección más volátil del trazado de rayos y su correcta implementación resulta crítica.

Finalmente, tras construir las diferentes *BLAS*, se procedió al desarrollo y construcción de la *TLAS*. Se decidió almacenar toda la escena en una única *TLAS*, ya que las funcionalidades requeridas no exigían el uso de múltiples estructuras, lo que además simplificó el desarrollo del proceso. Para ello, se siguió un procedimiento similar al utilizado en la construcción de una *BLAS*, aunque más extenso. En concreto, se realizaron los siguientes pasos:

1. Obtención de las matrices de transformación.
2. Creación de instancias para *BLAS*.
3. Creación de la *TLAS*.

El primer paso consiste en obtener, para cada *BLAS*, su matriz de transformación y convertirla en el objeto *Vulkan* correspondiente. Dado que la transformación ya se había aplicado previamente, se utilizó la matriz identidad para evitar modificar nuevamente la posición de cada *BLAS*. A continuación, se crearon las instancias para las *BLAS*. Cada instancia incluye la propia *BLAS* junto con información adicional: la matriz de transformación, el índice de instancia (útil en etapas posteriores), una máscara que indica con qué rayos puede intersectar (en este caso, todos), y la *Flag* para desactivar el *culling* de triángulos no visibles.

Con esta información, se procedió a la creación de la *TLAS*. Esta estructura se compone de instancias en lugar de únicamente *BLAS*, lo que permite incorporar información adicional relevante. El proceso de construcción de la *TLAS*, al igual que el de las *BLAS*, requiere atención al detalle y se compone de los siguientes pasos:

1. Crear un búfer de instancias.
2. Copiar los datos de las instancias al búfer.
3. Configurar la geometría.
4. Preparar la información para la construcción.
5. Obtener la información de tamaños.

6. Crear el búfer auxiliar.
7. Crear el búfer para la *TLAS*.
8. Crear la *TLAS*.
9. Construir la *TLAS*.

El primer paso de este proceso consiste en crear un nuevo búfer para almacenar todas las instancias. Esta operación se realizó solicitando directamente el recurso al núcleo *Vulkan*. A continuación, se copiaron los datos de las instancias al búfer. Es necesario agrupar todas las estructuras de aceleración en un único búfer, ya que la construcción de la *TLAS* requiere subir toda la información en una sola operación.

El siguiente paso es configurar la geometría de la *TLAS*. Este proceso consiste en indicar que la estructura de aceleración se construirá a partir de instancias, así como especificar su ubicación en memoria. Aunque existen otras opciones, como utilizar triángulos, *Bounding boxes* o esferas, se optó por el uso de instancias debido a las funcionalidades adicionales que ofrecen.

A continuación, se proporciona la información necesaria para la construcción: número de instancias, primera instancia, desplazamiento de las instancias y desplazamiento de las transformaciones. En este caso, todos los valores son cero excepto el número de instancias.

Posteriormente, se calculó el tamaño real de las instancias, siguiendo el mismo procedimiento utilizado para las *BLAS* al finalizar su geometría. Los pasos siguientes fueron la creación de un búfer auxiliar para la *TLAS* y, a continuación, la creación del búfer definitivo que se utilizará. El tamaño de ambos difiere, ya que el búfer final emplea una disposición optimizada del espacio para la tarea.

Una vez completados estos pasos, se procedió a crear y construir la *TLAS*. Este proceso consiste, en primer lugar, en invocar el método *Vulkan* para crear la *TLAS* y, posteriormente, construirla utilizando su dirección. Al finalizar, se dispone tanto de la referencia a la *TLAS* como del búfer que contiene su información. Esta *TLAS* será posteriormente enviada al *shader* para ser utilizada en el trazado de rayos.

Descriptor Sets

La siguiente tarea, tras la creación de las estructuras de aceleración, fue la generación de los diferentes *Descriptor Sets*, utilizados para transferir información desde la CPU a la GPU en forma de uniformes o *Buffers*. En total, se crearon tres *Descriptor Sets* para enviar datos a distintos puntos de la *Pipeline*. La decisión de utilizar múltiples *Descriptor Sets* en lugar de uno único se basó en que no toda la información es necesaria en todos los puntos. Esta separación permite simplificar la programación, reducir el número de índices de atributos y optimizar el uso de memoria. Los *Descriptor Sets* definidos son:

- *Descriptor Set* para estructura de aceleración e imagen de salida.
- *Descriptor Set* para matrices de transformación.
- *Descriptor Set* para geometría.

El proceso de creación es común para todos los *Descriptor Sets*, aunque varía en los argumentos y la organización interna de cada uno. Los pasos seguidos fueron:

1. Crear la disposición del *Descriptor Set*.
2. Asignar el espacio al *Descriptor Set*.
3. Escribir los datos en el *Descriptor Set*.

El primer paso para la creación de los *Descriptor Sets* consiste en definir la disposición que estos tendrán. Este es el paso más relevante, ya que determina qué atributos se utilizarán y dónde estarán ubicados dentro del set. El proceso incluye la creación de un *Descriptor Pool*, un objeto que especifica el tipo de cada recurso a transferir, y del *Layout* que define su organización.

Por cada atributo que contendrá el *Descriptor Set*, se debe crear una *Pool* con el tipo correspondiente, lo que permitirá la transferencia de información como dicho tipo. Una vez creado este objeto, se procede a la creación de la disposición. Esto implica generar un objeto de *Vulkan* que será enviado a la *Pipeline* para indicar cómo y dónde se ubican los atributos.

Las disposiciones definidas son las siguientes:

Para el *Descriptor Set* de estructura de aceleración e imagen de salida:

- Estructura de aceleración en el índice 1.
- Imagen de almacenamiento en el índice 2.

Estas se utilizarán en el *Shader* de *Raygen*.

Para el *Descriptor Set* de matrices de transformación:

- Matriz en el índice 1.

Estas se utilizarán en el *Shader* de *Raygen*. La matriz que se transferirá será únicamente la matriz de vista-proyección, ya que las matrices de modelo se incluyen en las estructuras de aceleración y no existe un caso en el que se utilicen por separado.

Para el *Descriptor Set* de geometría:

- Vértices en el índice 0.
- Índices en el índice 1.
- Normales en el índice 2.
- Coordenadas de textura en el índice 3.
- Texturas en el índice 4.

Estas se utilizarán en el *Shader* de *Closest Hit*. Este *Descriptor Set* permitirá acceder, desde el *Shader*, a las propiedades del triángulo impactado, como sus normales.

Una vez definidas las disposiciones para todos los *Descriptor Sets*, se procedió a asignar el espacio necesario para almacenarlos. Esta asignación se realiza a partir de la disposición previamente creada y se guarda en un nuevo objeto *Vulkan*. Dentro de este proceso, se puede especificar cuántas instancias de cada disposición se crearán. En el caso de la librería, solo se requiere una instancia de cada *Descriptor Set*, ya que no se necesitan duplicados ni copias de los datos. Esta funcionalidad sería útil si se pretendiera almacenar dos escenas simultáneamente, lo cual no aplica en este contexto.

Finalmente, el último paso para completar la creación de los *Descriptor Sets* es la escritura de los datos en los mismos. Este paso no requiere ejecutarse en el momento de su creación, pero sí debe realizarse antes de la creación de la *Pipeline*. Una vez creada la *Pipeline*, la escritura de datos en el *Descriptor Set* puede realizarse en cualquier momento.

Para transferir información a los *Descriptor Sets*, se utiliza un objeto de *Vulkan* denominado *Write Descriptor Set*, que actúa como estructura intermedia donde se especifican los datos, el índice de destino y su tipo. Posteriormente, se emplea la función correspondiente de *Vulkan* para realizar la transferencia.

Es importante destacar que, durante el desarrollo de la librería, esta función se utilizará de forma recurrente cada vez que cambien los parámetros. Además, *Vulkan* permite la escritura parcial en los *Descriptor Sets*, lo cual resulta fundamental para reducir significativamente la cantidad de datos transferidos durante la ejecución. Por ejemplo, la imagen de salida se transfiere una única vez, mientras que la estructura de aceleración debe actualizarse cada vez que se modifique. Esto evita transferencias innecesarias y reduce el tiempo de ejecución de esta tarea.

Creación de la *Pipeline*

El paso más importante a completar en el núcleo de trazado de rayos fue la creación de la *Pipeline* para trazado de rayos, que constituye el objeto principal utilizado para ejecutar dicho proceso. Esta se encarga de generar, procesar y obtener la salida de todos los rayos.

Antes de crear la *Pipeline*, es necesario definir los *Shaders* que se utilizarán. Se determinó que se emplearían los *Shaders* de *Raygen*, *Miss* y *Closest Hit*, utilizando únicamente uno de cada tipo, ya que no se requería funcionalidad diferenciada según el tipo de intersección o fallo.

Una vez tomada esta decisión, se procedió a crear un objeto *Vulkan* para cada una de las *Stages*, asociando el código del *Shader* con la etapa correspondiente de ejecución. Posteriormente, se creó un objeto *Vulkan* adicional, el *Shader Group*, que indica a la *Pipeline* cómo, qué y cuándo ejecutar cada *Shader*. En este caso, se definieron dos grupos de tipo general, que contienen los *Shaders Raygen* y *Miss*, respectivamente, y un grupo de tipo intersección con triángulos, que se activa al detectar una colisión con geometría y contiene el *Closest Hit*.

Una vez definidas las *Stages*, se creó la disposición de la *Pipeline*. Para ello, se combinaron las *Stages* con los *Descriptor Sets* y sus respectivas disposiciones, especificando así la configuración completa de la *Pipeline*.

Finalmente, se integraron todos los componentes previamente definidos y se invocó el método de *Vulkan* encargado de crear la *Pipeline*. Adicionalmente, se estableció el nivel máximo de recursión de los rayos, que en este caso es 2. Este valor es estático y no puede modificarse una vez creada la *Pipeline*.

En conjunto con la creación de la *Pipeline*, se debe generar la *Shader Binding Table* para completar la funcionalidad del trazado de rayos. Esta tabla proporciona la dirección física y el contenido de los *Shaders* a la *Pipeline* en tiempo de ejecución. La *Shader Binding Table* se define mediante regiones de memoria que contienen una referencia al *Shader* asociado y su código.

El proceso de creación de la tabla se organizó de la siguiente manera:

1. Cálculo del tamaño de las *Stages*.
2. Obtención de los controles de las *Stages*.

3. Creación de un *Buffer* para almacenar la *Shader Binding Table*.
4. Mapeado y copia de datos al *Buffer*.
5. Configuración de la *Shader Binding Table*.

El cálculo del tamaño se realiza a partir del tamaño de los *Shader Groups* y del tamaño predeterminado de la referencia que contiene cada grupo. A partir de estos valores se obtiene el tamaño alineado de cada grupo. La fórmula utilizada es:

Tamaño grupo alineado $= (\text{tamaño referencia} + \text{alineamiento base} - 1)\&(\text{alineamiento base} - 1)$.

Una vez calculado, se obtienen las referencias de los grupos, es decir, las direcciones en memoria donde estarán ubicados. A continuación, se calcula el tamaño total de la tabla como el tamaño de un grupo alineado multiplicado por el número de grupos. Con este valor, se crea un *Buffer* que almacenará la tabla. Este *Buffer* debe configurarse con una opción adicional que indique su uso específico para la *Shader Binding Table*, lo que implica una disposición especial en memoria.

El siguiente paso consiste en copiar los datos de las referencias al *Buffer* en su ubicación correspondiente, situada al inicio de cada grupo. Finalmente, se debe indicar la dirección de cada grupo y su desplazamiento. Con esto, la tabla queda completada sin necesidad de utilizar funciones adicionales de *Vulkan*.

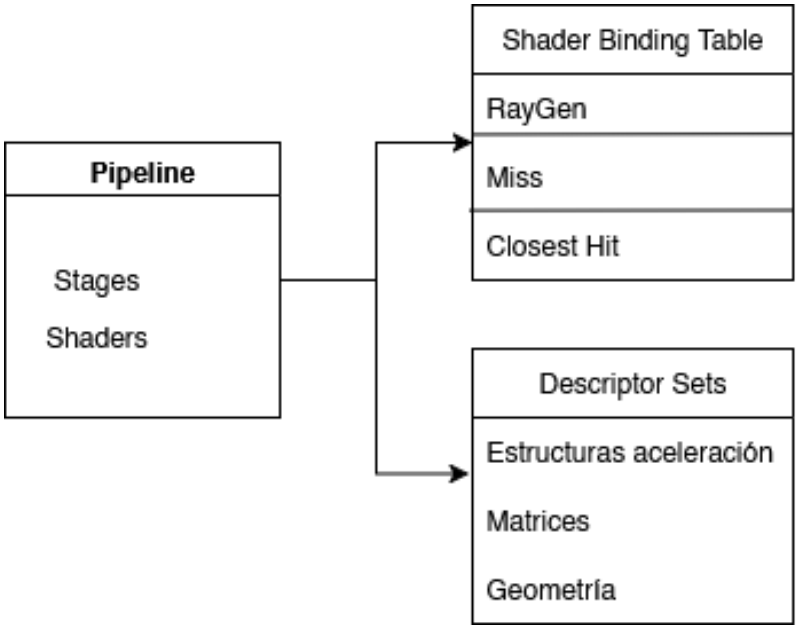


Figura 4.8: Diagrama de la estructura final de la *Pipeline*.

Renderizado

Finalmente, se alcanzó el punto en el que se inicia el proceso de renderizado de la escena mediante trazado de rayos. Este procedimiento se simplificó considerablemente gracias a las estructuras creadas previamente, centrándose en la invocación de la función de trazado de rayos de *Vulkan*. Esta función integra todos los componentes definidos y lanza los rayos dentro de la *Pipeline*.

Para ejecutar esta función, es necesario preparar toda la información requerida por la *Pipeline*. En primer lugar, se debe convertir el formato de la imagen de salida al formato

preferido por la *Pipeline*. Además, se habilita la escritura en esta imagen desde los *Shaders*, permitiendo modificar su color.

A continuación, se enlazan tanto la *Pipeline* como los *Descriptor Sets*. Aunque estos últimos ya hayan sido enlazados previamente, es necesario repetir el proceso, ya que podrían utilizarse otros *Descriptor Sets* con la misma disposición pero con información distinta, lo cual no aplica en este caso.

Una vez completada la preparación, se invoca la función de trazado de rayos. Esta requiere las regiones de los *Shaders* definidas en la *Shader Binding Table*, las dimensiones (ancho × alto) de la imagen y el número de rayos por píxel. Aunque lanzar múltiples rayos por píxel puede mejorar el detalle de la imagen debido a variaciones en los cálculos, en este trabajo se optó por utilizar un único rayo por píxel para optimizar el tiempo de ejecución.

El resultado se almacena directamente en la imagen de salida. Aunque existe la opción de utilizar el *Buffer* de salida de color de la *Pipeline*, esta alternativa implica operaciones adicionales para recuperar la información, por lo que se descartó.

Una vez finalizado el trazado de rayos, se restaura el formato de la imagen a su estado estándar para permitir su uso en tareas posteriores. Todo el proceso se registra en un *Command Buffer* y se transfiere a la GPU mediante la *Queue* seleccionada previamente. Con ello, la imagen queda renderizada mediante trazado de rayos.

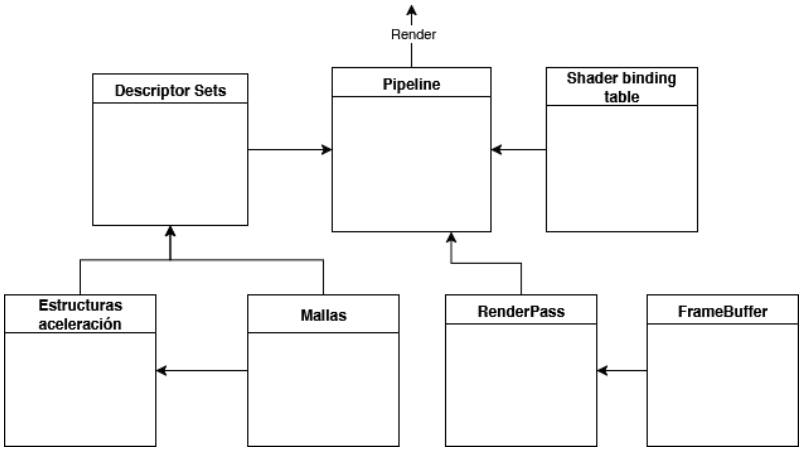


Figura 4.9: Diagrama de la estructura final de la *Pipeline*.

Shaders

Una vez implementada toda la funcionalidad del renderizado, se procedió a la creación de los *Shaders*, los cuales determinan cómo se dibuja la escena. Estos *Shaders* contienen el comportamiento necesario para generar la imagen final deseada. Tal como se indicó anteriormente, en este trabajo se implementaron tres *Shaders*:

- *Raygen*, encargado de generar los rayos y almacenar los resultados.
- *Miss*, que se ejecuta cuando un rayo no impacta con ningún objeto.
- *Closest Hit*, que se ejecuta cuando un rayo impacta con la primera geometría.

Cada uno de estos se almacenó en un archivo independiente y se carga dinámicamente durante la ejecución del programa, utilizando el núcleo *Vulkan* para leerlos desde disco y compilarlos desde GLSL a SPIR-V.

Antes de implementar cualquiera de estos *Shaders*, fue necesario definir la estructura de los rayos a lanzar, los cuales propagan información en dos direcciones. Estos rayos deben contener toda la información necesaria para permitir el paso de datos durante la ejecución de la librería. Inicialmente, se implementó un rayo que transmitía únicamente el color asociado al punto de impacto. Posteriormente, se amplió esta definición a una estructura denominada *RayPayload*, que incluye:

- Color.
- Indicador de impacto.
- Profundidad.

El color mantiene su función de propagar, para cada píxel, el color correspondiente al punto de impacto. El indicador de impacto actúa como mecanismo de validación para evitar el lanzamiento de rayos adicionales en caso de no haber colisión, y modifica el comportamiento del paso de color en función del resultado. La profundidad sirve como control adicional para limitar la generación de rayos recursivos innecesarios, reduciendo así la carga de trabajo para *Vulkan*.

Cabe destacar que, en GLSL, los rayos utilizan tipos específicos dentro del *Shader*, como *Uniform* para datos uniformes o *In* para entradas. En el contexto del trazado de rayos, se emplean los modificadores *RayPayloadEXT* y *RayPayloadInEXT*, equivalentes a *Out* e *In*, respectivamente, es decir, salida y entrada.

Una vez definidos los rayos utilizados para el paso de información, se procedió a la creación del primer *Shader*, el *Raygen*. Este *Shader* se encarga de generar los rayos primarios, que atraviesan la pantalla hacia la geometría, y de almacenar los resultados en la imagen de salida. Se ejecuta automáticamente una vez por píxel al invocar el renderizado.

El primer paso consiste en determinar el punto de la pantalla desde el cual se lanzará el rayo. Para ello, se utiliza una variable asociada automáticamente que indica la posición en pantalla, y mediante una serie de transformaciones se obtiene el punto central del píxel en el espacio de cámara.

A continuación, se construye el rayo de salida que será lanzado por el *Shader*. Este paso presentó una dificultad técnica: el rayo, si se genera directamente con la información disponible, se encuentra en el espacio de cámara, mientras que la geometría reside en el espacio del mundo. Dado que la geometría no se transforma antes del lanzamiento del rayo, una vez creada la estructura de aceleración, se plantearon dos alternativas:

1. Transformar y reconstruir la estructura de aceleración en cada iteración del programa, multiplicándola por la matriz de vista.
2. Transformar el rayo al espacio del mundo, multiplicándolo por la inversa de la matriz de vista-proyección.

Se optó por la segunda opción, ya que la creación de estructuras de aceleración es más costosa y se ejecuta en CPU, mientras que la transformación del rayo en GPU es más eficiente.

Una vez resuelto este aspecto, se definió el rayo a lanzar. Este tiene su origen detrás de la cámara en el espacio de vista, simulando la posición del observador, y se dirige hacia el centro de cada píxel. Tanto el origen como la dirección se transforman mediante la inversa de la matriz de vista-proyección.

También se establecieron los valores iniciales del rayo: color (0, 0, 0) en RGB, profundidad 0, e indicador de no impacto. Aunque el color inicial es irrelevante, ya que será sobrescrito en todos los casos, se define como valor por defecto.

Una vez configurado el rayo, se procede a su lanzamiento mediante una función que requiere varios argumentos. Los más relevantes son:

- Estructura de aceleración con la que se realizará la intersección.
- *Flags*, configuradas para permitir impacto con cualquier triángulo, incluso si no está orientado hacia el rayo.
- Máscara de bits que define qué rayos pueden impactar; en este caso, todos.
- Distancia mínima del rayo: 0.001.
- Distancia máxima del rayo: 1000.
- Origen del rayo.
- Dirección del rayo.
- Índice del *Shader* de tipo *Miss*; en este caso, 0.
- Índice de ubicación del atributo rayo que será lanzado.

Una vez lanzado el rayo, se ejecuta todo el proceso recursivo asociado y, al finalizar, se retorna la ejecución al *Shader* de *Raygen*. La última operación que realiza este *Shader* consiste en tomar el color del rayo, modificado por otro *Shader*, y copiarlo en la imagen de salida en las coordenadas de pantalla calculadas al inicio. De este modo, los colores se transfieren correctamente a la imagen de salida.

El siguiente *Shader* implementado fue el de tipo *Miss*. Una vez generados los rayos, ya sea desde el *Raygen* o desde el *Closest Hit*, si no impactan con ninguna geometría, se ejecuta el *Shader* de tipo *Miss*. Este fue el más sencillo de implementar, ya que su único propósito es actuar como sumidero de rayos, asignándoles un color y devolviéndolos al *Shader* que los originó.

En concreto, recibe un rayo como entrada y devuelve un color predeterminado, indicando que no ha habido impacto. Este color permanece constante en todos los casos de fallo, lo que permite identificar visualmente cuándo un rayo no ha colisionado, independientemente de su posición o número de rebotes. Dicho color se muestra en la figura 4.10.

El último *Shader* implementado fue el de tipo *Closest Hit*. Este *Shader* se encarga de devolver al *Shader* de *Raygen* el color correspondiente en función de la lógica definida. En concreto, ante un rayo entrante, si el impacto no se produce con una fuente de luz, se debe reflejar el rayo. Si el impacto ocurre con una luz, se devuelve el color de dicha fuente. Si se alcanza el límite de rebotes permitidos, se indica que no ha habido impacto con ninguna luz.

En general, si un rebote no alcanza una fuente de luz, se colorea el punto de impacto con el color del objeto en ese lugar. Si los rebotes alcanzan finalmente una luz, se asigna el color de la luz al punto. Si los rebotes no impactan con ninguna geometría, se vuelve a aplicar el color del objeto. De este modo, los objetos se visualizan normalmente, salvo en los puntos donde reciben luz directa o indirecta.

La implementación de este *Shader* se realizó de la siguiente manera. Primero, se definieron dos rayos: uno de entrada, que llega al *Shader*, y otro de salida, que se propaga



Figura 4.10: Color utilizado en el shader de *Miss*.



Figura 4.11: Captura del resultado de colorear de un color rojo aquellos puntos donde se ejecuta el shader *Closest Hit*.

desde él. Además, se recuperaron todos los atributos necesarios desde el *Descriptor Set* de geometría, incluyendo vértices, normales, texturas, colores, entre otros.

El primer paso en la ejecución consiste en obtener el índice de la malla impactada. Este índice es un atributo definido previamente al crear las instancias de la *TLAS*, lo que permite identificar la geometría correspondiente en esta etapa.

Una vez obtenida la malla, es necesario determinar con qué triángulo de dicha malla se ha producido el impacto. Este triángulo está identificado mediante una variable automática proporcionada por la *Pipeline* al *Shader*, similar a la posición en pantalla utilizada en el *Raygen*. Con esta información, se pueden obtener los índices y, por tanto, los vértices del triángulo impactado. Esto se realiza extrayendo, a partir del índice de la malla, los tres índices del triángulo, definidos como Índice de primitiva $* 3 + (0, 1, 2)$.

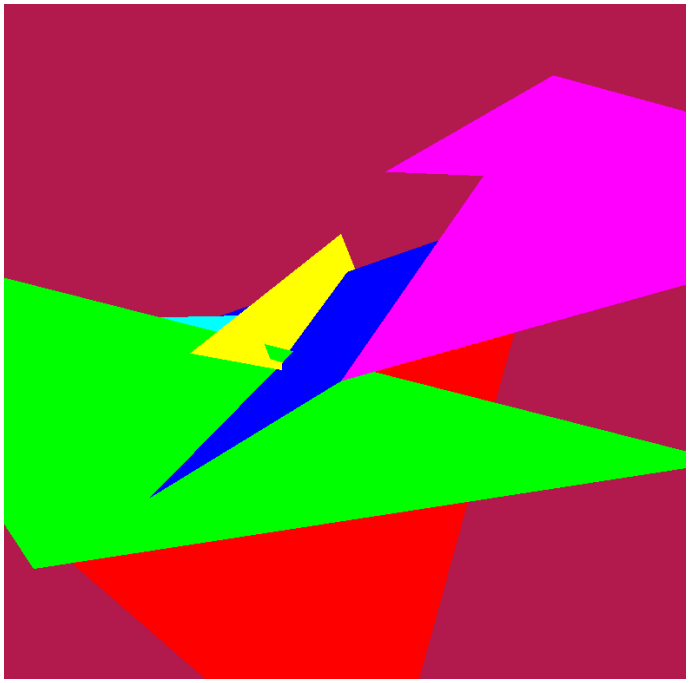


Figura 4.12: Captura mostrando de un color cada malla en función de su índice.

Una vez obtenidos los índices, los vértices se recuperan utilizando el índice de la malla y el índice correspondiente. Para calcular el punto exacto de impacto, es necesario realizar una operación adicional. En cada instancia de ejecución del *Shader* de tipo *Closest Hit*, se genera automáticamente un objeto con las coordenadas baricéntricas del triángulo, siendo estas $(1-u-v, u, v)$.

Estas coordenadas representan la posición local del impacto dentro del triángulo y deben multiplicarse por los tres vértices obtenidos, realizando una interpolación lineal, para obtener el punto real del impacto.

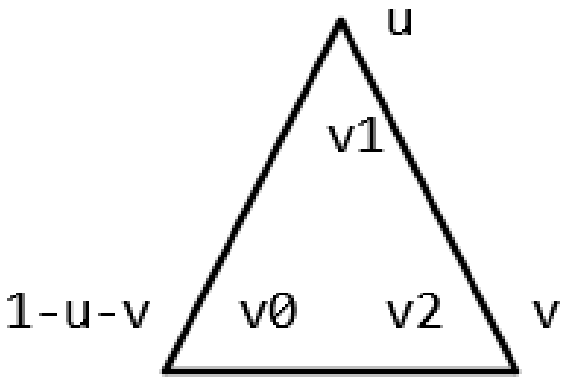


Figura 4.13: Esquema explicando el funcionamiento de coordenadas baricéntricas, extraído de Tutorial de trazado de rayos de Nvidia.

Una vez obtenido el punto de impacto, se realiza el mismo proceso de interpolación para obtener la normal en dicho punto. Esta será necesaria para calcular el rayo reflejado en etapas posteriores.

El siguiente paso consiste en determinar si el punto impactado pertenece a una fuente de luz. Para ello, se complementa la geometría de la malla con un nuevo índice que indica si el triángulo corresponde a una luz. En caso afirmativo, y si el impacto se encuentra dentro de los puntos emisores de luz, se devuelve directamente el color de dicha fuente, indicando que se ha producido un impacto con luz. Este color se propaga hasta el *Shader* de tipo *Raygen*.

Una vez establecida esta condición de finalización, se inicia el proceso recursivo mediante el cual se evalúa si el rayo impacta o no con una fuente de luz. Se define inicialmente un color base, correspondiente al color de la malla, y una profundidad máxima, que como se explicó anteriormente, es igual a 2. Además, se obtiene la dirección del rayo entrante.

El *Shader* se configuró para que solo se reflejen rayos cuya dirección forme un ángulo mayor a 90 grados respecto a la normal. Esto se debe a que, en caso contrario, el rayo estaría atravesando la parte posterior del triángulo, lo cual no es deseado.

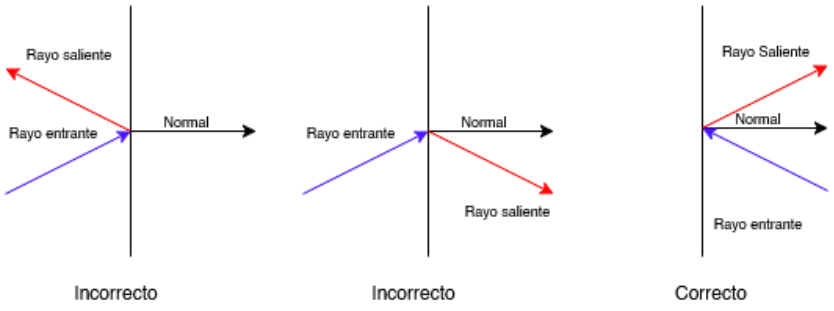


Figura 4.14: Diagrama explicando qué tipo de reflexión se permitirá en la ejecución del shader.

Tras esto, comienza el bloque recursivo. Si la profundidad del rayo es menor que la máxima permitida, se lanza un nuevo rayo. Este rayo no ha impactado con ninguna geometría, no tiene color, y su profundidad es incrementada en uno respecto al rayo entrante. La dirección del nuevo rayo se calcula como la reflexión de la dirección del rayo entrante respecto a la normal, y su origen es el punto de impacto.

El rayo se lanza utilizando los mismos argumentos que en el *Shader* de tipo *Raygen* para los parámetros restantes, manteniendo así la misma funcionalidad. Al retornar, si el rayo ha impactado con una fuente de luz, se propaga el color del impacto. En caso contrario, se devuelve el color del objeto en el que se está ejecutando el *Shader*, junto con la indicación de no impacto. Si se alcanza la profundidad máxima, se propaga directamente el no impacto.

Dado este comportamiento, se utiliza el indicador de impacto como mecanismo de propagación. Si hay impacto, se transmite el color hacia atrás. Si no lo hay, se devuelve el color del objeto, asegurando que cada objeto mantenga su color específico.

Para verificar la funcionalidad de los rebotes, se realizó una prueba que se muestra en la figura 4.15, donde se representa el número de rebotes realizados por un rayo desde un punto, utilizando un código de colores. También se observa un problema de *Z-fighting* cuando dos mallas se superponen, por lo que se recomienda tener en cuenta esta situación durante el desarrollo.

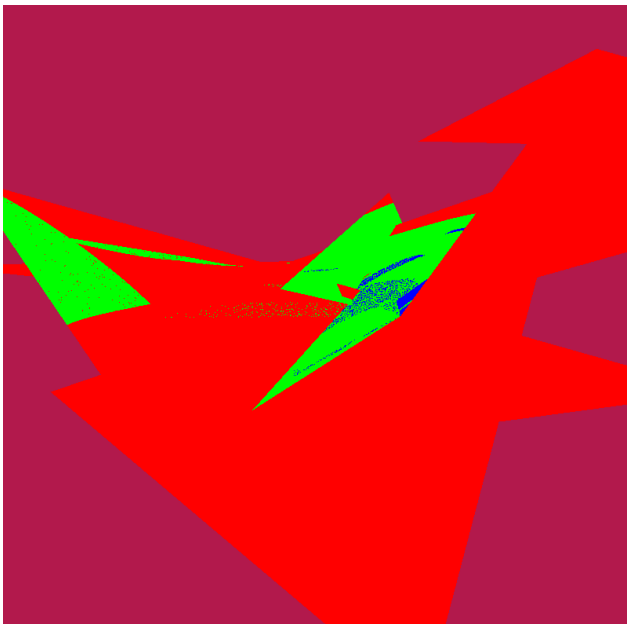


Figura 4.15: Captura mostrando un ejemplo de ejecución donde se colorea de diferentes colores en función de la cantidad de rebotes que se realizan a partir de cada punto.

Inclusiones adicionales

Adicionalmente, durante la fase final del desarrollo se decidió incorporar funcionalidades no previstas en el diseño original, con el objetivo de ampliar las capacidades de los *Shaders*.

En primer lugar, se añadió al *Shader* de tipo *Closest Hit* una funcionalidad para declarar ciertas mallas como no inspeccionables, impidiendo que reflejen rayos. Para ello, se transfiere a la GPU un nuevo valor booleano que indica esta condición, permitiendo establecer una salida adicional durante la ejecución del *Shader*.

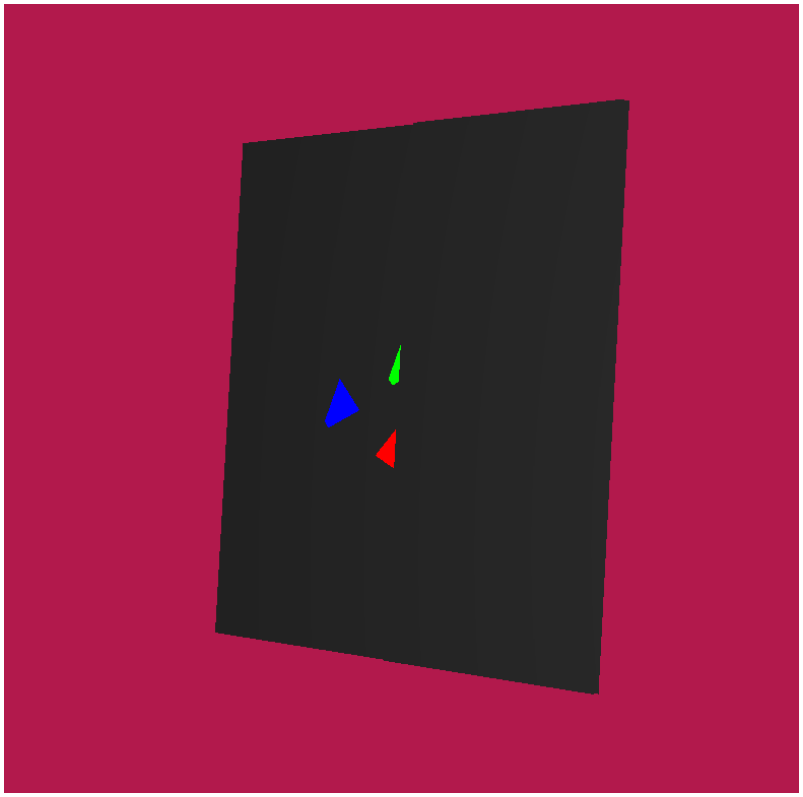


Figura 4.16: Ejemplo de oclusiones en el sistema.

Asimismo, se incorporó una funcionalidad adicional al *Shader* de generación de rayos y al *Descriptor Set* de matrices. Se añadió una nueva matriz que define una cámara de espectador, utilizada para obtener una vista externa desde distintos ángulos de la escena observada por la cámara principal. De este modo, se dispone de dos cámaras en la escena, con la posibilidad de utilizar únicamente la cámara de inspección, desde la cual se realiza el trazado de rayos, al establecer la cámara de vista (la de espectador) como matriz identidad.

Esta nueva matriz requiere ajustes en el *Descriptor Set* para acomodar el tamaño de dos matrices, así como modificaciones en la interfaz, que se detallarán posteriormente. El uso de esta cámara en el *Shader* implica modificar las coordenadas de escritura en la imagen de salida. Para ello, se calcula la nueva coordenada multiplicando la coordenada original por la inversa de la matriz de inspección (para pasar a espacio de mundo), y posteriormente por la matriz de vista (para obtener la nueva posición).

Dado que este proceso puede dejar regiones de la imagen sin rellenar, se inicializa previamente la imagen con el color de fondo antes de ejecutar el trazado de rayos.

4.3 Renderer

Una vez construidas todas las funcionalidades del núcleo de trazado de rayos, el último paso para completar la librería consiste en la creación del *Renderer*, que será la parte accesible de la librería y deberá ofrecer todas las funcionalidades necesarias al usuario para ejecutar el trazado de rayos. Esta se basa en la implementación de una interfaz denominada *Renderer*, la cual define el conjunto de métodos necesarios para acceder a toda la funcionalidad desarrollada.

Como se ha explicado anteriormente, el objetivo del *Renderer* es permitir que un usuario externo pueda subir toda la geometría deseada para el trazado de rayos y observar su comportamiento sobre dicha geometría. Esto implica múltiples operaciones internas tanto en el núcleo de *Vulkan* como en el núcleo de trazado de rayos, aunque dichas operaciones serán completamente opacas al usuario.

De forma general, se busca que el usuario pueda subir la geometría junto con sus texturas asociadas, definir las fuentes de luz que iluminarán la escena, posicionar una cámara para observar el funcionamiento del sistema, y obtener la salida del trazado de rayos, ya sea para almacenarla o para utilizarla en otra parte del código. Además, se incluirán funciones auxiliares que faciliten el uso de toda la funcionalidad.

Por ello, se definió una lista de funciones accesibles para el usuario, que serán:

- Init.
- DefineMesh.
- AddMesh.
- AddTexture.
- DeleteTexture.
- AddLight
- RemoveMesh.
- ClearScene.
- SetCamera.
- SetOutputResolution.
- Render.
- CopyResultBytes.
- GetResultTextureId.

A continuación, se explica la funcionalidad y el propósito de cada una de las funciones disponibles. Estas permiten añadir contenido a la escena, obtener resultados del trazado y modificar la disposición de los elementos en la misma.

Antes de definir el comportamiento de cada método del *Renderer*, fue necesario establecer la estructura interna de la clase. Principalmente, se requiere almacenar las mallas una vez definidas, así como las texturas asociadas. También es necesario conservar la textura de salida. Por ello, se decidió operar de la siguiente manera: la clase *Renderer* contiene tres listas principales: lista de mallas candidatas, lista de texturas candidatas y lista de mallas a dibujar.

Cuando se define una malla, esta se añade a la lista de mallas candidatas. Posteriormente, puede ser incluida en la lista de mallas a dibujar, realizando una copia de la original. Las texturas, una vez añadidas, se insertan en la lista de texturas candidatas. Estas se transfieren al *Shader*, pero no se utilizan a menos que se especifique explícitamente.

Dado el funcionamiento del sistema, únicamente las luces emplean texturas como *Lightmap*, es decir, para identificar los puntos emisores de luz. Por tanto, al crear una luz, que también se representa como una malla, se actualiza un índice en dicha malla que indica qué textura utilizar, a diferencia de las mallas normales.

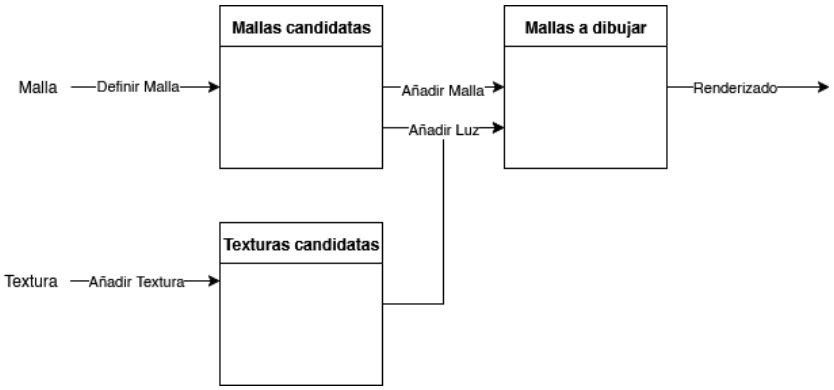


Figura 4.17: Diagrama mostrando la funcionalidad principal de dibujar mallas.

Init

El método *Init* es imprescindible para configurar y ejecutar la librería. Este se encarga de inicializar todos los componentes necesarios para el trazado de rayos, incluyendo el núcleo de *Vulkan*, los *Descriptor Sets*, la *Pipeline* y los *Shaders* requeridos. Los *Shaders* se cargan desde sus archivos correspondientes y se traducen al lenguaje adecuado mediante la librería *Glslang*.

Este método presenta varias consideraciones: debe ejecutarse antes que cualquier otro método, excepto los métodos de borrado, ya que todos los demás dependen de los componentes configurados en él. Su ejecución incorrecta puede provocar comportamientos erróneos. El método devuelve **True** si se ha ejecutado correctamente. En caso de devolver **False** o producirse un error interno en *Vulkan*, esto indica que las librerías asociadas no se han instalado correctamente o que el dispositivo no es compatible con las funcionalidades de trazado de rayos. Cualquier error de este tipo se detectará en este método.

La ejecución de *Init* puede ser extensa, principalmente debido a la compilación de los *Shaders*, pero esto reduce la carga de trabajo en los métodos posteriores.

DefineMesh

El método *DefineMesh* es el primer paso que debe realizar el usuario para añadir mallas al proceso de renderizado por trazado de rayos. Este método recibe vértices, índices, normales y coordenadas de textura, y define internamente una nueva malla que se añade a la lista de mallas candidatas.

Para los índices y coordenadas de textura, se crea un *Buffer* inmediatamente al ejecutar el método, ya que estas propiedades no se modificarán durante la ejecución de la librería. En cuanto a los vértices y normales, se almacenan como una copia de la entrada, y su *Buffer* se genera posteriormente, cuando puedan ser transformados mediante las matrices de transformación.

La cantidad de vértices de la malla se determina a partir del número de índices proporcionados. A cada malla definida se le asigna un identificador incremental, que permite localizarla en la lista de mallas candidatas y añadirla posteriormente a la lista de mallas a dibujar. Este identificador no se reutiliza, incluso si una malla ha sido eliminada.

Las mallas definidas mediante este método constituyen la base para su posterior inclusión en la lista de mallas a dibujar, permitiendo replicar una misma malla varias veces mediante el método correspondiente.

AddMesh

El método *AddMesh* permite añadir mallas a la escena como objetos. Este método recibe una matriz de modelo, un color y un identificador de malla, y añade dicha malla a la escena con el color y la posición especificados.

El primer paso consiste en verificar si la malla existe. En caso contrario, el método devuelve false. Si la malla se encuentra, se asignan el color y la matriz de transformación, y se multiplican todos los vértices y normales por dicha matriz. Como se explicó anteriormente, este enfoque reduce la carga en tiempo de ejecución a costa de un mayor tiempo de carga.

Una vez transformados los vértices y normales, se crea un *Buffer* para cada uno, que será accedido por el *Shader*. Finalmente, la malla se añade a la lista de mallas a dibujar.

AddTexture

El método *AddTexture* permite crear texturas que posteriormente podrán utilizarse para definir luces. Estas texturas se definen mediante su amplitud, altitud, contenido de píxeles y número de bytes por píxel. El sistema está diseñado para trabajar exclusivamente con texturas en formato RGBA, por lo que el usuario debe tener en cuenta esta limitación.

Para crear la textura, el *Renderer* accede a la funcionalidad de creación de texturas del núcleo de *Vulkan*. Al igual que con las mallas, se asigna a cada textura un identificador único, que no se reutiliza incluso si la textura es destruida mediante el método correspondiente.

A nivel interno, todas las texturas se suben al *Descriptor Set*, independientemente de si se utilizan o no. El acceso a la textura se realiza mediante un identificador de referencia que se asigna a las luces. Por tanto, estas texturas se emplean exclusivamente para definir luces.

AddLight

El método *AddLight* se encarga de cargar mallas que funcionarán como fuentes de luz. Este método recibe, entre otros parámetros, un identificador de malla y un identificador de textura, y utiliza ambos para configurar la funcionalidad de iluminación. El color generado por la luz se propaga mediante el trazado de rayos.

El comportamiento de este método es similar al de *AddMesh*, con la diferencia de que se asigna a la malla el índice de la textura que utilizará, lo cual no ocurre en mallas que no representan luces. El identificador de luz es proporcionado por el usuario y permite a la librería identificar la fuente de luz correspondiente.

En concreto, este método recibe los mismos argumentos que *AddMesh*, añadiendo los identificadores de luz y textura para habilitar las funcionalidades descritas.

RemoveMesh

El método *RemoveMesh* busca la malla en la lista de mallas a dibujar utilizando el identificador proporcionado como argumento. Si la malla existe, se elimina de la lista.

DeleteTexture

El método *DeleteTexture* localiza la textura en la lista de texturas mediante su identificador. Si se encuentra, se elimina completamente, impidiendo su reutilización.

ClearScene

El método *ClearScene* elimina todas las mallas presentes en la lista de mallas a dibujar.

SetCamera

El método *SetCamera* tiene como única función asignar el valor correspondiente a la matriz de vista-proyección. Esta matriz debe ser calculada externamente a la librería, y el método se encarga de transferirla al *Shader* de tipo *Raygen*.

Dado que dicho *Shader* utiliza la inversa de esta matriz y no se emplea en ningún otro lugar, la inversa se calcula previamente a su transferencia. Es decir, el usuario proporciona la matriz original, y el *Renderer* la transforma internamente para que el *Shader* reciba la versión adecuada.

Esta operación no tiene efecto inmediato hasta que se ejecuta el método de renderizado. Solo se conserva la última matriz proporcionada.

SetOutputResolution

El método *SetOutputResolution* cumple dos funciones principales. La primera consiste en crear una nueva imagen de salida para el trazado de rayos. Por defecto, se genera una imagen de dimensiones (800×800) para almacenar la salida de la librería. No obstante, se recomienda configurar esta imagen mediante este método antes de ejecutar el renderizado.

La imagen de salida se define por la amplitud y altitud proporcionadas como argumentos. Estas dimensiones son necesarias para el renderizado, ya que el método de *Vulkan* requiere conocer el tamaño de la pantalla para lanzar un rayo por cada punto disponible. Si se lanza un rayo por píxel, el número total de rayos será Número Rayos $=$ *Altura* $\times$ *Anchura*.

Al ejecutar este método, se destruye cualquier imagen de salida previamente existente, liberando así el espacio no utilizado.

Render

El método *Render* se encarga de iniciar las operaciones del núcleo de trazado de rayos. Antes de hacerlo, verifica si se ha modificado la geometría de la escena. En caso afirmativo, se reconstruyen las estructuras de aceleración para todas las mallas y se actualizan sus atributos en los correspondientes *Descriptor Sets*.

Además, debido a restricciones en el orden de creación de los distintos componentes, ciertos atributos de la *Pipeline* se generan en este punto, ya que no pueden crearse si

las mallas están vacías o son nulas. Esta operación se realiza únicamente en la primera ejecución del método.

CopyResultBytes

El método *CopyResultBytes* realiza la operación de copia entre la imagen de salida del núcleo de trazado de rayos y un *Buffer* proporcionado como argumento. Esta operación se ejecuta en el núcleo de trazado de rayos, ya que los datos residen allí, aunque sigue una lógica similar a las operaciones de copia del núcleo de *Vulkan*.

El resultado del método es el tamaño de la imagen copiada al *Buffer*. Es necesario proporcionar el tamaño del *Buffer* destino; si este no es suficientemente grande, el método no se ejecutará correctamente y se imprimirá un mensaje de error en consola indicando que no hay espacio suficiente en el *Buffer* de salida para copiar todos los datos.

GetResultTextureId

El método *GetResultTextureId* realiza una operación similar a *CopyResultBytes*, pero en lugar de copiar los datos al *Buffer* en CPU, copia la imagen directamente en GPU. Para ello, se utilizan funcionalidades de interoperabilidad entre *OpenGL* y *Vulkan*, siguiendo los siguientes pasos:

1. Obtener los requisitos de la imagen de salida.
2. Obtener el *Handle* de la memoria en GPU correspondiente a la imagen.
3. Crear un *Memory Object* de *OpenGL* para copiar directamente los datos de memoria.
4. Importar los datos desde *Vulkan* al *Memory Object*.
5. Crear una textura en *OpenGL*.
6. Generar el contenido de la textura a partir del *Memory Object*.

Todos estos pasos requieren funciones adicionales que se importan durante la ejecución del método. El método devuelve el identificador de la textura creada, que contiene la imagen de salida y puede utilizarse en un contexto *OpenGL*.

Este método representa una alternativa más rápida al copiado de *Buffers* en CPU, aunque su ejecución es más sensible debido a los requisitos de interoperabilidad.

Inclusiones adicionales

Al igual que en el núcleo de trazado de rayos, al final del desarrollo se decidió incorporar funcionalidades adicionales para ampliar las capacidades del simulador. En el caso del *Renderer*, estas extensiones se implementaron principalmente para acomodar las funcionalidades añadidas al núcleo de trazado de rayos.

En primer lugar, se añadió un argumento opcional al método *AddMesh* para indicar si la malla es inspeccionable. Este argumento es un valor booleano que, en caso de estar desactivado, impide que la malla refleje rayos.

Además, se añadieron dos nuevos métodos: *SetViewCamera* y *SaveResultToFile*. Como consecuencia de la incorporación de *SetViewCamera*, el método *SetCamera* fue renombrado a *SetInspCamera*, con el objetivo de clarificar su funcionalidad sin modificar su comportamiento.

El método *SetViewCamera* se encarga de establecer el valor de la cámara de espectador, siguiendo la misma lógica que *SetInspCamera*. Por defecto, esta cámara se define mediante la matriz identidad, es decir, sin aplicar ninguna transformación. Es importante destacar que esta matriz se transfiere directamente, sin calcular su inversa, a diferencia de la matriz de inspección. No obstante, este detalle es transparente para el usuario.

Por otro lado, el método *SaveResultToFile* invoca las funcionalidades del sistema para guardar la imagen resultante del trazado de rayos. Para su correcto funcionamiento, es necesario haber ejecutado previamente el método *Render*, ya que este método copia el contenido de la imagen de salida a un archivo en disco, utilizando una ruta especificada. Internamente, se emplea la funcionalidad de *CopyResultBytes* para obtener el contenido de la imagen, que posteriormente se guarda mediante la librería *stbi*.

CAPÍTULO 5
Implantación

A continuación se mostrarán las diferentes formas en las que se ha implantado el sistema para afrontar pruebas y tareas reales. Estas pretenderán demostrar por un lado el correcto funcionamiento del sistema desarrollado y por otro lado las ventajas e implicaciones que este presenta.

5.1 Diseño experimental

Antes de iniciar las pruebas con la librería, se consideró necesario diseñar detalladamente la metodología de evaluación, con el objetivo de mantener consistencia entre los resultados obtenidos.

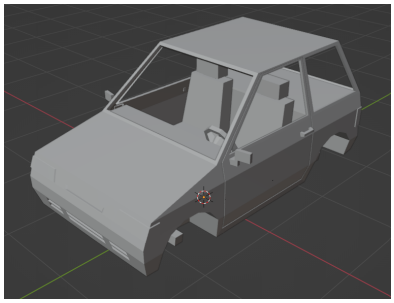
En primer lugar, se seleccionó el equipo donde se realizaron las pruebas. Este cuenta con una GPU `NVIDIA GeForce RTX 3050 Laptop GPU` y una CPU `AMD Ryzen 7 5800H with Radeon Graphics 3.20 GHz`. Todas las pruebas se ejecutaron sobre este hardware, lo que garantiza coherencia entre los resultados obtenidos.

Una vez definido el entorno de ejecución, se estableció la forma en que se llevarían a cabo las pruebas. El objetivo del trabajo consiste en analizar la reflexión de luz sobre la carrocería de distintos vehículos. Para ello, se descargaron modelos 3D de varios coches que fueron utilizados en la aplicación. En concreto, se seleccionaron tres modelos realistas y un modelo adicional no realista, empleado para pruebas de verificación del sistema. Los modelos realistas utilizados son:

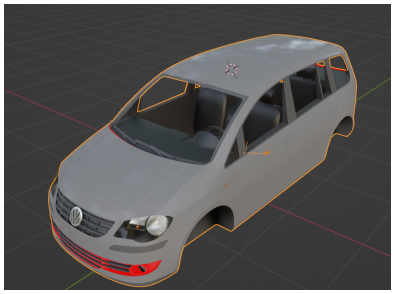
- Volkswagen Touran 2007 con 18,356 triángulos.
- Tesla con 77,984 triángulos.
- Chevrolet Camaro 1969 con 381,332 triángulos.

Durante la selección de los modelos, se buscó que presentaran diferentes cantidades de triángulos, con el fin de evaluar los límites de la librería desarrollada. El modelo no realista consta de 1,297 triángulos. Todos los modelos fueron obtenidos gratuitamente desde la página https://www.turbosquid.com/. A cada uno de ellos se les eliminaron las ruedas y los cristales, ya que estos elementos podrían interferir innecesariamente en la ejecución de la librería y no forman parte de la funcionalidad real del sistema de inspección.

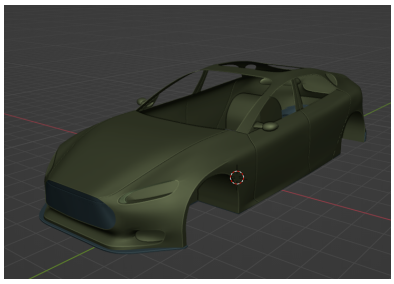
Una vez seleccionados los modelos sobre los que se realizarán las pruebas, se definieron las distintas pruebas a ejecutar. Se llevarán a cabo cuatro pruebas principales y una prueba adicional de validación para validación interna del sistema. Esta última tiene



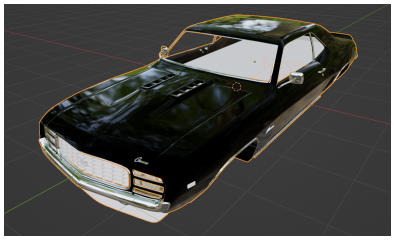
(a) Coche no realista.



(b) Volkswagen Touran 2007.



(c) Tesla.



(d) Chevrolet Camaro 1969.

Figura 5.1: Capturas de los diferentes coches utilizados en las pruebas.

como único propósito verificar que la librería funciona correctamente, carga los modelos adecuadamente y ejecuta el algoritmo de trazado de rayos según lo esperado. Las pruebas restantes evaluarán diferentes configuraciones de modelos y luces, con el objetivo de observar cómo se refleja la luz en los objetos y analizar el rendimiento del sistema. En todas las pruebas se utilizarán modelos sin textura y de color blanco, lo que permitirá observar con claridad los efectos de la iluminación. Posteriormente se realizará una serie de pruebas comparando el simulador creado en *Vulkan* con el simulador existente de *OpenGL*.

5.1.1. Pruebas de corrección

Se realizarán una serie de pruebas para comprobar la corrección del simulador desarrollado en *Vulkan* y poder observar qué configuración del mismo produce mejores resultados dadas unas pruebas determinadas.

Prueba de validación

La prueba de validación consistirá en cargar el modelo del coche no realista y posicionarlo frente a una banda de luces con distintos colores. El objetivo es comprobar que los colores se reflejan correctamente en la superficie del coche, permitiendo visualizar la banda reflejada.

1ª prueba

La primera prueba seguirá una metodología similar a la prueba de validación, pero utilizando el modelo del Volkswagen Touran. Este se colocará frente a la misma banda de luces para evaluar el comportamiento en un modelo realista, donde los reflejos no son uniformes en toda la superficie.

Ambas pruebas se organizarán de forma que la banda de luces se sitúe en el lado derecho del coche, iluminando su superficie para facilitar la observación de los reflejos.

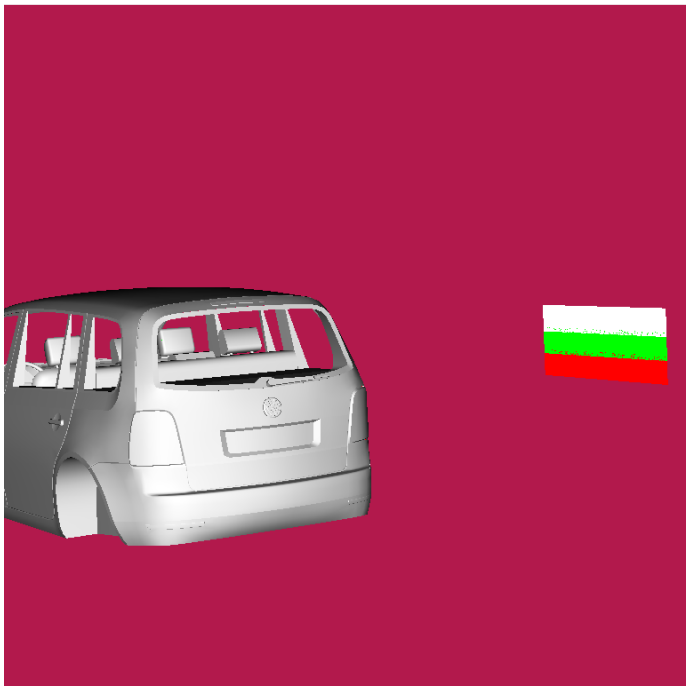


Figura 5.2: Captura mostrando la disposición de la banda frente al coche.

Pruebas 2ª, 3ª y 4ª

Las tres pruebas siguientes tienen como objetivo situar los modelos en un contexto de uso realista para evaluar su comportamiento. Se simula un túnel de inspección por reflexión especular, en el que un arco de luces rodea el vehículo, iluminándolo y generando reflejos sobre su superficie. En este escenario, el modelo del coche se posiciona en el centro, rodeado por cinco luces que imitan el arco. Estas luces presentan distintos colores para facilitar su identificación y, en el espacio de la librería, se representan como cubos que actúan como luces superficiales, sin atenuación ni focalidad. Por tanto, cualquier rayo que impacte sobre una luz será coloreado.

5.1.2. Pruebas de comparación con sistema existente

El propósito de estas pruebas es comparar el simulador desarrollado con otro existente implementado en *OpenGL*. El objetivo principal es evaluar el tiempo de ejecución de ambos sistemas para determinar cuál ofrece mayor eficiencia. Todos los resultados de estas pruebas se medirán en microsegundos.

Prueba base

La prueba base consiste en diseñar una escena con tres luces, una superficie plana inspeccionable y otra superficie no inspeccionable situada frente a la anterior. Esta configuración permite verificar si las oclusiones de los reflejos se gestionan correctamente. Se capturarán imágenes desde distintos ángulos para comprobar la validez del sistema desde diferentes perspectivas. Además, se medirá el tiempo de ejecución del proceso de generación de cada captura para calcular la eficiencia de ambos sistemas. Se tomarán diez muestras y se calcularán la media, la mediana y la desviación estándar.

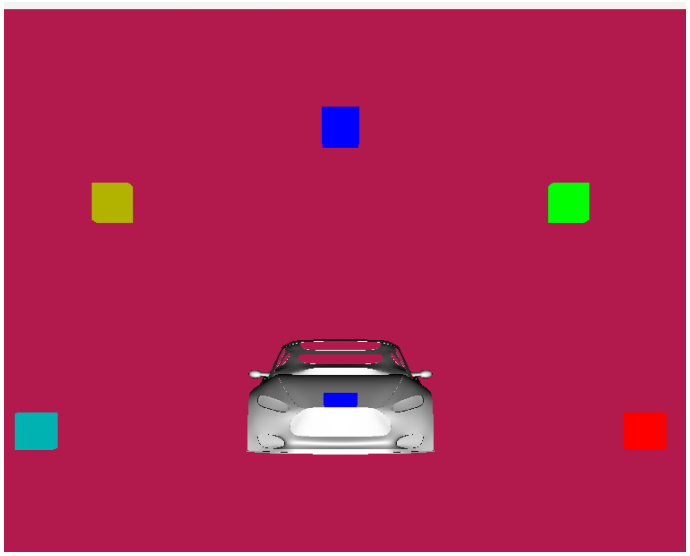


Figura 5.3: Captura mostrando la disposición de las diferentes luces en el espacio.

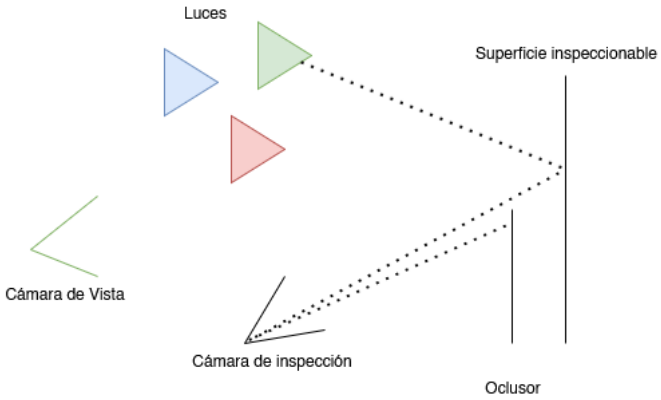


Figura 5.4: Diagrama mostrando el resultado esperado para ambos simuladores del sistema.

Prueba con vehículos

Tras la prueba base, se realizó una nueva prueba basada en un caso de uso realista. Esta prueba consiste en colocar en escena los distintos vehículos previamente introducidos y analizar la eficiencia del sistema en función del número de triángulos del modelo y la cantidad de luces que lo iluminan. Se ejecutarán tres pruebas con distintas disposiciones para evaluar el rendimiento del sistema en escenarios de complejidad variable:

- El coche no realista y 8 luces.
- El Volkswagen Touran y 32 luces.
- El Chevrolet Camaro y 256 luces.

Las luces se dispondrán en un aro alrededor del vehículo, ubicado en el plano de los ejes X e Y. En dicho aro, las luces estarán distribuidas de forma uniforme. El objetivo de esta disposición es observar el cambio gradual en la iluminación de los reflejos según la posición de cada luz.

Estas pruebas se realizarán con el objetivo de observar qué sistema funciona más rápidamente en cada caso, pero para ello se deberá tomar en cuenta el diferente funcionamiento de los simuladores. El simulador de *Vulkan* opera como se ha explicado, lanzando

rayos desde la cámara. Por otro lado, el simulador de *OpenGL* funciona mediante rasterización del modelo y, desde cada fragmento dibujado del modelo, trazado de rayos hacia las luces. Esto puede provocar una disparidad de tiempos en la carga de los modelos frente a el tiempo de trazado de rayos.

Esta prueba pretenderá también analizar estos casos para observar qué simulador funciona mejor en cada entorno. Cabe destacar que para esta prueba no se tomarán en cuenta oclusiones en el sistema, ya vinieran dadas por el modelo del vehículo o otros oclusores.

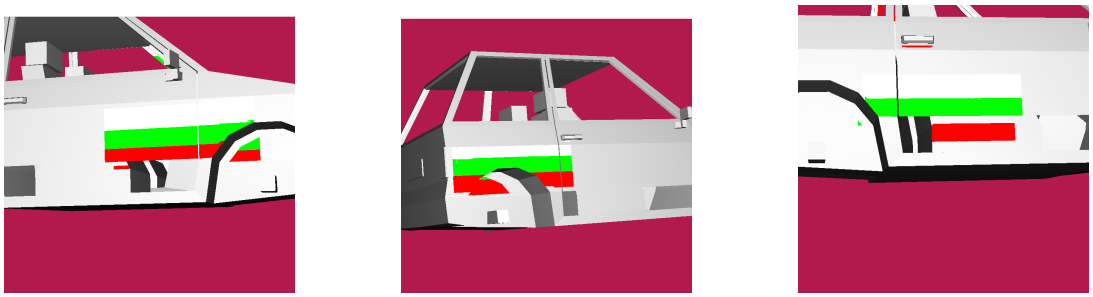
5.2 Pruebas

A continuación, se realizaron todas las pruebas comentadas y se presentarán los resultados.

Prueba de validación

Esta prueba de validación se centró en verificar el correcto funcionamiento de la librería en una escena simple, con la menor cantidad posible de variables, lo que facilita el procesamiento por parte del sistema. Tal como se muestra en la Figura 5.5, los resultados son claros: se observa la banda de luces situada en el lateral del vehículo, el cual presenta una geometría plana que permite visualizar los reflejos con precisión.

En general, esta prueba arroja resultados correctos y esperados, ya que fue diseñada como prueba de control para confirmar el funcionamiento adecuado de la librería.



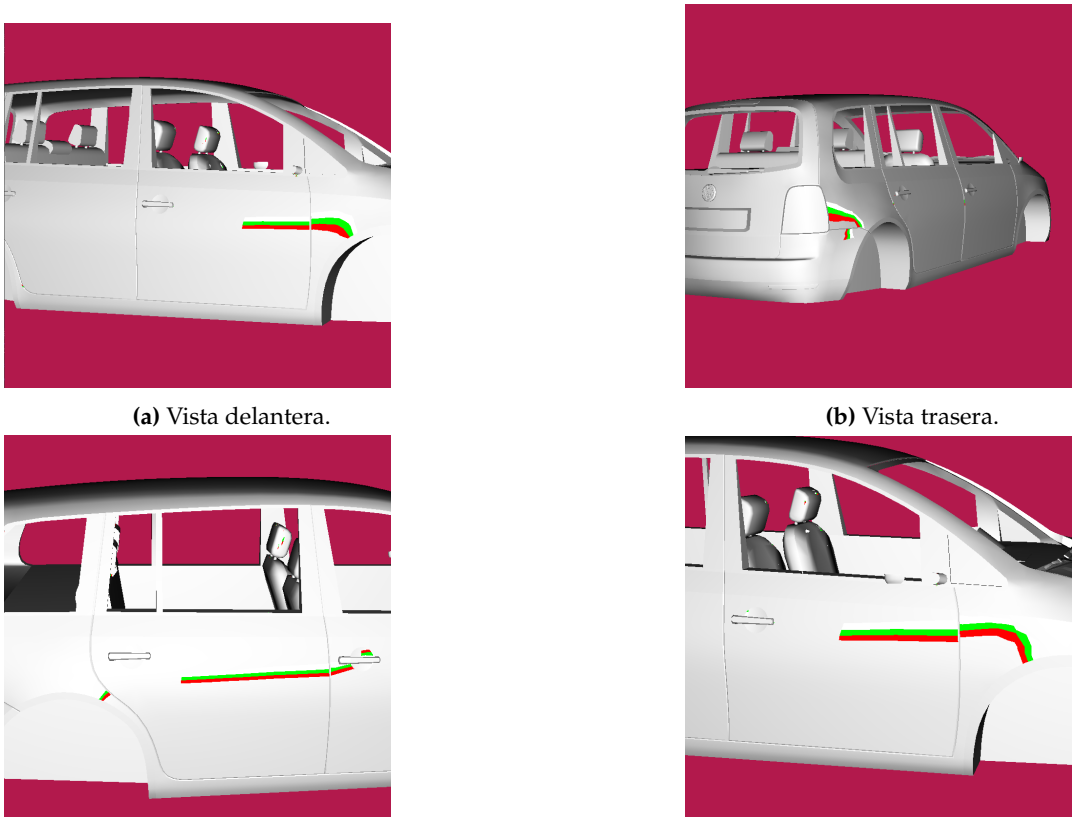
(a) Vista delantera. **(b)** Vista trasera. **(c)** Vista lateral.

Figura 5.5: Capturas desde diferentes ángulos del coche de la prueba.

Prueba 1

El objetivo de la prueba 1 fue evaluar la misma disposición utilizada en la prueba de validación, pero con un modelo de vehículo real, con el fin de comprobar que el comportamiento esperado no se debía únicamente a la simplicidad del modelo inicial. Para ello, se utilizó el modelo del Volkswagen Touran, que contiene aproximadamente diez veces más triángulos, lo que permite analizar el rendimiento del sistema en un entorno más complejo y realista.

Tal como se muestra en la Figura 5.6, las bandas de luz que en la prueba de validación eran anchas y definidas presentan ahora un comportamiento más realista, deformándose en función de la geometría del vehículo. También se observan otros efectos relevantes, como en la vista trasera, donde las dos superficies que reflejan la luz lo hacen en direcciones distintas, o en la zona del mango de la puerta, donde la forma provoca una inversión del reflejo.



(a) Vista delantera.

(b) Vista trasera.

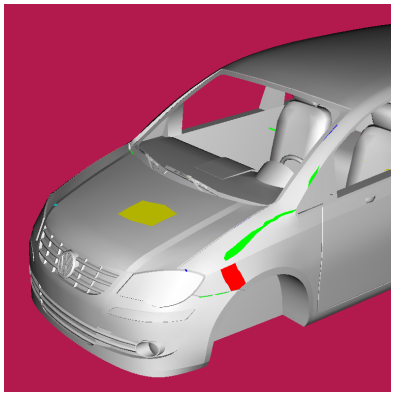
(c) Vista inferior.

(d) Vista superior.

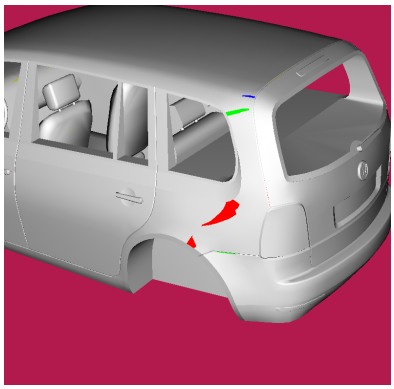
Figura 5.6: Capturas desde diferentes ángulos del coche de la prueba.

Prueba 2

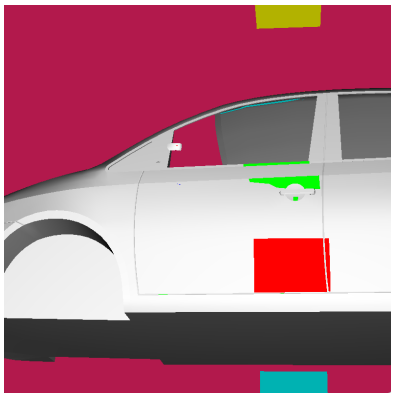
La prueba 2 tiene como objetivo someter la librería a un caso de uso realista, replicando la funcionalidad de un túnel de inspección mediante cinco luces que iluminan el vehículo. El modelo utilizado en esta prueba es nuevamente el Volkswagen Touran. Tal como se muestra en la Figura 5.7, los reflejos generados por las luces sobre el vehículo son relativamente planos, debido a la geometría del modelo. Además, se observa cómo cada luz incide de forma distinta sobre la superficie, produciendo reflejos diferenciados. En esta configuración no se aprecian rebotes secundarios.



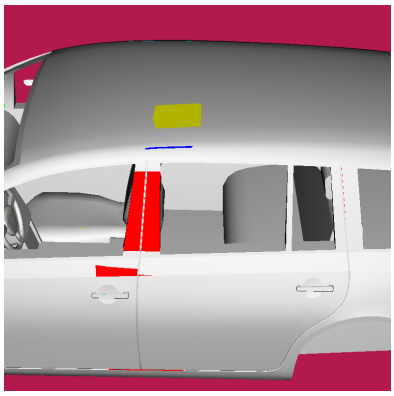
(a) Vista delantera.



(b) Vista trasera.



(c) Vista inferior.



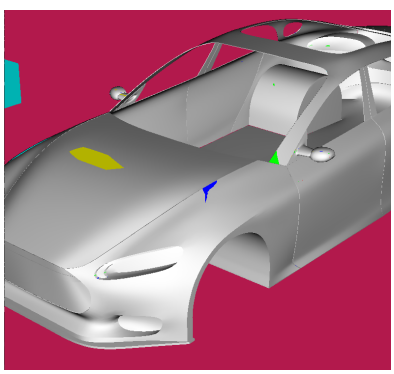
(d) Vista superior.

Figura 5.7: Capturas desde diferentes ángulos del coche de la prueba.

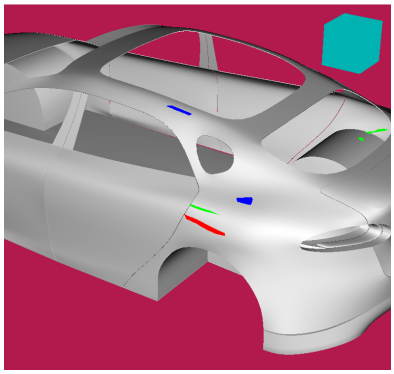
Prueba 3

La prueba 3 tiene como objetivo aumentar la complejidad respecto a la prueba 2, utilizando el modelo del Tesla, el cual contiene aproximadamente cuatro veces más triángulos y presenta superficies menos planas. Tal como se muestra en la Figura 5.8, los reflejos generados por el vehículo son más estrechos y pequeños, debido a su geometría.

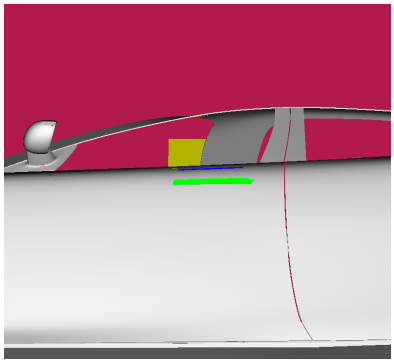
A pesar de ello, los reflejos se aprecian con claridad y de forma correcta, incluyendo la presencia de rebotes secundarios visibles desde la vista superior.



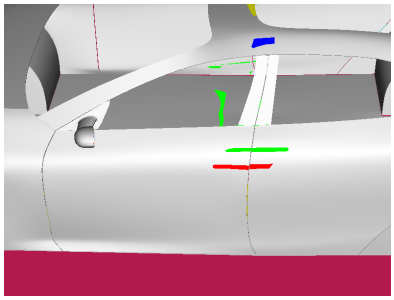
(a) Vista delantera.



(b) Vista trasera.



(c) Vista inferior.

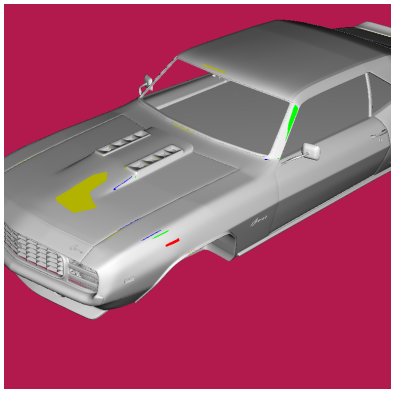


(d) Vista superior.

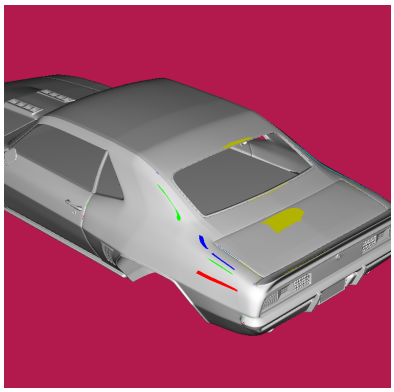
Figura 5.8: Capturas desde diferentes ángulos del coche de la prueba.

Prueba 4

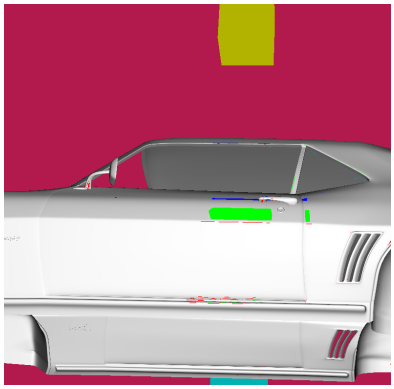
La prueba 4 tiene como objetivo llevar el sistema al límite, utilizando un modelo con aproximadamente cinco veces más triángulos. El modelo empleado es el Chevrolet Camaro 1969, seleccionado para exigir al sistema el máximo nivel de procesamiento. Tal como se muestra en la Figura 5.9, el sistema reproduce correctamente el comportamiento de los reflejos. Debido a la geometría más compleja del modelo, los reflejos presentan mayor irregularidad, aunque se generan conforme a lo esperado. También se observan reflejos secundarios, como en la parte inferior de la puerta.



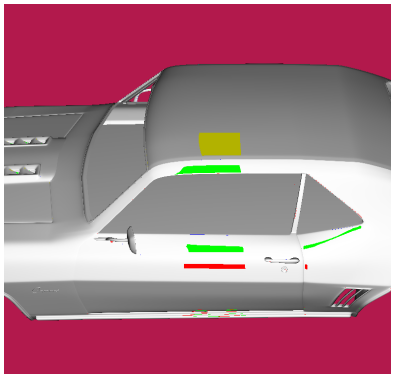
(a) Vista delantera.



(b) Vista trasera.



(c) Vista inferior.



(d) Vista superior.

Figura 5.9: Capturas desde diferentes ángulos del coche de la prueba.

Prueba de comparación

La prueba de comparación tiene como objetivo analizar las diferencias en tiempos de ejecución y precisión entre el simulador desarrollado en este trabajo y el sistema existente implementado en *OpenGL*. El procedimiento consiste en generar diez capturas de la misma configuración, observadas desde distintos puntos del espacio. Los resultados se muestran en la Figura 5.10. A la izquierda se presenta el resultado obtenido con *OpenGL* y a la derecha el correspondiente a *Vulkan*.

En ambos casos, desde diferentes ángulos, se aprecia claramente el efecto de las oclusiones, impidiendo la visualización completa de las luces. Las diferencias en los colores se deben únicamente a la configuración específica de cada simulador y no tienen relevancia en el análisis.

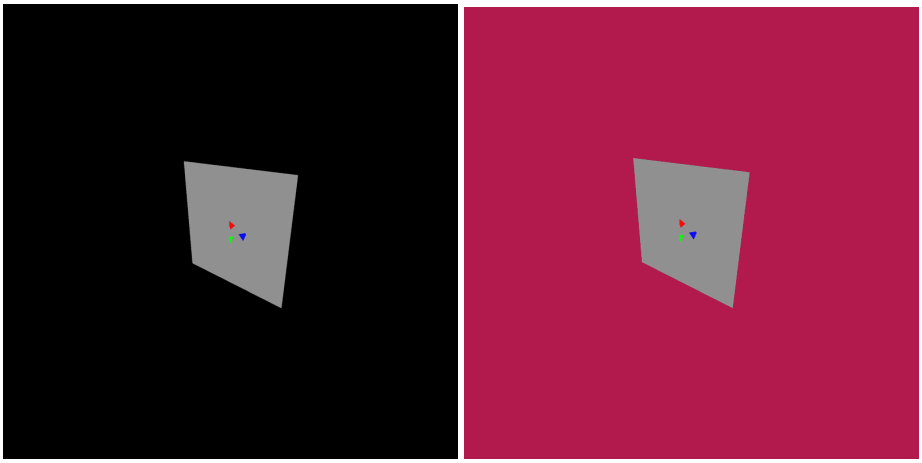


Figura 5.10: Captura tomada para ambos sistemas de la prueba, a la izquierda *OpenGL* y a la derecha *Vulkan*.

Prueba de comparación con vehículos

La prueba de comparación con vehículos tiene como objetivo analizar la eficiencia del sistema en casos de uso realistas y con distintos niveles de complejidad, centrándose principalmente en el tiempo de ejecución.

El procedimiento, similar al de la prueba anterior, consiste en capturar diez imágenes de una misma escena utilizando ambos *Renderers*, y calcular sobre los resultados los valores estadísticos de media, mediana y desviación estándar.

Los resultados se presentan en la Figura 5.11, donde se observa el reflejo generado por las distintas luces dispuestas en la escena. A medida que aumenta el número de luces, el cambio de color sobre la superficie del vehículo se vuelve más gradual, generando un efecto similar a un gradiente.

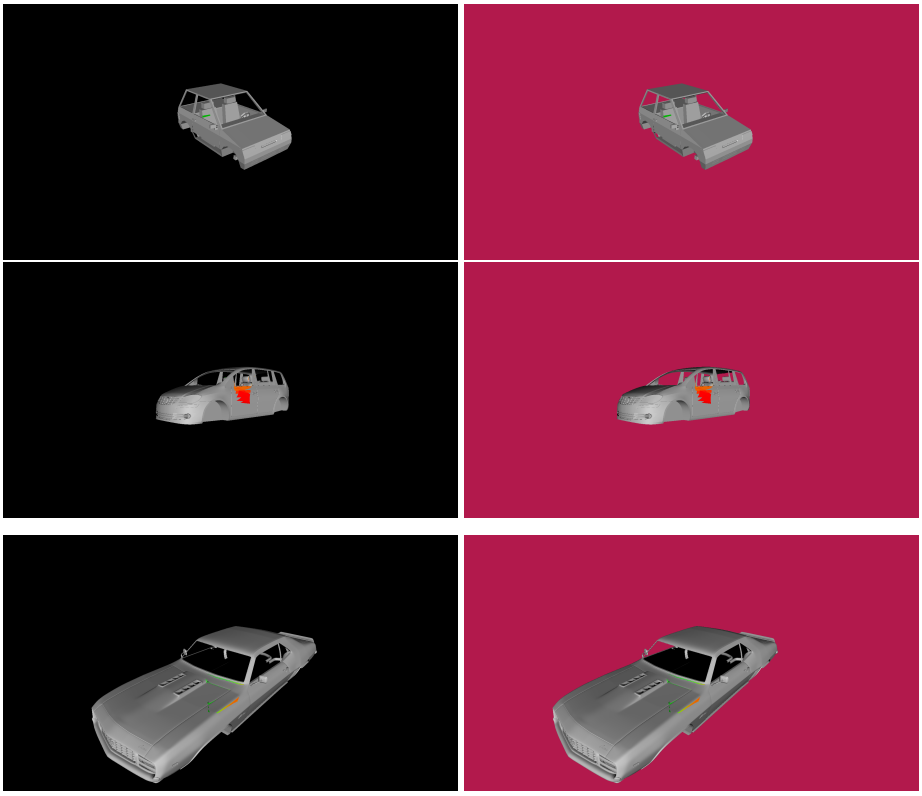


Figura 5.11: Captura tomada de la prueba para las 3 disposiciones explicadas.

5.3 Resultados

A continuación, los resultados obtenidos en las distintas pruebas serán analizados con el objetivo de determinar si han sido satisfactorios.

En cuanto a la corrección visual de los resultados, las figuras presentadas en la sección anterior muestran un comportamiento adecuado en la forma en que los distintos vehículos reflejan las luces. Se observan reflejos coherentes con la posición de las fuentes de luz, así como rebotes secundarios cuando corresponden. Además, se aprecia que, a mayor complejidad en la malla del modelo, los reflejos son más definidos y específicos, lo cual se atribuye a la reducción de superficies planas en geometrías más detalladas.

A continuación, se presentan los resultados numéricos en forma de tabla, seguidos de un análisis reflexivo sobre los mismos.

Pruebas 2, 3 y 4

T carga/T ejecución (s)	Copia bytes	Compartir textura
Volkswagen Touran	0.0193, 0.0279	0.0193, 0.0124
Tesla	0.0285, 0.0279	0.0285, 0.0124
Chevrolet Camaro	0.0252, 0.0282	0.0252, 0.0124

La tabla presentada contiene diversos resultados relevantes. Para obtener los tiempos de ejecución, se tomaron cinco muestras, excluyendo la primera ejecución debido a su mayor duración, y se calculó la media de los valores restantes.

En primer lugar, se observa que el tiempo de ejecución dentro de una misma técnica permanece constante para todos los modelos. Esto se debe a que se lanza un número fijo de rayos determinado por la resolución de pantalla, por lo que el tiempo de ejecución varía muy poco, independientemente de la escena.

Por otro lado, el tiempo de carga de los modelos —incluyendo su conversión a estructuras de aceleración— no guarda una relación directa con el número de triángulos. Esto se evidencia al comparar el modelo del Tesla, que presenta mayor tiempo de carga que el Chevrolet Camaro, a pesar de tener menos triángulos. Este comportamiento puede explicarse por el número de partes independientes que componen cada modelo. Un mayor número de partes implica la creación de más estructuras de aceleración, lo que incrementa el tiempo de carga.

Asimismo, se observa que evitar la transferencia de datos a memoria para realizar la copia de bytes mejora significativamente el rendimiento. El sistema pasa de aproximadamente 36 FPS con copia de bytes a 80 FPS utilizando compartición de textura. Es importante destacar que estos resultados tienen como único propósito comparar ambos métodos y no representan la eficiencia real del sistema, ya que el proceso utilizado puede optimizarse considerablemente. El objetivo de estas pruebas es evaluar cuál de los métodos ofrece mejor rendimiento en un caso de uso concreto.

Prueba de comparación

Los resultados de la prueba de comparación se obtuvieron a partir de nueve muestras de un caso de uso real. Originalmente se tomaron diez muestras, pero la primera fue descartada por no ser representativa, ya que implicaba la inicialización completa del sis-

tema. Los datos obtenidos fueron analizados y organizados en una gráfica para facilitar su interpretación.

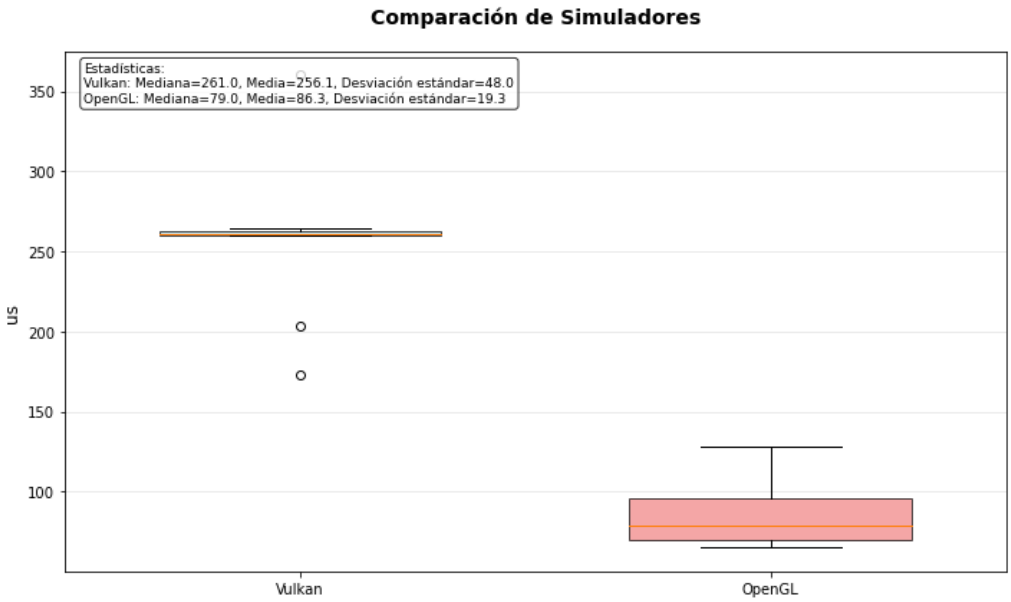


Figura 5.12: Gráfica mostrando los resultados de la prueba.

Como se muestra en la Figura 5.12, se observa que el simulador desarrollado en *OpenGL* presenta un tiempo de ejecución inferior. La mediana para este simulador es de 79 microsegundos, frente a los 261 microsegundos del simulador basado en *Vulkan*. Además, la desviación estándar es mayor en el segundo caso, lo que se atribuye a la presencia de muestras con valores significativamente más bajos en comparación con el resto.

Prueba de comparación con vehículos

Similarmente a la prueba anterior, se tomaron en total 9 muestras, al descartar la primera. Estos datos se tomaron tanto para el tiempo de dibujado inicial previamente a el cálculo de los rebotes como para el procesado posterior. Los datos fueron posteriormente analizados y los resultados son los siguientes:

En la Figura 5.13 se observa que el simulador desarrollado en *OpenGL* presenta un rendimiento superior en tiempo de ejecución para los tres casos evaluados, tanto en tiempo de carga como en tiempo de ejecución. Aún este dato, en tiempo de ejecución de rebotes la diferencia en tiempos no es tan grande como en carga, habiendo casos de frames donde el simulador de *Vulkan* funciona más rápidamente que el de *OpenGl*. Para facilitar la interpretación de estos resultados, se agruparon en una única tabla.

En la Figura 5.14 se presentan los resultados consolidados de la prueba. El primer aspecto destacable es que el simulador en *OpenGL* muestra mayor eficiencia en todas sus configuraciones. Por otro lado, se observa que, en el caso del simulador en *Vulkan*, el tiempo medio de procesamiento aumenta conforme se incrementa la complejidad de la prueba. Sin embargo, en el simulador de *OpenGL*, la prueba con el modelo del Volkswagen Touran presenta una media inferior a la obtenida con el modelo no realista. Como era esperable, la media correspondiente al Chevrolet Camaro es superior, aunque no de forma significativa.

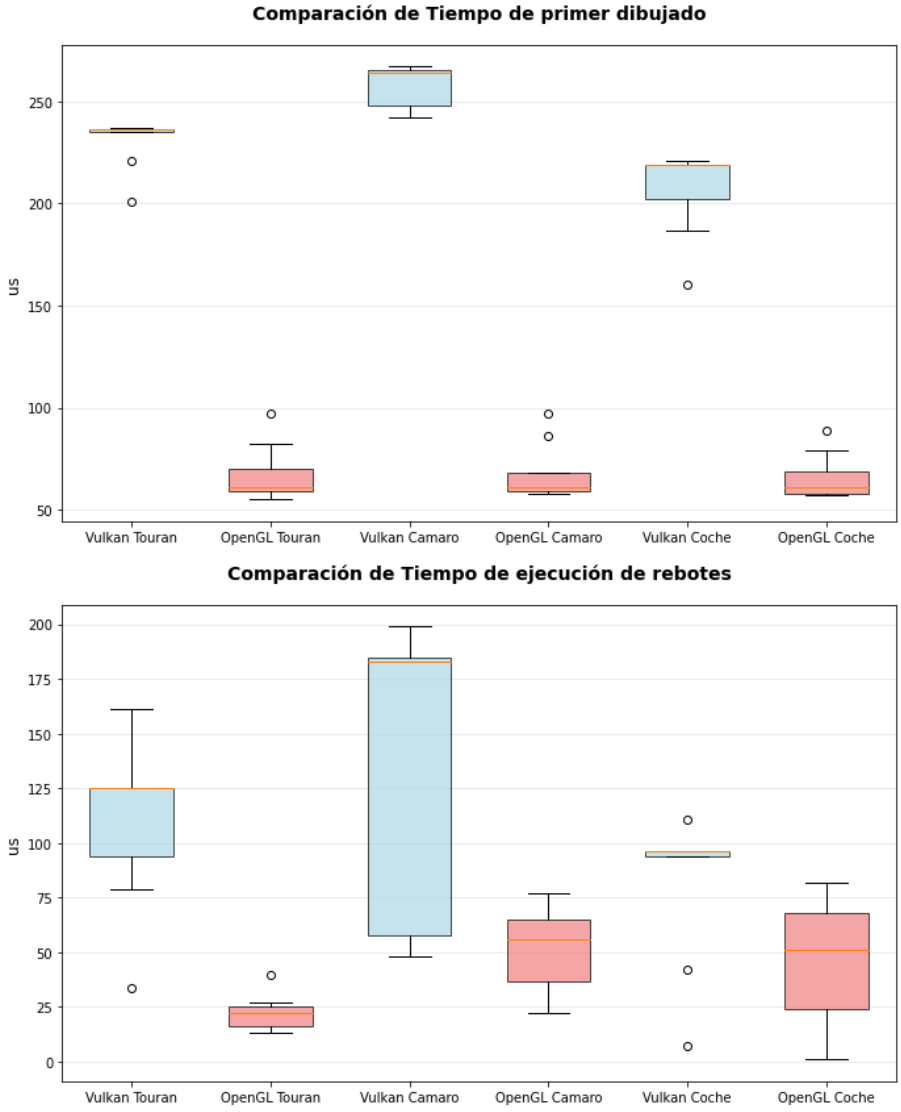


Figura 5.13: De arriba a abajo, comparaciones de esta prueba para el tiempo de carga y de rebotes.

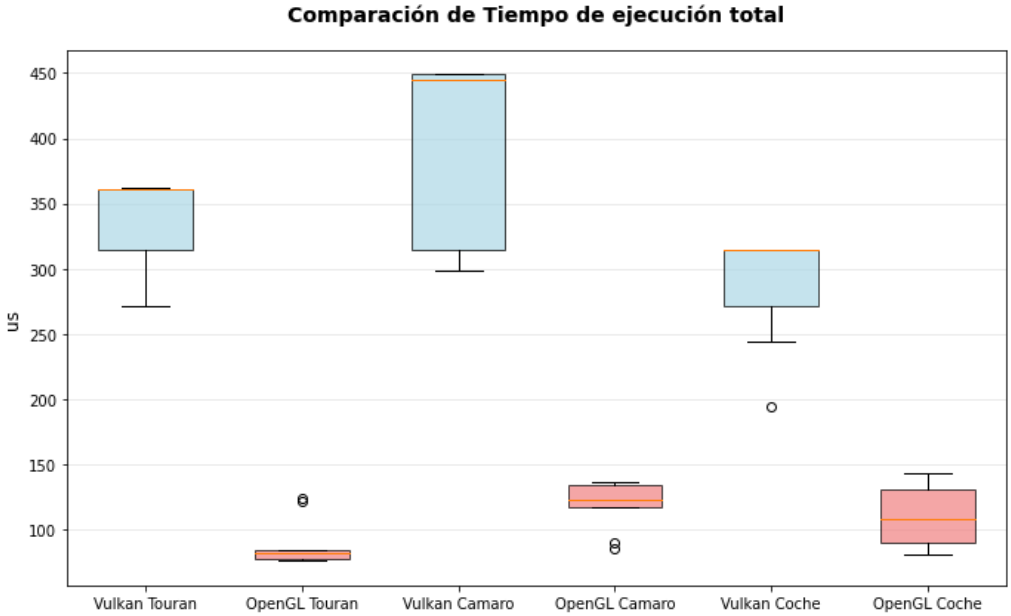


Figura 5.14: Comparación de los tres casos de prueba para ambos simuladores.

Resultados pruebas de comparación

El análisis de los resultados obtenidos en esta prueba permite concretar las siguientes observaciones:

El hecho de que el simulador en *OpenGL* presente un menor tiempo medio de ejecución para el modelo del Volkswagen Touran en comparación con el coche no realista sugiere que el número de triángulos no es el factor más determinante. En escenarios con menor complejidad geométrica, la cantidad de rebotes de rayos parece tener mayor influencia. Esto se confirma al observar que el modelo del Chevrolet Camaro, con mayor número de triángulos, presenta un tiempo medio de ejecución más elevado, como era esperable. A su vez, en la Figura 5.11 se puede observar como en la imagen para el Volkswagen Touran, el vehículo aparece ligeramente más pequeño que el coche no realista, de tal manera que hay menos fragmentos ocupados por el mismo y, por tanto, menos lugares desde que lanzar rayos.

Se puede observar claramente en las gráficas que para el tiempo de carga del modelo inicial, el simulador de *OpenGL* es notablemente más rápido, dado a que rasterizar el modelo es un proceso más rápido que el trazado de rayos. Por otro lado, la diferencia en tiempo de ejecución de los rebotes en la escena es menor, habiendo casos en los que el simulador *Vulkan* es más rápido a pesar de ser más lento de media. Esto presenta el potencial de este simulador de ser más adecuado al problema si este presentara ciertas optimizaciones ayudando a reducir este tiempo de rebotes.

Existen diversas razones por las cuales el simulador desarrollado en *OpenGL* muestra un rendimiento superior al de *Vulkan*. Las principales son:

- El simulador en *Vulkan* no incorpora todas las optimizaciones posibles.
- El tiempo de desarrollo del simulador en *Vulkan* ha sido considerablemente menor en comparación con el de *OpenGL*.
- El simulador en *Vulkan* fue desarrollado por un usuario con menor experiencia en el campo, frente a desarrolladores más experimentados en *OpenGL*.

Entre las optimizaciones pendientes en el simulador de *Vulkan* se encuentran un uso más eficiente de los recursos y una distribución más equilibrada del trabajo entre las distintas *Queues*. En condiciones ideales, el uso de 16 *Queues* podría permitir una mejora teórica de hasta 16 veces en el tiempo de ejecución.

En términos generales, se concluye que el simulador desarrollado en *Vulkan* posee un alto potencial para superar en eficiencia al de *OpenGL*, siempre que se implementen las optimizaciones necesarias. No obstante, esto requeriría una inversión adicional de tiempo y esfuerzo en su desarrollo.

CAPÍTULO 6
Conclusiones

A lo largo del desarrollo de este trabajo donde se ha creado un simulador se han completado los siguientes objetivos:

- Se ha diseñado y desarrollado un simulador que permite el uso de trazado de rayos para detectar defectos en coches.
- Se ha creado una librería a partir de este simulador que permita a un usuario externo utilizar sus funcionalidades.
- Se han realizado una serie de pruebas para verificar la funcionalidad del simulador desarrollado
- Se han comparado los resultados de estas pruebas frente a los de un simulador existente.

Es posible concretar que se han completado los objetivos planteados al comienzo de este trabajo en la sección de objetivos de manera satisfactoria. Para completar estos ha sido necesario investigar sobre técnicas de detección de defectos en vehículos y trazado de rayos. Adicionalmente, ha sido necesario el aprendizaje de utilizar diferentes librerías y APIs, principalmente *Vulkan* para poder crear toda la escena en que ocurrirá el trazado de rayos y el propio trazado de rayos. A su vez, se han utilizado diferentes librerías para simplificar y facilitar el proceso de desarrollo, como pueden ser *glslang* para manejar los shaders o glm para facilitar las operaciones matemáticas con vectores y matrices.

El desarrollo del simulador ha conllevado crear 3 partes diferentes que interactuaran entre estas, un núcleo de *Vulkan* que se encarga de completar todas las funcionalidades *Vulkan*, un núcleo de trazado de rayos que se encarga de configurar el propio trazado de rayos y un renderizador que se encarga de poner a disposición del usuario las funcionalidades de los otros núcleos. El desarrollo de estas partes ha conllevado tomar una serie de decisiones acotadas al problema en cuestión, como fue decidir qué parte debe hacer determinada tarea o dónde se deben hacer operaciones, en CPU o GPU.

En general, durante el desarrollo de este proyecto se han aprendido diferentes cuestiones centradas en diferentes dimensiones. Por un lado, de organización y metodología, con conceptos como ser organizado y consistente, preparar el trabajo antes de comenzarlo y planear en el tiempo. Por otro lado, se aprendió mucho sobre el problema de la detección de defectos y, sobre todo, del uso de la API de *Vulkan*, siendo esta la herramienta central al desarrollo del TFM.

6.1 Relación con los estudios cursados

El trabajo desarrollado ha tomado conocimientos previamente mostrados en diferentes asignaturas cursadas. Principalmente, se tomaron conceptos estudiados en la asignatura de Programación gráfica, siendo esta la asignatura donde se enseña gran parte del funcionamiento de APIs de programación gráfica como *OpenGL*, el manejo de diferentes objetos relevantes con estos como mallas o shaders y de diferentes componentes compartidos por las APIs como los búferes para dichas mallas. También es cierto que *Vulkan* aparece como sucesor de *OpenGL* y por tanto toma muchas partes existentes de este y se podría considerar el siguiente paso lógico para continuar con el aprendizaje de programación gráfica. A su vez, también se aplicaron conocimientos obtenidos en la asignatura de gráficos por computador, la cuál cubre principalmente el uso de *three.js*, otra API gráfica, y mucho conocimiento teórico de como debe estar organizada la aplicación gráfica, sobretodo en las partes más matemáticas, como son el manejo de vértices y transformaciones en el espacio 3D.

6.2 Trabajo Futuro

Habiendo cumplido satisfactoriamente los objetivos designados para el TFM, igualmente existen campos en los que este puede ser expandido o mejorado, si bien surjan de limitaciones personales o temporales.

Por un lado, un aspecto a mejorar evidente en el simulador es que la inexperiencia con la API de *Vulkan* ha causado que parte de la arquitectura subyacente desarrollada no esté optimizada completamente y haga operaciones redundantes o que no son apropiadas. Un ejemplo es la creación de búferes para las mallas, donde es mucho más eficiente crear un búfer con todos los atributos unificados en lugar de diferentes búferes fragmentados para cada atributo. En este campo, también el manejo de memoria en el simulador no es el más eficiente, en parte debido a estos búferes fragmentados y en parte debido a no poseer un entendimiento profundo de los diferentes usos de memoria para poder dar uno correcto adecuado a la situación específica.

Por otro lado, se podría mejorar la funcionalidad de todo el simulador investigando cuáles son las técnicas más avanzadas y estandarizadas en el uso extenso de esta API para así entender y poder obtener el mayor rendimiento de las diferentes partes del sistema. Un ejemplo de esto es el sistema de manejo de *queues*, que se podría optimizar para repartir la carga de trabajo más adecuadamente.

En cuanto a funcionalidades, podrían ser extendidas estas para mejorar las funcionalidades del simulador. Principalmente podrían ser integradas 2 funcionalidades que son el manejo de transformaciones a los modelos en tiempo real, esto es moverlos, rotarlos, etc dentro de la escena si tener que recargarlos y recrear estructuras de aceleración, y el soporte de texturas para estos modelos. A su vez se podría añadir una interfaz para poder modificar diferentes aspectos y variables en el sistema de manera interactiva.

Bibliografía

[1] J M Parker y Z Hou. «A numerical investigation of diffuse images for effective defect detection». en. En: *Proceedings of the Institution of Mechanical Engineers, Part B: Journal of Engineering Manufacture* 216.7 (jul. de 2002), págs. 1073-1079. ISSN: 0954-4054, 2041-2975. DOI: 10.1243/09544050260174274. URL: https://journals.sagepub.com/doi/10.1243/09544050260174274 (visitado 30-06-2025).

[2] Timothy J. Purcell et al. «Ray tracing on programmable graphics hardware». en. En: *Proceedings of the 29th annual conference on Computer graphics and interactive techniques*. San Antonio Texas: ACM, jul. de 2002, págs. 703-712. ISBN: 9781581135213. DOI: 10.1145/566570.566640. URL: https://dl.acm.org/doi/10.1145/566570.566640 (visitado 30-06-2025).

[3] Graham Sellers. *Vulkan programming guide: the official guide to learning Vulkan*. OpenGL series. Boston: Addison-Wesley, 2017. ISBN: 9780134464541.

[4] Yangdong Deng et al. «Toward Real-Time Ray Tracing: A Survey on Hardware Acceleration and Microarchitecture Techniques». en. En: *ACM Computing Surveys* 50.4 (jul. de 2018), págs. 1-41. ISSN: 0360-0300, 1557-7341. DOI: 10.1145/3104067. URL: https://dl.acm.org/doi/10.1145/3104067 (visitado 30-06-2025).

[5] Fei Chang et al. «A mobile vision inspection system for tiny defect detection on smooth car-body surfaces based on deep ensemble learning». En: *Measurement Science and Technology* 30.12 (dic. de 2019), pág. 125905. ISSN: 0957-0233, 1361-6501. DOI: 10.1088/1361-6501/ab1467. URL: https://iopscience.iop.org/article/10.1088/1361-6501/ab1467 (visitado 30-06-2025).

[6] Qinbang Zhou et al. «An Automatic Surface Defect Inspection System for Automobiles Using Machine Vision Methods». en. En: *Sensors* 19.3 (feb. de 2019), pág. 644. ISSN: 1424-8220. DOI: 10.3390/s19030644. URL: https://www.mdpi.com/1424-8220/19/3/644 (visitado 30-06-2025).

[7] Jing Zhang et al. «Defect inspection of coated automobile roofs using a single camera». en. En: *IEEJ Transactions on Electrical and Electronic Engineering* 15.4 (abr. de 2020), págs. 616-625. ISSN: 1931-4973, 1931-4981. DOI: 10.1002/tee.23096. URL: https://onlinelibrary.wiley.com/doi/10.1002/tee.23096 (visitado 30-06-2025).

[8] Adam Marrs. *Ray Tracing Gems II: Next Generation Real-Time Rendering with DXR, Vulkan, and OptiX*. eng. Ed. por Peter Shirley e Ingo Wald. Erscheinungsort nicht ermittelbar: Springer Nature, 2021. ISBN: 9781484271841.

[9] Xiaoqing Zheng et al. «Recent advances in surface defect inspection of industrial products using deep learning techniques». en. En: *The International Journal of Advanced Manufacturing Technology* 113.1-2 (mar. de 2021), págs. 35-58. ISSN: 0268-3768, 1433-3015. DOI: 10.1007/s00170-021-06592-8. URL: http://link.springer.com/10.1007/s00170-021-06592-8 (visitado 30-06-2025).

APÉNDICE A
Objetivos de desarrollo sostenible

En cuanto a los objetivos de desarrollo sostenible ODS cabe destacar que este trabajo se centra y aporta en los siguientes:

Principalmente el trabajo se centra en el objetivo de Industria, innovación e infraestructuras, al pretender mejorar una herramienta de la industria e innovar en este proceso de tal manera que pueda ser mejorado.

A su vez, también implica el objetivo de Trabajo decente y crecimiento económico, al tratar de mejorar un sistema existente en un entorno real empresarial para mejorar el trabajo con dicha y, por tanto, poder aportar cierto crecimiento a la propia empresa.

Finalmente, también se centra en cierta manera con el objetivo de Producción y consumo responsables al pretender ahorrar recursos al utilizar el sistema desarrollado, de tal manera que estos recursos ahorrados puedan ser utilizados en otros ámbitos o simplemente no usados, reduciendo el consumo.

A continuación el grado de relación del trabajo con los objetivos de desarrollo sostenible en una tabla:

Objetivos de Desarrollo Sostenibles	Alto	Medio	Bajo	No Procede
ODS 1. Fin de la pobreza.				X
ODS 2. Hambre cero.				X
ODS 3. Salud y bienestar.				X
ODS 4. Educación de calidad.				X
ODS 5. Igualdad de género.				X
ODS 6. Agua limpia y saneamiento.				X
ODS 7. Energía asequible y no contaminante.				X
ODS 8. Trabajo decente y crecimiento económico.		X		
ODS 9. Industria, innovación e infraestructuras.	X			
ODS 10. Reducción de las desigualdades				X
ODS 11. Ciudades y comunidades sostenibles				X
ODS 12. Producción y consumo responsables.			X	
ODS 13. Acción por el clima.				X
ODS 14. Vida submarina.				X
ODS 15. Vida de ecosistemas terrestres.				X
ODS 16. Paz, justicia e instituciones sólidas.				X
ODS 17. Alianzas para lograr objetivos				X